[Thesis title]

Author: Adria Espinoza

Defence date: [Defence date]

Director: [Director's name]
[Department / institution / company]

Degree: Master's Degree in Data Science

FACULTAT D'INFORMÀTICA DE BARCELONA (FIB)

UNIVERSITAT POLITÈCNICA DE CATALUNYA (UPC) – BarcelonaTech

Abstract

A publication of constructive news has to find, among many sources, the few stories worth offering on the topics it covers, and check their claims before it publishes them. This thesis builds and evaluates a low-cost, end-to-end system that does both, in English and Spanish, with open-weight models running locally and no paid service. It reads the 33 enabled sources of a 37-source catalogue for up to three topic groups, screens out what the publication never covers, ranks the candidates and lets an editor, a language model or both choose at most twenty to analyse. For each chosen article it extracts and enriches the text, admits or rejects it, selects two to four checkable claims, searches the open web for evidence through a self-hosted metasearch engine, ranks the evidence with a gate that cuts off-target sources, and asks a local language model for one of five verdicts that must cite and quote its sources; the verdict's confidence is then recomputed from the evidence, ungrounded verdicts are downgraded, and failed searches and unreachable models are reported apart from verdicts.

The evaluation is the contribution. It is organised around four research questions — finding the news, verifying it, how close a model comes to a person, and how much of a journalist's work the joined system takes on — and measures each stage against a person wherever a person takes the same decision. It uses an evaluation harness that reports every metric with a bootstrap interval and its provenance; X-Fact as an external, open-retrieval benchmark; and a custom validation set of 139 claims from constructive news (109 natural, 30 synthetic refuted claims built by substitution), the first of its kind to the author's knowledge.

On a 64-claim X-Fact pilot the zero-shot 3-billion-parameter verifier reached 29.7% accuracy [18.8, 40.6] with a Cohen's κ against gold indistinguishable from zero, and 19.4% [13.3, 25.5] on the 165 Spanish claims of X-Fact's in-domain test set; retrieved evidence added 10.9 points [3.1, 20.3] over the model alone on the pilot. The pilot also showed how much of an open-retrieval score on an old benchmark measures time rather than verification: most decidable claims cited evidence newer than the claim, and the fact-checker's own verdict was retrieved for over a third of the English ones. Before the verdict, the ranked rounds separated stories worth offering from the rest with an AUC of 0.78 [0.67, 0.87], and the claim selector's proposals were checkable in 64% of cases.

[To write: the custom-set results for RQ2–RQ4 with their intervals, the agreement between the AI labelling pass and the annotator, and the limitations that remain; then rewrite this abstract to one or two pages.]

Keywords: automated fact-checking, open-domain retrieval, large language models, constructive journalism, multilingual NLP, evaluation.

Acknowledgements

[To write: optional; only the author can write it.]

Contents

Abstract i

Acknowledgements iii

Acronyms xii

1 Introduction 1
- 1.1 Context . . . 1
- 1.2 Motivation . . . 2
- 1.3 Problem Statement . . . 2
- 1.4 Objectives . . . 3
  - 1.4.1 General Objective . . . 3
  - 1.4.2 Specific Objectives . . . 3
- 1.5 Research Questions . . . 4
- 1.6 Scope . . . 4
  - 1.6.1 In Scope . . . 4
  - 1.6.2 Out of Scope . . . 5
- 1.7 Structure of the Document . . . 6

2 State of the Art 7
- 2.1 Automated Fact-Checking . . . 7
  - 2.1.1 The Fact-Checking Pipeline . . . 7
  - 2.1.2 Closed- and Open-Retrieval Verification . . . 8
  - 2.1.3 Language Models as Verifiers . . . 8
- 2.2 Benchmarks for Fact Verification . . . 9
- 2.3 Constructive and Solutions Journalism . . . 10
- 2.4 News Discovery, Selection and Recommendation . . . 10
- 2.5 Existing Tools and Services . . . 11
- 2.6 Positioning of This Work . . . 12

3 Project Management 13
- 3.1 Methodology . . . 13
  - 3.1.1 Iterative Development in Sprints . . . 13
  - 3.1.2 Parallel Tracks . . . 13

3.1.3 Definition of Done and Status Tracking . . . 13
3.1.4 Decision Records and Replanning . . . 14
3.1.5 Tools . . . 14
3.2 Planning . . . 14
3.2.1 Workload . . . 14
3.2.2 Phases and Tasks . . . 14
3.2.3 Gantt Chart . . . 16
3.2.4 Milestones . . . 18
3.3 Deviations from the Initial Plan . . . 18
3.4 Risk Management . . . 20
3.5 Budget . . . 22
3.5.1 Personnel Costs . . . 22
3.5.2 Hardware and Software Costs . . . 23
3.5.3 Infrastructure Costs . . . 23
3.5.4 Contingency and Total . . . 24

4 Proposed Method: Specification and Design 25
4.1 Requirements . . . 25
4.1.1 Functional Requirements . . . 25
4.1.2 Non-Functional Requirements . . . 26
4.2 System Architecture . . . 26
4.3 Data Architecture . . . 27
4.4 Candidate Selection . . . 28
4.5 Enrichment . . . 30
4.6 The Fact-Checking Pipeline . . . 31
4.6.1 Admission Filter . . . 32
4.6.2 Claim Selection . . . 32
4.6.3 Evidence Retrieval . . . 33
4.6.4 Evidence Ranking and the Pertinence Gate . . . 34
4.6.5 Verification with a Language Model . . . 34
4.6.6 Confidence Recalibration . . . 35
4.6.7 The Second Look . . . 36
4.6.8 Article-Level Aggregation . . . 37
4.7 Concurrency Design . . . 37
4.8 Interface Design . . . 38

5 Development 40
5.1 Technology Stack . . . 40
5.2 News Acquisition: Discovery and Extraction . . . 41
5.2.1 Overview . . . 41
5.2.2 The Source Catalogue . . . 42
5.2.3 The Shared Fetcher and the URL Guard . . . 43
5.2.4 Discovery Strategies . . . 45

5.2.5 Discovery by Topic Group . . . 46
5.2.6 What Discovery Keeps . . . 47
5.2.7 The Extraction Cascade . . . 49
5.2.8 Metadata and Publication Time . . . 51
5.2.9 Reading Evidence Pages . . . 51
5.2.10 Monitoring the Sources . . . 52
5.2.11 Verification Against the Live Sources . . . 53
5.2.12 Testing . . . 54
5.2.13 Limitations . . . 54
5.3 The Inference Service . . . 55
5.4 Fact-Checking Implementation . . . 57
5.5 Storage, Journal and Per-Run Thresholds . . . 58
5.6 Frontend . . . 59
5.7 The Labelling Tool . . . 60
5.8 Deployment . . . 61
5.9 Testing and Quality Assurance . . . 62
5.10 Writing, Reviewing and Publishing an Article . . . 64

6 Experimental Design 66
6.1 Research Questions and Metrics . . . 66
6.2 Selection and Justification of the Evaluation Dataset . . . 67
6.2.1 The Closed-Retrieval Paradigm: FEVER . . . 68
6.2.2 Why FEVER Is Not Suitable for This Work . . . 68
6.2.3 X-Fact as an Open-Retrieval Benchmark . . . 69
6.2.4 Domain Limitation and a Custom Validation Set . . . 70
6.3 Construction of the Custom Validation Set . . . 70
6.3.1 Size and Stratification . . . 71
6.3.2 Record Format . . . 72
6.3.3 Label Vocabulary . . . 73
6.3.4 Annotation Procedure . . . 73
6.3.5 Tie-Break Rules . . . 74
6.3.6 Verification of the Labels . . . 75
6.3.7 Synthetic Refuted Claims . . . 76
6.3.8 The Tuning and Test Halves . . . 77
6.3.9 Limitations . . . 79
6.4 Evaluation Harness . . . 79
6.4.1 Record Format and Run Layout . . . 80
6.4.2 Metrics and Confidence Intervals . . . 80
6.4.3 Retrieval Evaluation . . . 81
6.4.4 Stage Attribution . . . 82
6.4.5 Journalist-Effort Metrics . . . 83
6.4.6 The Tuning Loop . . . 83
6.4.7 Run-to-Run Consistency . . . 84

6.5 Experiments on the Stages Before the Verdict . . . 84
6.6 Model Benchmarks . . . 85
6.6.1 Verification Models . . . 85
6.6.2 Writing Models . . . 86
6.7 Experimental Setup . . . 87

7 Results **89**
7.1 Pilot on X-Fact . . . 89
7.1.1 What Was Run . . . 89
7.1.2 Results . . . 90
7.1.3 Beside the X-Fact Baselines . . . 91
7.1.4 What the Years Between Claim and Search Do . . . 93
7.2 RQ1 — Finding Constructive News on Chosen Topics . . . 95
7.3 RQ2 — Verifying the Claims . . . 96
7.4 RQ3 — How Close a Model Comes to a Person . . . 99
7.5 RQ4 — How Much of the Work the System Takes On . . . 100
7.6 Model Comparison . . . 101
7.6.1 Verification Models . . . 101
7.6.2 Writing Models . . . 102
7.6.3 Cost and Response Time . . . 103
7.7 The Stages Before the Verdict . . . 104
7.8 Discussion . . . 105
7.9 Threats to Validity . . . 106

8 Sustainability and Ethical Implications **108**
8.1 Sustainability Report . . . 108
8.1.1 Environmental Dimension . . . 108
8.1.2 Economic Dimension . . . 109
8.1.3 Social Dimension . . . 109
8.2 Ethics of Data Management and Analysis . . . 109
8.2.1 Collection of Third-Party Content . . . 109
8.2.2 Personal Data and Privacy . . . 110
8.2.3 Responsibility for Automated Verdicts . . . 111
8.3 Equity and Bias Analysis . . . 111
8.3.1 Biases in the Data . . . 111
8.3.2 Biases in the Processes . . . 112
8.3.3 Actions Taken to Reduce Them . . . 112
8.3.4 Critical Analysis of the Impact . . . 113
8.3.5 Inclusive Language, References and Examples . . . 113
8.4 Alignment with the Sustainable Development Goals . . . 114

9 Conclusions **115**
9.1 Answers to the Research Questions . . . 115

9.2 Achievement of the Objectives 116
9.3 Limitations 116
9.4 Future Work 117
9.4.1 An API-Keyed Search Engine 117
9.4.2 Further Work 119
9.5 Final Remarks 119

References **120**

A Detailed Planning **125**

B News Source Catalogue **128**

C Annotation Guide **130**

D Reproducibility **133**

List of Figures

3.1 Gantt chart of the plan of 30 September 2026 (docs/roadmap.md), as it stood on 6 October (dashed line). From 14 October the thesis priorities of Table 3.3 replace its calendar where the two disagree; work done ahead of these dates is listed in Section 3.3. 17

4.1 The services and how they talk. Every arrow from the backend to an external service passes through that service's concurrency ceiling (Section 4.7). 27
4.2 The graph schema. Every relationship carries `method` (`pipeline` or `manual`). 28
4.3 The seven stages. Stages 2–6 run per claim, and claims run concurrently; each claim records the furthest stage it reached. The dashed boxes are outcomes reported apart from the verdicts. 31

5.1 The acquisition layer. Solid arrows are the normal path; dashed arrows are escalations, taken only when the previous step could not help. A missing page, a timeout or a guard refusal stops the extraction at once. 42

List of Tables

2.1 Benchmarks for fact verification relevant to this work. 9
2.2 The work against the closest benchmarks and tools. "n/a": a dataset, not a system. 12

3.1 Phases, dates and planned hours (docs/roadmap.md, rebalanced on 30 September 2026). 14
3.2 Hours per phase after each replanning (docs/roadmap.md). The earlier columns are regrouped into the current phases so that the three add up the same way. 18
3.3 Thesis priorities from 14 October 2026 (docs/roadmap.md). Where they disagree with the calendar of Section 3.2, they win. 20
3.4 Risks observed during the project, with their mitigation. 21
3.5 Planned hours per role, from Table 3.1. 22

4.1 Functional requirements. 25
4.2 Non-functional requirements and how each is met. 26

5.1 The technology stack and why each piece was chosen. 40
5.2 Fields of a source that acquisition reads. 43
5.3 Outcomes of an extraction attempt. 44
5.4 Discovery strategies, in the order they are tried (since 9 October 2026). . . . 45
5.5 Enabled sources per topic group and language. A source may cover several groups. 47
5.6 Rules applied to discovered links, and the case behind each. 48
5.7 The extraction cascade. 49
5.8 What happens after a failed step. 49
5.9 Checks of the acquisition layer against the live sources (2026). 53
5.10 Unit tests of the acquisition layer, by module. 54

6.1 Research questions and the measurements that answer them. 66
6.2 The custom set on 14 October 2026, by label and topic group: 109 natural claims (74 in English, 35 in Spanish) and 30 synthetic refuted ones (21 and 9). Of the natural claims, 31 were labelled by the annotator and 78 by an AI labelling pass awaiting the annotator's review (Subsection 6.3.4). 72

6.3 Record format of the custom validation set. The first ten keys are those of the X-Fact records used in this work, in the same order. 72
6.4 Annotation labels, after AVeriTeC [33], and the system verdicts they map to. The X-Fact mapping used in this work sends X-Fact's *partly true/misleading* to `MISLEADING` and its *complicated/hard to categorise* to `UNVERIFIED`, so both sets are scored on the same five classes. 73
6.5 The two halves of the custom set 78
6.6 How a wrong verdict is attributed to a stage. The first row that matches decides. 82
6.7 The experiments on the stages before the verdict. 85

7.1 The pipeline (`llama3.2:3b`) on the X-Fact pilot and on the Spanish rows of X-Fact's in-domain test set, with 95% bootstrap intervals. "Without leak flags" leaves out claims for which a cited source is newer than the claim or a ranked source is the fact-checker's own page (Section 7.1.4). 90
7.2 The X-Fact baselines as published (Gupta and Srikumar, Tables 3 and 4): macro-F_1 in percent over seven labels, mean (standard deviation) of four runs. α_1: 3,826 claims in the training languages and sources, Spanish (Chequeado) among them; α_2: 2,368 claims from unseen sources of four training languages; α_3: 3,381 claims in twelve unseen languages. 92
7.3 The system beside the X-Fact baselines. The two halves differ in label set, training, evidence, sample and year of search; the list that follows says how, and none of the differences is small. 92
7.4 How well each signal separates the candidates of 6 October (AUC, 95% bootstrap interval, 86 candidates the screen keeps). Labels by the AI labelling pass. . . 95
7.5 Ten one-group rounds re-scored at each topic emphasis: counts over their first twenty places (200), and over their first ten (100) in the last column. Labels by the AI labelling pass. 96
7.6 The custom set's test half, first pass 97
7.7 Run-to-run consistency on the re-check sample 100
7.8 The verifier models compared 101
7.9 The writer trial 102
7.10 The reviewer trial 103
7.11 Response time per role 104

9.1 The specific objectives of Section 1.4 as of 16 October 2026. 117
9.2 Measurements required before adopting an API-keyed engine. 119

A.1 Planned hours per week from 29 September 2026 (docs/roadmap.md). 125
A.2 Tasks per sprint and their status on 16 October 2026. 125

B.1 The source catalogue. 128

Acronyms

API	Application Programming Interface
AUC	Area Under the (ROC) Curve
ECTS	European Credit Transfer and Accumulation System
FIB	Facultat d'Informàtica de Barcelona
LLM	Large Language Model
MDS	Master's Degree in Data Science
NER	Named Entity Recognition
NLP	Natural Language Processing
RQ	Research Question
SDG	Sustainable Development Goal
UPC	Universitat Politècnica de Catalunya

Chapter 1

Introduction

1.1 Context

Three problems of today's news meet in this work.

False information travels faster than its correction. On Twitter between 2006 and 2017, false stories reached more people, faster and further than true ones, and the difference was driven by people rather than by automated accounts [39]. Professional fact-checking grew in response: by the middle of the last decade dozens of fact-checking organisations were active in Europe alone [15]. Their method is manual — a trained person chooses a claim, searches for evidence and writes a verdict — and it cannot keep pace with the volume of what is published. Automated fact-checking, the research field that tries to support or partly automate that work, frames it as a pipeline of claim detection, evidence retrieval and verdict prediction [17], and increasingly as a tool to assist fact-checkers rather than to replace them [28].

Distribution optimises for engagement. The systems that decide what a reader sees are usually optimised for clicks and attention, not for accuracy or usefulness, and personalisation tuned that way tends to narrow what each reader is shown [22, 30].

The news cycle is dominated by negative content. Readers react more strongly to negative news than to positive news [36], and a news supply shaped by that reaction erodes trust without necessarily giving readers anything they can act on. *Constructive journalism* answers by reporting problems together with the responses to them, keeping journalism's core functions while applying what positive psychology has learned about how news affects its audience [26].

The project behind this thesis starts from the combination of the three: a source of constructive news on chosen topics whose claims are checked against real evidence before they are published, and whose verdicts can be inspected by whoever reads them. This thesis builds and evaluates the first part of that project — finding the news and verifying it — and leaves its distribution to future work.

1.2 Motivation

Verification inside the flow, not after it. A fact-check usually arrives after a story has circulated, and is read by fewer people than the story it corrects. A publication of constructive news has a stronger reason than most to check before it publishes: its value to readers is that the good news it reports is true, and a single inflated figure or misattributed result costs it that value. The motivation of this work is therefore to make verification a step of the flow that finds and prepares the news, so that every published article carries a verdict per claim, with the evidence behind it, rather than an editorial label nobody can inspect. The same flow also does the work before verification that takes a journalist's time — reading many sources to find the few stories worth offering — so that the two can be measured together.

Failing towards caution. An automated verifier that is sometimes confidently wrong is worse than none: a reader cannot tell its right answers from its wrong ones. The system is designed to prefer "not verified" to a confident verdict it cannot ground, to report a search or model failure as a failure rather than as a verdict, and to show the evidence and reasoning behind every answer.

Low cost and the open web. The evidence for a claim about last week's study or this month's project is not in any fixed corpus: it is on the web, if anywhere, on the day the claim is checked. The system therefore searches the open web at verification time, as a person would. It does so without paid services: the search goes through a self-hosted metasearch engine, and the language model, entity recogniser, sentiment classifier and embedding model all run locally from open weights. The whole stack runs on a laptop, or on a single cloud virtual machine of about 115 USD a month (Section 5.8). A small newsroom or a single researcher can run it and reproduce every number in this thesis, and no result depends on a provider's undisclosed model or index. The price is a small model and a search engine that is sometimes refused by the engines behind it, and both are measured rather than assumed.

1.3 Problem Statement

Given a set of topics chosen by an editor, the system has to do what a journalist of a constructive publication does: find the recent articles on those topics that are worth offering, decide which of them to analyse, and verify the claims each one makes before it is published. Four properties of the domain make this harder than the problem automated fact-checking usually studies.

1. **Claims mix facts with interpretations.** A constructive article reports a study, a project, a figure or an event, and interprets it ("a hopeful step for the sector"). The first kind can be checked; the second cannot, and a system that tries to check it either wastes effort or produces verdicts with no meaning. The checkable claims have to be told apart from the rest before any evidence is sought.

2. **Evidence is on the live web, often in one place.** The claims are recent and specific,

so no fixed corpus holds their evidence: it has to be found on the open web at the time of checking. Frequently the only source is the press release of the organisation the story is about, which is not independent confirmation.

3. **Two languages.** The sources publish in English and in Spanish, so extraction, claim selection, search and verification must work in both, and be measured in both.

4. **False claims are rare.** Constructive outlets seldom publish a claim that the evidence refutes; errors are more often in a detail than in the substance. A system can look accurate by answering "true" to everything, and a benchmark from this domain has almost no refuted class to test it against.

Formally, for each claim c selected from an article published at time t_c, the system retrieves a set of evidence documents E from the open web at time $t_v \geq t_c$ and predicts a verdict $y \in \{\text{TRUE}, \text{PARTIALLY TRUE}, \text{MISLEADING}, \text{FALSE}, \text{UNVERIFIED}\}$ together with the subset of E it rests on. UNVERIFIED is a legitimate answer whenever the evidence does not decide the claim, and it is preferred to a guess. The gap between t_c and t_v matters: evidence published after the claim was made, including a fact-check of the claim itself, can make a claim look easier to verify than it was when it was published [14].

1.4 Objectives

1.4.1 General Objective

To design, build and evaluate a low-cost, end-to-end system that finds constructive news on topics chosen by an editor, in English and Spanish, and verifies its claims against evidence retrieved from the open web, measuring at each stage how close it comes to the work of a journalist.

1.4.2 Specific Objectives

O1 **Discovery and selection.** A flow that reads the configured English and Spanish sources for up to three topic groups, keeps what fits the publication's mission, ranks the candidates, and lets a person or a language model choose at most twenty to analyse (RQ1, RQ3).

O2 **Extraction and enrichment.** A pipeline that turns any article URL into clean text with its metadata, and annotates it with topics, entities, keywords, sentiment, quality scores, candidate claims and an embedding, stored in a data lake that keeps the lineage of every record.

O3 **Open-web fact-checking with auditable verdicts.** A pipeline that admits an article, selects its checkable claims, retrieves evidence from the open web, and returns for each claim a verdict that cites its sources and can be traced stage by stage (RQ2).

O4 **A validation set and an evaluation harness.** A hand-labelled set of claims from the system's own domain, and a harness that runs the pipeline over it and over X-Fact

and reports every metric with its uncertainty (RQ2, RQ3, RQ4).

O5 **Model benchmarks.** A comparison of language models as verifiers on the labelled sets, and of a local and a hosted model as the editor behind the text corrector, with their cost and latency (RQ2).

O6 **A reader view.** A public view of the published articles that shows each claim's verdict, its evidence and why each source was trusted.

1.5 Research Questions

The work is organised around four research questions. The first three follow the journalist's own sequence — finding the news, checking it, and deciding — and the fourth asks what the three amount to when they are joined into one system.

RQ1 **Finding.** Can a retriever that reads the open web, scoped to a chosen set of topics, simplify a journalist's task of finding constructive news on those topics?

RQ2 **Verifying.** Can the claims of the news it retrieves be verified automatically by the pipeline proposed in this work — claim selection, evidence retrieval on the open web and a verdict from a language model — and how can the quality of that verification be quantified?

RQ3 **Person and model.** Given the same news and the same evidence, how close does a large language model come to a person's judgement, both when choosing which news is worth publishing and when deciding whether a claim is supported?

RQ4 **The system.** Joined into one system, to what extent can these components take over the work of a journalist — finding, selecting and verifying constructive news — and at which points is a person still needed?

RQ4 is deliberately asked as a question of extent rather than as a yes-or-no question: whether a system "replaces" a journalist is not measurable, whereas the share of the work it performs at a stated quality, and the points at which it does not, are.

1.6 Scope

1.6.1 In Scope

- **Discovery and selection** of articles from 33 enabled news sources (20 in English, 13 in Spanish) grouped into five topic groups — society, science, environment, culture and health — with a screen for the publication's mission, a ranking of the candidates and a choice by a person or a language model.
- **Extraction** of article text and metadata through a cascade of strategies, for the articles analysed and for every web page read as evidence.

- **Enrichment** with local models: entities, keywords, topics, sentiment, editorial quality, candidate claims and embeddings.
- **Fact-checking**: admission, claim selection, open-web evidence retrieval, ranking with a pertinence gate, verification by a local language model, confidence recalibration and article-level aggregation.
- **Storage**: a three-layer data lake with lineage, a vector store and a graph database written by every analysis.
- **Evaluation**: a pilot on X-Fact, a hand-labelled validation set from the system's domain, an evaluation harness, and experiments on the stages before the verdict.
- **A reader view** of the published articles, and a deployment to Google Cloud designed and checked against a local emulator.
- **Writing a new article** from an analysed one, its related news and its fact-checks; reviewing, rewriting and publishing it to the reader with the AI disclosed (Section 5.10). Built on 15 October 2026; it answers none of the research questions and is described, not evaluated.

1.6.2 Out of Scope

- **User accounts, profiles, bookmarks and recommendations**, cut on 29 September 2026: none of them answers a research question, and each is a subsystem to build, secure and test. The reader view is what remains of the user-facing application.
- **The recommendation system and the mobile application** of the wider project (docs/arquitectura-tecnica.md, sections 3 and 4). The graph database this work writes is their foundation, but neither is built.
- **Evaluating the writing flow**, and publishing to social networks. The flow is built (Section 5.10), but none of the four research questions asks for it, so its measurement — the share of supported claims and the precision of its citations — is future work (Section 9.4), as are the media a social network would need.
- **Training or fine-tuning any model.** Every model is used as published; nothing is trained on the evaluation data.
- **Languages other than English and Spanish.** A source in a third language is refused by the test suite rather than scored with the English word lists.
- **Paid search or model APIs**, and a real cloud deployment: the infrastructure is written and checked against an emulator, but has never been applied to a billed project.

1.7 Structure of the Document

Chapter 2 reviews automated fact-checking, its benchmarks, constructive journalism, news selection and the existing tools, and positions this work among them. Chapter 3 describes how the project was planned and managed, how the plan changed, its risks and its budget. Chapter 4 specifies the system: its requirements, architecture, data model and the design of each stage. Chapter 5 describes how it was built and tested. Chapter 6 fixes how each research question is measured and describes the datasets and the evaluation harness. Chapter 7 reports what was measured and discusses it. Chapter 8 analyses the project's sustainability and its ethical implications, and Chapter 9 answers the research questions and lists the work left. The appendices give the detailed planning, the source catalogue, the annotation guide and the commands that reproduce every result.

Chapter 2

State of the Art

2.1 Automated Fact-Checking

2.1.1 The Fact-Checking Pipeline

Automated fact-checking is commonly framed as a pipeline that mirrors the work of a human fact-checker [17, 37]. Guo, Schlichtkrull and Vlachos organise the field into three stages and a fourth output:

- **Claim detection**: deciding which sentences make a claim worth checking. ClaimBuster is the early reference: it scores sentences of political debates for *check-worthiness*, the likelihood that a fact-checker would want to check them [19].
- **Evidence retrieval**: finding the documents, and the passages in them, that bear on the claim.
- **Verdict prediction**: classifying the claim against the evidence, from a binary true/false to the fine-grained scales fact-checkers publish.
- **Justification**: producing an explanation of the verdict that a reader can check.

A recurring conclusion of the field is that full automation is not yet realistic and that the most useful systems assist fact-checkers — finding claims, finding evidence, detecting claims already checked — rather than replace them [28]. This is the question RQ4 puts to the system built here: not whether it replaces a journalist, but which share of the work it can take on, at what quality.

The proposed method follows the same decomposition (Section 4.6). Claim selection plays the part of claim detection, with a structural rather than a learned notion of what is checkable; evidence retrieval and ranking find and filter open-web documents; a language model predicts the verdict; and its justification is the set of sources it cites, each with the verbatim passage it relied on, which the system checks mechanically.

2.1.2 Closed- and Open-Retrieval Verification

Verification systems differ first in where their evidence comes from.

In the **closed** setting, the evidence is a fixed corpus known in advance. FEVER, the most influential dataset of the field, built its claims by altering sentences of Wikipedia and defines the correct evidence as sentences of the same Wikipedia dump [38]. Systems index that corpus and are evaluated on whether they find the annotated sentences and reach the annotated label. The setting makes evaluation exact and reproducible, and it is unlike a real fact-check, where nobody knows in advance which document holds the answer.

In the **open** setting, evidence is sought on the web. MultiFC and X-Fact attach to each claim the snippets a search engine returned for it when the dataset was built [2, 18]; AVeriTeC goes further and has annotators answer questions about each real-world claim from web documents, restricted to those published before the claim, so that the evidence is what a fact-checker could have found at the time [33].

Open retrieval brings a problem the closed setting does not have: *leaked evidence*. Glockner, Hou and Gurevych showed that much of the evidence in such datasets was published after the claim, often by the fact-checkers themselves, so a system can "verify" a claim by reading its verdict; with only the evidence available before the claim, performance drops sharply [14]. This matters for any system that searches today's web for yesterday's claims, as this one does, and the evaluation flags such cases rather than counting them (Sections 6.1 and 7.1.4).

The system built here works in the open setting by necessity: the claims of constructive news are recent and specific, and no corpus holds their evidence. Why that rules out FEVER as its benchmark is argued in Section 6.2.2.

2.1.3 Language Models as Verifiers

Large language models changed the verdict-prediction stage. A pre-trained model holds a good deal of factual knowledge in its weights and can answer some factual questions with no evidence at all [31]; this *closed-book* behaviour is the baseline against which the contribution of retrieval is measured in this work (Section 6.1). Retrieval-augmented generation instead conditions the model on documents retrieved for the question [24], which is the shape of an evidence-based verifier: retrieve, then judge the claim against what was retrieved. More elaborate verifiers decompose a complex claim into simpler questions and answer each against evidence [29].

The same models *hallucinate*: they produce fluent statements that are unsupported by, or contradict, their input or the world [21]. For a verifier this is the central risk: a verdict whose explanation sounds right and rests on nothing. Three lines of work address it.

- **Citation.** Asking the model to cite the sources each statement rests on makes its grounding checkable; ALCE measures citation precision and recall automatically [11].
- **Atomic checking.** FActScore splits a generated text into atomic facts and checks each

against a knowledge source [27], the same unit of verification this work uses for claims.

- **Consistency and calibration.** SelfCheckGPT detects hallucinations from the disagreement of several samples of the same model [25]; more generally, the confidence a neural model reports is often poorly calibrated [16], so it cannot be taken at face value.

The verifier of this work takes the first and the last of these into its design rather than its evaluation. It reduces the model's task to a closed classification over five verdicts, requires it to cite the evidence it used and to quote the passage each source supports or contradicts, discards any quote that does not appear verbatim in its source, and recomputes the final confidence from properties of the evidence instead of trusting the model's own (Sections 4.6 and 5.4).

2.2 Benchmarks for Fact Verification

Table 2.1 summarises the benchmarks most relevant to this work. They differ in four respects that decide whether they can evaluate an open-web verifier of constructive news: where the claims come from, where the evidence comes from, the label scale, and the languages.

Benchmark	Claims	Claims from	Evidence	Labels
FEVER [38]	185,445	Altered Wikipedia sentences (English)	Wikipedia sentences (closed)	3
LIAR [42]	12.8k	PolitiFact statements (English)	None; claim and metadata only	6
MultiFC [2]	34,918	26 fact-checking websites (English)	Search-engine snippets	per site
SciFact [41]	1.4k	Rewritten citation sentences (English)	Scientific abstracts (closed)	3
X-Fact [18]	31,189	Fact-checkers, 25 languages	Search-engine snippets	7
AVeriTeC [33]	4,568	Real-world claims checked by fact-checkers (English)	Web documents from before the claim, with question–answer annotations	4

Table 2.1: Benchmarks for fact verification relevant to this work.

FEVER and SciFact are closed: their evidence is a fixed corpus, which makes them precise and unlike the task this system performs. LIAR gives no evidence at all, so it measures what can be guessed from a claim and its speaker. MultiFC and X-Fact are open, real-world and, in X-Fact's case, multilingual, with Spanish among the languages of its test sets; their evidence is search snippets collected when each dataset was built. AVeriTeC is the closest to how a fact-checker works — web evidence, dated before the claim — but is English only.

Two observations follow for this work. First, every benchmark with real-world claims takes them from fact-checking organisations, which choose claims because someone disputed them; most are political, and none is drawn from constructive news. Second, the benchmarks report a verdict's quality, not the effort it saves, which is what RQ4 asks. The choice made here — X-Fact as an external, open-retrieval benchmark and a custom validation set for the system's own domain — and the reasons for it are in Section 6.2.

2.3 Constructive and Solutions Journalism

Constructive journalism is defined by McIntyre and Gyldensted as journalism that applies techniques from positive psychology to news work in order to create more productive, engaging stories while holding to journalism's core functions [26]. It does not mean only good news: a constructive story reports a problem together with the responses to it, the evidence that they work, and what readers can take from it. Solutions journalism, a close relative, centres the story on a response to a problem and on the evidence of its results. Both react to a news supply that leans to the negative, which audiences attend to more strongly [36] and which can leave readers informed but discouraged.

For this work the publication's *mission* is that kind of news on five groups of topics: society, science, environment, culture and health. In practice the mission has to be stated operationally, because the same outlets publish both kinds of story. The system's discovery stage keeps what a constructive publication would offer and leaves out what it would not — party politics, crime, war, disasters, celebrity, sport, markets, service pages — and the line between hard news and constructive news on the same topic is left to a judgement, by a person or a language model (Section 4.4).

Why its claims are harder to check. Fact-checking practice and its datasets were built around political statements: a quote, a speaker, a disputed figure. The claims of constructive news are different in ways that make them harder, not easier, to verify automatically.

- **They are new.** A study published this week or a project launched this month has little independent coverage yet, and the open web may hold nothing but the announcement itself.
- **They are often sourced to their protagonist.** The figures of a positive story frequently come from the press release of the organisation the story is about. That confirms what was said, not that it is true, and the annotation guide of this work treats such a claim as not enough evidence (Section 6.3.5).
- **They mix facts and interpretation.** "The first hospital in the region to…" is checkable; "a hopeful step for the sector" is not, and both appear in the same paragraph.
- **They are rarely false.** The custom validation set of this work found no refuted claim among 109 natural ones (Section 6.3.1). Errors, when they occur, are in a detail — a figure, a date, an attribution — which is the `PARTIALLY_TRUE` case rather than the `FALSE` one.

2.4 News Discovery, Selection and Recommendation

Before a claim can be checked, someone has to decide which articles are worth reading. Two traditions address that decision.

Editorial selection. In a newsroom, an editor reads what the sources published and chooses what to run; aggregators of constructive news do the same by hand. The decision is a judgement of relevance and of fit with the publication's line, and it is the step this work tries to support rather than personalise: the system reads the sources, screens out what the publication never covers, ranks what is left, and leaves the choice to a person or to a language model that reads titles and summaries. Language models have been reported to match or exceed crowd workers on several text-annotation tasks [13], which makes them plausible stand-ins for a reader's first pass; whether they agree with an editor on *this* judgement is part of what RQ3 measures.

Recommendation. News recommender systems choose, per reader, from a large and fast-changing pool. Their survey by Karimi, Jannach and Jugovac lists what makes news a hard domain — short item lifetimes, the importance of recency, popularity effects, and the risk of narrowing what each reader sees [22] — the last of which Pariser called the filter bubble [30]. Industrial systems combine content similarity with graph structure: Twitter's SimClusters represents users and items by the communities they belong to [32], and TwHIN learns embeddings over a heterogeneous graph of users, items and topics [10].

The wider project behind this thesis plans a recommender of the second kind over the graph that every analysis writes, with the verification verdict as one of its signals (docs/arquitectura-tecnica.md, section 3). That recommender is out of scope here (Section 1.6); this thesis builds the editorial selection that comes before it and the graph that would feed it.

2.5 Existing Tools and Services

Fact-checking organisations. Professional fact-checkers publish verdicts on claims, mostly political, under their own rating scales [15]. PolitiFact in the United States and Chequeado in Argentina are the sources of the English and Spanish claims of the X-Fact pilot (Section 7.1); in Spain, Newtral and Maldita play the same role. Their articles are evidence of the highest value for a claim they have checked, and this system rates both Spanish organisations as evidence sources. It does not ingest them as news: a fact-check quotes the claim it debunks, and claim extraction would take that quotation for the article's own assertion (Section 5.2.2).

Automated tools. The research tools closest to practice assist a step of the fact-checker's work: ClaimBuster scores sentences for check-worthiness [19], and the systems surveyed by Nakov and colleagues find claims already checked, retrieve evidence or flag claims for review [28]. General assistants built on large language models answer questions with web search and citations, but they are not designed to abstain, to refuse a source they consider off-topic, or to report a failed search as a failure; their verdicts and their confidence cannot be audited in the way this work requires.

Constructive news services. Outlets such as Good News Network, Positive News and Reasons to be Cheerful, all in this system's source catalogue, curate constructive stories

editorially. Their selection is human and their verification is the ordinary editorial process of each outlet; neither is automated or exposed to the reader claim by claim.

No existing tool combines the three steps this work joins: finding constructive news on chosen topics, choosing what to analyse, and verifying its claims with a verdict that shows its evidence.

2.6 Positioning of This Work

Table 2.2 sets the work against the closest benchmarks and tools on the properties the problem statement (Section 1.3) requires.

	Open-web evidence	*English and Spanish*	*Constructive news*	*Local, no paid API*	*Cited, checkable verdict*	*Finds the news*
FEVER, SciFact	–	–	–	n/a	–	–
MultiFC, X-Fact	snippets	X-Fact	–	n/a	–	–
AVeriTeC	yes	–	–	n/a	yes	–
ClaimBuster	–	–	–	yes	–	–
Web-search LLM assistants	yes	yes	–	–	partly	–
This work	yes	yes	yes	yes	yes	yes

Table 2.2: The work against the closest benchmarks and tools. "n/a": a dataset, not a system.

The contribution is therefore not a new model or a new benchmark result. It is an end-to-end pipeline that reads constructive news in two languages, finds and ranks what is worth publishing on the topics asked for, and verifies its claims on the open web with local models and verdicts that can be audited — *evaluated stage by stage, against a person wherever a person takes the same decision.* The pipeline is the artefact; the evaluation is the contribution. Its custom validation set is, to the author's knowledge, the first labelled set of claims drawn from constructive news, and its X-Fact pilot documents what years between a claim and the search for its evidence do to an open-retrieval score.

Chapter 3

Project Management

3.1 Methodology

3.1.1 Iterative Development in Sprints

The work is organised into phases, and each phase into sprints of one to three weeks with a budget of hours. Every sprint closes with a working system: a phase never leaves a stage of the pipeline half-built for a later one to finish. The first two phases built the pipeline end to end (Phase 0, foundations) and then stabilised it (Phase 1); every later phase extends a system that already runs.

3.1.2 Parallel Tracks

Since 30 September 2026 the work runs on tracks side by side rather than one after another:

- **Development** (Phases 2, 3 and 6): scraping, fact-checking, the graph, the infrastructure and the interface.
- **Evaluation** (Phase 4): the validation set, the harness that runs the pipeline over it, and the tuning those two make possible, in that order, since each needs the one before it. The model benchmarks (Phase 5) start once Phase 4 has delivered labels and a harness to run them on.
- **Testing and quality assurance**, transversal and weighted towards the end, because testing follows what each phase builds. It ends one week before the report, so that the report describes a system that no longer changes.

3.1.3 Definition of Done and Status Tracking

A task is done when `./scripts/check.sh` passes: the backend and inference tests, the frontend typecheck and the labelling tool's tests. The rules most costly to break are enforced as tests (`backend/tests/test_invariants.py`) rather than kept as conventions. The roadmap marks each item done, partly done (with what is left), or not done, and states the verification behind each mark; it records planned hours, not hours spent.

3.1.4 Decision Records and Replanning

Every design decision and every incident that motivated one is written down in `docs/decisions/`. The plan is revised when the evidence changes, keeping the total of 900 hours fixed: a task that grows is paid for by cutting or shrinking another, and each cut is recorded with its reason (Section 3.3). Work that answers none of the research questions is cut or moved to future work.

3.1.5 Tools

Git, with a development branch merged into the main one; GitHub Actions running the same `check.sh` targets on every push; Docker Compose for the services; `uv` and `npm` for dependencies. Two further tools keep the plan honest. The roadmap (`docs/roadmap.md`) is checked against the code rather than the plan: an item is ticked only when it is done and verified, and its note says how. Decision records (`docs/decisions/`) keep the reason behind each design choice and each incident.

[To write: follow-up meetings with the director: frequency and format. Not recorded in the repository; only the author can state it.]

3.2 Planning

3.2.1 Workload

The thesis carries 30 ECTS at 30 hours per credit (Art. 62 of the FIB regulations), which is the 900 hours the plan distributes. The weeks run at about 30–35 hours, the pace of Phases 0 and 1, and each block's dates follow from its hours.

3.2.2 Phases and Tasks

Table 3.1: Phases, dates and planned hours (docs/roadmap.md, rebalanced on 30 September 2026).

Phase	Content	Dates (2026)	Hours
0	Foundations: scraping core, enrichment, fact-checking v1, job API and frontend v1	Jun 1 – Jul 31	300
–	Vacation pause	Aug 1 – 31	0
1	Pipeline completion: stabilisation, testing and bugfixing	Sep 1 – 28	140
2	Development: scraping strategies (Sprint 3) and fact-checking improvements (Sprint 4)	Sep 29 – Oct 19	120
3	Neo4j (done on Sep 29) and infrastructure deployment	Oct 20 – Nov 9	50
4	Evaluation: labelling, harness and tuning	Sep 29 – Nov 16	90
5	AI model testing: writing (Sprint 6) and verification (Sprint 7) models	Nov 10 – Dec 10	80
6	Interface: reader view	Dec 11 – 17	35
–	Testing and QA, transversal	Sep 29 – Dec 24	50
7	Thesis report	Dec 25 – 31	35
	Total		**900**

Phase 0 — Foundations. Four sprints built the pipeline end to end: the repository, the API skeleton and the declarative source catalogue with RSS discovery and trafilatura extraction (Sprint 0.1); the enrichment stack — keywords, entities, claims, topics, sentiment, quality and embeddings (0.2); the first fact-checking pipeline, with admission, claim selection, SearXNG and vector retrieval, ranking, the language-model verifier and the confidence scorer (0.3); and the job API with polling, the text corrector and the first frontend (0.4).

Phase 1 — Pipeline completion. Sprint 1 stabilised the core: pipeline edge cases, progress events for every stage, the correctness of the analysis cache and the shared test fakes. Sprint 2 was testing and bug-fixing: a full regression pass, concurrency tests, a load test of the synchronous endpoint and hardened handling of language-model failures. An audit on 21 September found and fixed a re-analysed URL being rejected as a duplicate of itself, a committed database password and the loss of job history on restart, and made the claims run concurrently.

Phase 2 — Development. Sprint 3 built the scraping strategies: an ingestion path, the BeautifulSoup and headless-browser steps of the extraction cascade, topic-page discovery, a source probe, and the catalogue's growth from 12 to 36 sources. Sprint 4 improved fact-checking — the search fix with a "search unavailable" state and a keyless second route, alternative retrieval strategies, and regression tests for ranking and confidence — and added, outside the original plan, discovery by topic group, the choice of up to twenty candidates by a person or the AI, the newsroom flow in the frontend, the mission screen, the candidate ranking and the topic emphasis (Section 3.3).

Phase 3 — Neo4j and infrastructure. The graph schema, its write path from every analysis, a backfill from the lake and the labelled facts, a read use-case and its live tests (done on 29 September); then the hosting decision, a production compose file, SearXNG for production, logging and monitoring, and the Google Cloud infrastructure as Terraform, checked against a local emulator (done on 1–2 October, except the first real deployment).

Phase 4 — Evaluation. The custom validation set (daily labelling batches, 150 facts as the target, a blind 20% re-label and its agreement), the evaluation harness (record format, resumable runs, metrics with bootstrap intervals, retrieval evaluation, stage attribution and journalist-effort metrics), a pilot on X-Fact, and the tuning of thresholds against the labels.

Phase 5 — AI model testing. Sprint 6 benchmarks a local and a hosted model as the editor behind the text corrector; Sprint 7 benchmarks language models as verifiers on the labelled sets, compares their verdicts with the human labels, re-tunes the confidence weights and selects the default model.

Phase 6 — Interface. A read-only feed of the published articles by topic, an article view with each claim's verdict, evidence and the reason each source was trusted, built on the existing design system, and public access to that view only.

Testing and quality assurance. Transversal: unit, concurrency and graph tests, continuous integration, a full-pipeline integration test, tests of the evaluation harness itself, a stress test of the guardrails against a model that bluffs, infrastructure smoke tests, load tests, and a final regression and bug-fix week.

Phase 7 — Report. Writing this document on a system that no longer changes.

The unfunded 18 hours. On 5 October an 18-hour block of experiments on the stages before the verdict — discovery, selection, admission, the topic classifier and time per stage — was added to Phase 4 without hours to pay for it. It is reported as such: the plan stands at 918 hours, an overrun of 2%, unless the writing-model benchmark of Sprint 6 (20 hours) is not run. The thesis priorities of 14 October (Section 3.3) moved that benchmark to "only if time is left" and moved ten of the eighteen hours — the discovery numbers and the blind selection rounds — into the work needed for RQ1 and RQ3, which is how the overrun is expected to be absorbed.

3.2.3 Gantt Chart

Figure 3.1 shows the plan. Green bars were done when the plan was last revised; blue bars are planned; the orange bar is work added on 5 October and not yet funded.

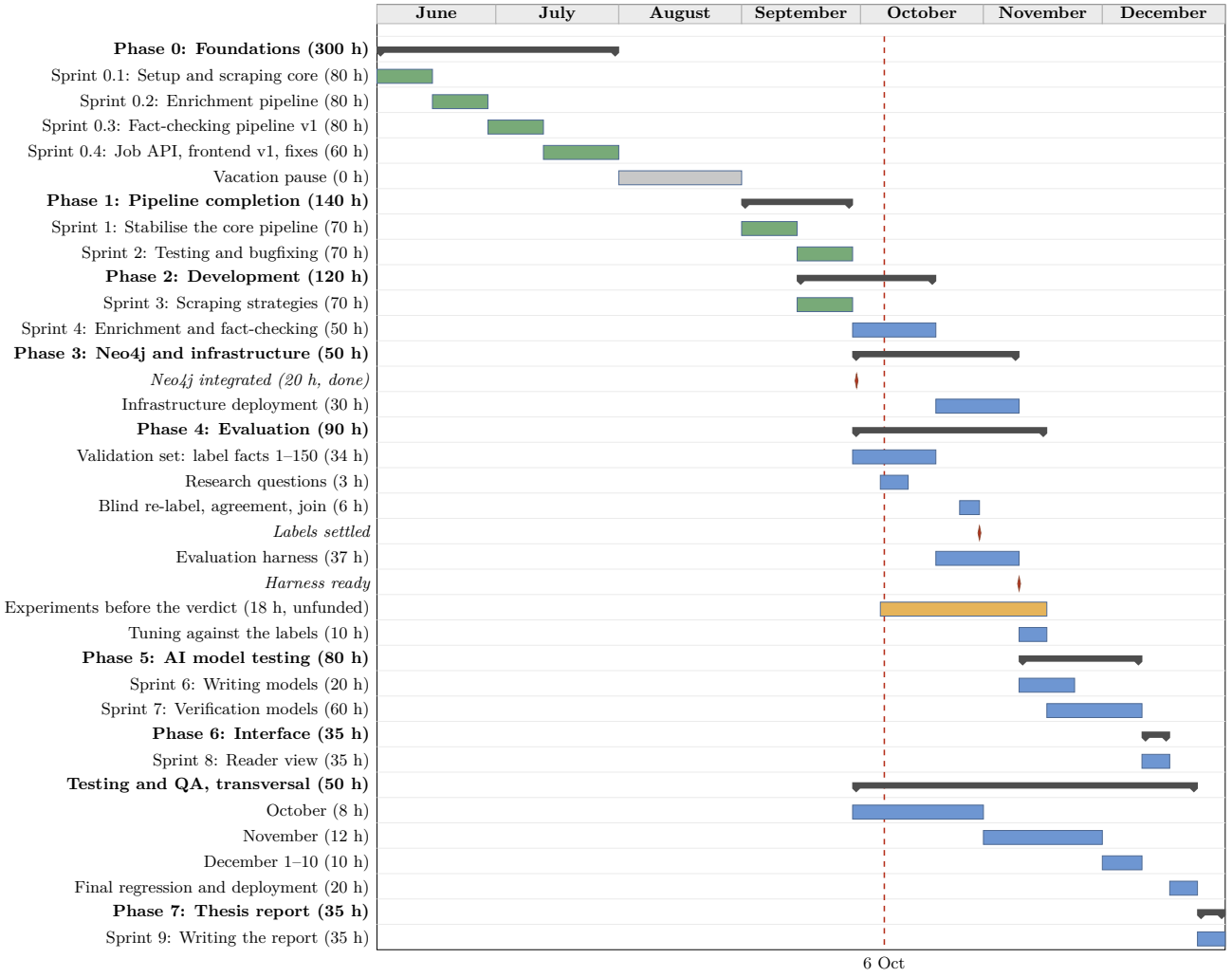


Figure 3.1: Gantt chart of the plan of 30 September 2026 (docs/roadmap.md), as it stood on 6 October (dashed line). From 14 October the thesis priorities of Table 3.3 replace its calendar where the two disagree; work done ahead of these dates is listed in Section 3.3.

3.2.4 Milestones

Besides the project's own milestones (labels settled on 30 October, harness ready on 9 November), the FIB regulations fix three:

- **Follow-up milestone** (Art. 9): a report on the state of the work, presented to the director at about half of it, and evaluated positively at least one month before the defence. After it, the project's data can no longer be changed, and confidentiality can no longer be requested (Art. 15).
- **Final report accepted** by the director at least seven days before the defence (Art. 10).
- **Defence** in public before a panel of three members of the FIB teaching staff (Art. 11).

The project's working targets, set on 14 October, are a full draft to the director by about 10 December 2026 and the remaining weeks for the corrections the review asks for, with the end of the plan on 31 December.

[To write: the official dates of the follow-up milestone, the delivery and the defence, once the defence period is chosen; check FIB's deposit date and the director's reading time against the 10 December draft.]

3.3 Deviations from the Initial Plan

The plan was rebalanced three times, always at 900 hours (Table 3.2).

Table 3.2: Hours per phase after each replanning (docs/roadmap.md). The earlier columns are regrouped into the current phases so that the three add up the same way.

Phase	Sep 28	Sep 29	Sep 30
0: Foundations	300	300	300
1: Pipeline completion	140	140	140
2: Development	140	130	120
3: Neo4j and infrastructure	70	65	50
4: Evaluation	40	80	90
5: AI model testing	120	80	80
6: Interface	40	20	35
Testing and QA, transversal	0	50	50
7: Thesis report	50	35	35
Total	**900**	**900**	**900**

28 September: the validation set is a task of its own. The custom validation set had been expected to fit into hours freed in Sprint 3. It turned out to be about 40 hours of work by itself — finding articles, extracting claims, searching evidence and labelling 150 facts — and it was given those 40 hours, paid for by the writing benchmark (Sprint 6, 70 to 50 hours) and the interface (Sprint 8, 60 to 40).

29 September: evaluation and testing get hours. Three things every later result depends on had no hours: the validation set's calendar, a harness that runs the pipeline over the labelled sets, and testing as a continuous activity. The validation set got its own window (29 September to 30 October, about ten facts a day, then the blind re-label a week later); testing became a transversal track of 50 hours, weighted towards the end; the harness got 40 hours; and the search fix, without which a benchmark would measure the search engine's uptime, got 10 hours inside Sprint 4. They were paid for by cutting the user-facing application to a read-only reader view (accounts, profiles, bookmarks and recommendations answer no research question), by reducing the writing benchmark to one local and one hosted model, and by de-scoping the topic classifier's accuracy, which is on no research question's path.

30 September: dates follow from hours. The previous plan kept the hours honest but not the calendar: one week held 65 hours of infrastructure while the 20-hour interface had two. Every block's dates were recomputed from its hours at the 30–35 hours a week the first two phases had run at; development and evaluation became two tracks run side by side; and the Neo4j integration, done on 29 September in far less than its 35 hours, gave 15 hours to the interface.

5 October: experiments before the verdict. Eighteen hours of experiments on discovery, selection, admission and the topic classifier were added without funding; how they are expected to be absorbed is in Section 3.2.2.

After the plan. The 30 September plan was overtaken in both directions.

- **Work done ahead of its dates.** The production compose file and the cloud infrastructure (planned for 20 October – 9 November) were done on 1–2 October, except the first real deployment, which needs a billed project and is left undone; the reader view (planned for 11–17 December) on 2–4 October; the evaluation harness, its metrics and retrieval evaluation (planned for 20 October – 9 November) on 2–4 October; continuous integration and the full-pipeline integration test on 2 October; and the X-Fact pilot on 13 October.
- **Work added outside the plan.** Discovery by topic group, the choice of up to twenty candidates by a person or the AI and the newsroom flow (5 October); the mission screen and the candidate ranking (6 October); the alignment of the rounds with the mission and the groups (9 October); and the person's topic emphasis (13 October). Each came from what a live round showed — election coverage in a Culture round, a match report ranked above an analysis of decarbonisation — and each was measured before it was kept (Section 4.4).
- **Work behind its dates.** The labelling: by 14 October the set held 109 natural facts, of which 31 were labelled by the annotator and 78 by an AI labelling pass not yet reviewed (Subsection 6.3.4); 39 facts from 17 off-mission articles had been moved out of the set; and no natural claim was `FALSE`.

14 October: feature freeze and thesis priorities. With about a third of this document written, mostly on scraping, and no result yet on the questions the thesis asks, the plan was reordered by what the thesis needs rather than by the calendar. The author fixed the four research questions; nothing new is built unless a priority needs it; and the remaining work was sorted into three levels (Table 3.3). Writing news from verified sources moved to the lowest level, since no research question asks for it.

15 October: the writing flow, outside the freeze. The next day the author decided to build it anyway: writing a new article from an analysed one, reviewing and rewriting it, and publishing it to the reader with the AI disclosed (Section 5.10). It was built and merged in one day, with its own tests, and it is the largest single deviation from the priorities of the day before. It does not change what the research questions need — the P0 items stand — but it took a day of the calendar they were written for, and its own evaluation is left as future work.

Table 3.3: Thesis priorities from 14 October 2026 (docs/roadmap.md). Where they disagree with the calendar of Section 3.2, they win.

Level	Items
P0 — without these there is no thesis	Fix the research questions and their metrics (done, 14 October); human adjudication of the gold set and the agreement between the AI pass and the annotator; a `FALSE` class (done, 14 October); a frozen tuning/test split; the verification runs on the test split under three evidence conditions; stage attribution; writing, about 140 hours, alongside everything else
P1 — what makes it a good thesis	Retrieval as an ablation on frozen claims; evidence against the age of the claim; two more verifier models; ten blind and ten assisted selection rounds (RQ3); the discovery numbers on five days (RQ1)
P2 — only if time is left	Tuning the ranking and confidence weights and the pertinence gate; the remaining experiments before the verdict; the writing-model benchmark; writing news from verified sources

3.4 Risk Management

Every risk in Table 3.4 was observed during the project rather than anticipated, which is why each has a date and a measured symptom. Probability and impact are rated low, medium or high as they stood when the risk appeared; the last column says what was done and what remains.

Table 3.4: Risks observed during the project, with their mitigation.

Risk	Prob.	Impact	Mitigation and status
Search engines refuse automated queries	High	High	On 25 September, 68 of 69 SearXNG queries came back empty (engines rate-limited, behind a CAPTCHA or timing out), and every affected claim looked like an honest `UNVERIFIED`. Mitigated by a closed engine list measured one by one, fewer concurrent searches, repeated queries sent once, a `searchUnavailable` state reported apart from the verdicts, and a keyless second route; the health check that sent a real search to every engine every five seconds was replaced. Remains: a search from a data-centre address is untested, and an API-keyed engine is future work (Section 9.4).
The language model times out on a CPU	High	High	On 1 October the first analysis on the production stack, CPU only, turned 3 of 4 verdicts into "provider unreachable" under timeouts sized for a GPU. Mitigated by separate production limits (one call at a time, 180 s) and by reporting `llmUnreachable` apart from verdicts. Remains: the development defaults are still sized for a GPU.
Sources block the scraper	Medium	Medium	EFE (HTTP 403) and Reuters (401) refused every request from 23 September to 6 October and were disabled on 6 October; they remain rated as evidence. Feeds that died (National Geographic, SINC, CNN, WHO) are read through section pages, a news sitemap or an API instead. The scraper names itself and does not try to pass for a browser.
Labelling takes longer than planned	High	High	Found on 28 September (the set is a 40-hour task, rebalanced). By 14 October an AI labelling pass had labelled 78 of 109 natural facts to keep pace, which created a second risk: a gold set labelled by a model, used to judge a model. Mitigated by disclosing every AI label in its record and in this document, by making the annotator's review a condition of the set, and by measuring the agreement between the two as a result (RQ3). Remains: the review itself.
No refuted claims in the domain	High	High	None of 109 natural claims was `FALSE`, so an always-`TRUE` system would score 83%. Mitigated by 30 synthetic refuted claims built by substitution, as FEVER and SciFact built theirs, and reported apart (Subsection 6.3.7).

Risk	Prob.	Impact	Mitigation and status
The cloud deployment is never applied	Medium	Low	The infrastructure is written and checked against a local emulator, which runs no guest, so the virtual machine's boot is unverified. The thesis does not depend on it: every result is produced on the development stack. Left as future work, with a step-by-step runbook.
Scope grows faster than results	Medium	High	Features added outside the plan in early October (Section 3.3) were each useful and measured, while the research questions had no answer. Mitigated by the feature freeze and the priorities of 14 October.
Native crash in the evaluation harness	Low	Low	The harness process crashed twice in native code while parsing scraped pages during the Spanish X-Fact run. Absorbed by the resume rule, which re-runs exactly the claims a crash lost; the cause is not located.

3.5 Budget

3.5.1 Personnel Costs

The 900 hours of Table 3.1 are carried by one person, but they are costed as the four roles a team would need. Each phase is assigned to the role whose work it is: building the system to the developer (Phases 0, 2, 3 and 6, and Sprint 1 of Phase 1); the validation set, the harness, the experiments and the model benchmarks to the data scientist (Phases 4 and 5); Sprint 2 and the transversal testing track to the tester; and planning, replanning and the report to the project manager (Phase 7). The allocation is approximate — the foundations also contain modelling work, and every phase some planning — but it adds up to the plan.

Table 3.5: Planned hours per role, from Table 3.1.

Role	Phases	Hours
Developer	0 (300), 1: Sprint 1 (70), 2 (120), 3 (50), 6 (35)	575
Data scientist	4 (90), 5 (80)	170
Tester	1: Sprint 2 (70), testing track (50)	120
Project manager	7 (35)	35
Total		**900**

The 18 unfunded hours of experiments (Section 3.2.2) are data-scientist hours and are added to that role if the writing benchmark they are expected to replace is run.

[To write: the gross hourly cost of each role, with its source (a published salary survey or collective agreement for Spain, plus employer social-security contributions), and the resulting cost per role and in total.]

3.5.2 Hardware and Software Costs

Hardware. All development and every evaluation run in this thesis used one laptop: an Intel Core i7-12700H, 16 GB of RAM and an NVIDIA RTX 3050 Laptop GPU with 4 GB, on which the 3-billion-parameter language model runs mostly on the GPU. Its cost is amortised over the project rather than charged in full:

$$\text{cost} = \text{price} \times \frac{\text{months of use in the project}}{\text{useful life in months}},$$

with seven months of use (June to December 2026, the August pause included) and an assumed useful life of four years.

[To write: the laptop's purchase price, and the amortised cost that results.]

Software. Every component is open source and free of licence cost: Python, FastAPI and `uv`; Next.js and TypeScript; Docker and Docker Compose; SearXNG, Qdrant, Neo4j Community Edition and Ollama; trafilatura, BeautifulSoup and Playwright; and the open-weight models GLiNER, the XLM-RoBERTa sentiment classifier, BGE-M3 and Llama 3.2. The repository is hosted on GitHub and its continuous integration runs on GitHub Actions, both within their free tiers. Software costs are therefore zero.

3.5.3 Infrastructure Costs

Cloud. The production deployment is sized for one Google Cloud Compute Engine virtual machine, an `e2-standard-4` (4 vCPU, 16 GB) in Madrid (`europe-southwest1`), at about 115 USD a month on demand or about 73 USD with a one-year commitment, plus a 60 GB balanced disk and daily snapshots kept seven days (Section 5.8). The size comes from measurements: about 9.6 GiB were resident across the production stack, and the language model needs about 45 s per claim on a CPU. The infrastructure was checked against a local emulator and never applied to a billed project, so its cost to date is zero; running it for one month of demonstration would cost about 115 USD.

Language-model providers. The local model has no per-token cost: its cost is the machine's time, which the evaluation reports beside every run (36.5 s per claim at the median in the X-Fact pilot). Hosted providers are priced per run from a table in the repository (`backend/data/evaluation/prices.json`), filled in from the provider's pricing page on the day of the run, never from memory. No hosted model has been run, so this cost is also zero to date.

Search. The self-hosted metasearch engine has no cost. An API-keyed search engine, the most effective remedy for refused searches, is future work precisely because it would add a recurring cost and a credential (Section 9.4).

3.5.4 Contingency and Total

Two unforeseen costs are identified and priced in advance: one month of the cloud virtual machine for a live demonstration (about 115 USD), and the hosted verifier model of the model comparison, whose price is read on the day of the run. The rest of the risk in this project is in hours rather than money (Section 3.4), and is absorbed by the replanning rule of Section 3.1.4: a task that grows is paid for by shrinking another. A contingency margin of 10% is applied to the personnel and hardware costs to cover what the plan does not foresee.

[To write: the total: personnel (Table 3.5 times the hourly costs), amortised hardware, software (zero), infrastructure (zero to date, 115 USD with the demonstration month), the hosted model run, and the 10% contingency.]

Chapter 4

Proposed Method: Specification and Design

4.1 Requirements

4.1.1 Functional Requirements

Table 4.1: Functional requirements.

Id	The system shall...
FR1	**Discover**: read the configured sources for up to three topic groups and list their recent articles with title, summary, date and language.
FR2	**Screen and rank**: leave out what the publication never covers and what is not an article, and order the rest by how well it fits the mission and the topics asked for.
FR3	**Select**: let a person, a language model, or both choose at most twenty candidates to analyse, and record who chose what.
FR4	**Extract**: turn any article URL into its text, title, author and publication time.
FR5	**Enrich**: annotate each article with topics, entities, keywords, sentiment, editorial quality, candidate claims and an embedding.
FR6	**Admit**: decide whether an article is on topic, constructive enough and not a duplicate before any evidence is sought, and record every reason it is turned away.
FR7	**Verify**: for each selected claim, retrieve evidence from the open web and the system's own corpus, rank it, and return one of five verdicts with a confidence, an explanation and the evidence it cites.
FR8	**Report failures as failures**: mark a claim whose search or model could not be reached as such, apart from the verdicts.
FR9	**Store**: keep the raw, processed and served form of every analysis with its lineage, and write it to a graph database.
FR10	**Publish**: show readers the published articles with each claim's verdict, its evidence and why each source was trusted.
FR11	**Trace**: show, during and after a run, every query sent, every source found and the rating each received.
FR12	**Evaluate**: run the claim-checking stage over a labelled set and report its metrics with their uncertainty.

Table 4.2: Non-functional requirements and how each is met.

Id	Requirement	How it is met
NFR1	**Low cost**: no paid API; runs on a laptop or one VM	Local open-weight models; self-hosted metasearch; one compose stack
NFR2	**Auditability**: every verdict traceable to its evidence	Cited sources with verbatim quotes; a per-run journal of every event; lineage on every stored record
NFR3	**Fail towards caution**	`UNVERIFIED` when the evidence does not decide; ungrounded verdicts downgraded; outages reported apart
NFR4	**Two languages**	English and Spanish lexicons, language detection, per-language queries; a third language fails the test suite
NFR5	**Reproducible evaluation**	Resumable runs keyed by dataset, model, thresholds and commit; seeded bootstrap; committed reports
NFR6	**Per-run tunability**	Every threshold can be overridden for one run without affecting concurrent runs
NFR7	**Bounded load on shared services**	One concurrency ceiling per external service, not per request
NFR8	**Security**	An address-based guard on every server-side fetch; an API key and a site password in production; read-only graph console

4.1.2 Non-Functional Requirements

4.2 System Architecture

The system is a set of services run together by Docker Compose (Figure 4.1).

- **Backend** (Python, FastAPI): the API and all the business logic — discovery, selection, extraction, enrichment, admission, fact-checking, storage and the evaluation harness. It holds the vector store (Qdrant, embedded in the process) and the analysis cache, and journals every job event to disk.
- **Inference service** (Python, FastAPI): a model server for the three transformer models — GLiNER for entities, an XLM-RoBERTa sentiment classifier and the BGE-M3 embedding model. It knows nothing about topics, thresholds or scoring; the backend reaches it over HTTP.
- **SearXNG**: a self-hosted metasearch engine that sends each query to a closed list of web and academic search engines and merges their answers.
- **Ollama**: the language model, behind an OpenAI-compatible endpoint, so a hosted provider can replace it by configuration alone.
- **Neo4j**: the graph database every analysis is written to.
- **Frontend** (Next.js, TypeScript): the newsroom and lab pages and the public reader view. It calls the backend from the server side only, so the API key never reaches a browser.

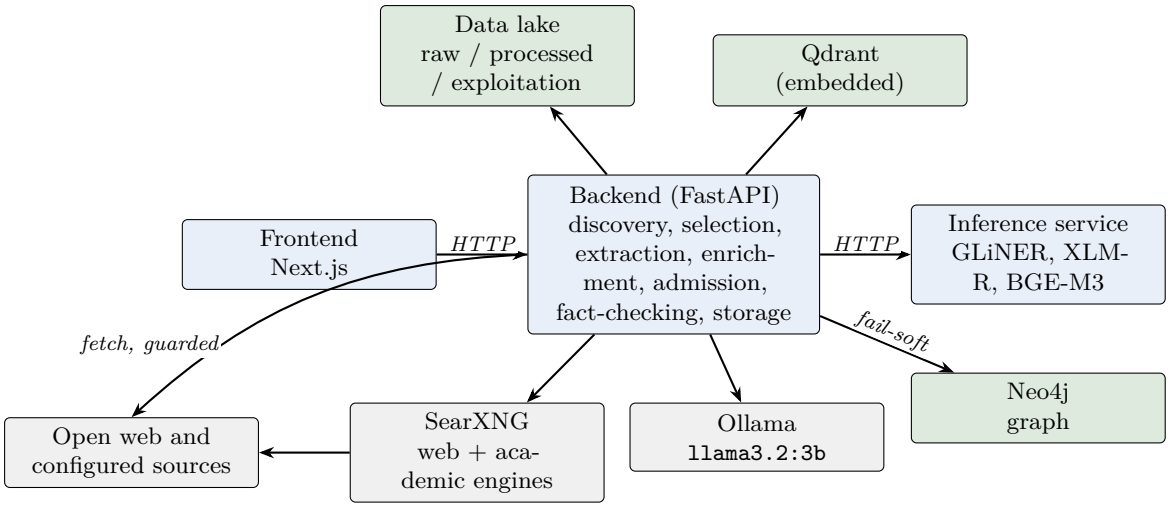


Figure 4.1: The services and how they talk. Every arrow from the backend to an external service passes through that service's concurrency ceiling (Section 4.7).

Why the models live in their own service. The three transformer models used to load inside the backend, beside the API. The split was made for three measured reasons: the machine-learning wheels dominated and broke the backend's image build; models loaded lazily on the first request, so the first analysis after a restart silently paid a 10–15 s load; and the models measure 3.58 GiB resident, while the backend container was capped at 1 GiB, which made an out-of-memory kill on the first real analysis likely. After the split the backend runs in about 230 MiB and the inference service in 3.58 GiB; the inference service reports itself healthy only once all three models are loaded, and the backend waits for it. A test of the repository fails if the backend ever imports a model library again.

The language model is a service too. The verifier, the selection step and the text corrector all reach the model through one client that speaks the OpenAI wire format. Switching from the local model to a hosted provider is a change of two settings, which is what makes the model benchmarks of Phase 5 one command per model.

4.3 Data Architecture

Three stores hold what the system produces, each for a different use.

A three-layer data lake. Every analysis is written into a lake of three layers, one write per stage as that stage completes:

- **Raw**: the extracted article and its fetch metadata. Nothing model-derived, so it stays valid when every model changes.
- **Processed**: the full enriched article and its fact-check report, rejections and discarded evidence included. It is the layer everything else is re-derived from.
- **Exploitation**: one flat, ready-to-serve record per analysis, with the editorial decision precomputed as *publishable* — admitted, and with no claim judged `FALSE` or `MISLEADING`.

Writing per stage means that a fact-check that fails (the search engine down, the model timing out) does not discard an enrichment that already cost the whole NLP stack. Every record carries its *lineage*: the run, the article, a hash of the body that joins the layers, its parent record, the pipeline version, the code revision, the model versions and any threshold the run overrode. Record identifiers are derived from the layer, the article and the run, so repeating a write overwrites rather than duplicates it, and an append-only manifest keeps the history of every write.

A vector store. Qdrant holds one embedding per analysed article. It serves three purposes: duplicate detection at admission, the system's own corpus searched for evidence, and the related-article signal. An article is added to it only after its fact-check, so it cannot be found as evidence for its own claims.

A graph. Every finished analysis is also written to Neo4j, in one transaction per article, after the lake. Its schema (Figure 4.2) has seven node types: articles, the sources that publish them and the evidence pages, the entities they mention, their topics, their claims and the five verdicts. Every relationship carries the method that wrote it — the pipeline or a manual label — so a hand label and the model's verdict on the same claim sit side by side instead of one overwriting the other. The lake is the record and the graph a view of it: a graph write that fails costs one event and never the analysis, and a synchronisation endpoint rebuilds the graph from the lake and the labelled facts at any time.

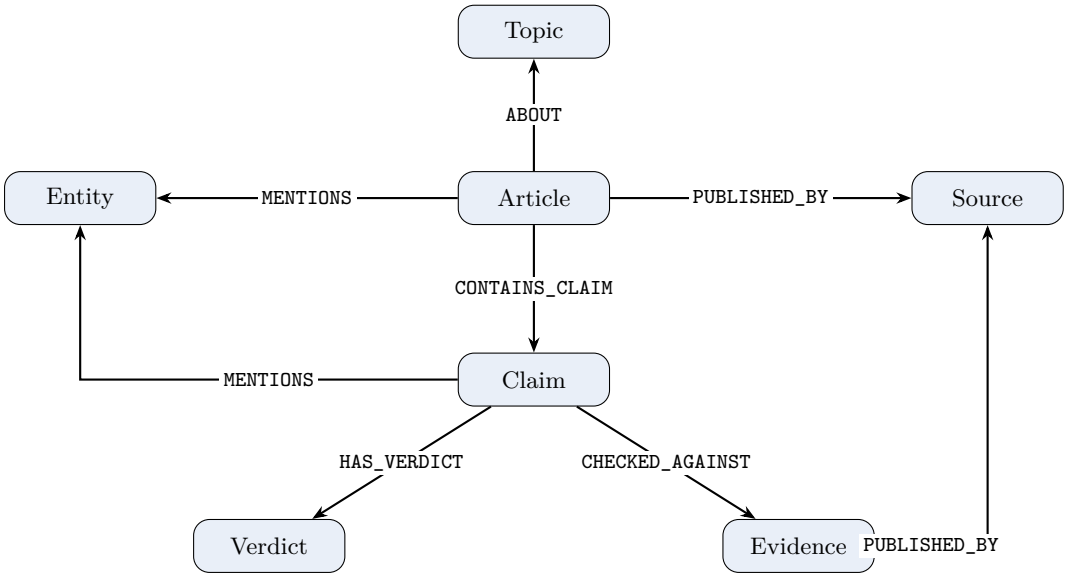


Figure 4.2: The graph schema. Every relationship carries `method` (`pipeline` or `manual`).

4.4 Candidate Selection

Between discovery and analysis sits a choice. Analysing an article costs a scrape, an enrichment and one language-model call per claim — about three minutes per article on the production CPU — so the system does not analyse everything it finds. A *round* lists the candidates discovery found for up to three topic groups, ranked; a person, a language model or both

choose at most twenty; only those become analysis jobs. Every round is recorded with what was found, what the model proposed and why, what was sent and who chose it, which is the data RQ1 and RQ3 are measured on. How candidates are found, and the filters applied to links before anything is fetched, are described in Section 5.2; this section covers what happens to them once found.

The mission screen. Topic keywords say nothing about tone: a Culture round listed election polls and celebrity interviews beside a free music-education programme, all on topic by their words. The screen leaves a candidate out on any of three grounds: it is not an article (a podcast, a roundup, a quiz, a gallery, a media advisory, a promotion, shopping), recognised by phrases in its title or summary; its headline reports a death; or its title and summary are nearer, in the embedding space, to one of nine descriptions of what the publication never covers — politics, strikes and protests, crime, accidents and disasters, war, celebrity, sport, markets and personal finance, service pages — than to every one of the 23 topics, by more than a margin. The margin was chosen by reading 527 live candidates by hand: 0.04 in general, any amount for politics, 0.02 for the categories whose doubtful band held almost nothing worth keeping. The three outlets that publish only positive news are exempt, because their editors already chose. If the inference service cannot be reached, nothing is left out and the round says so; the screen never passes silently. What it cannot do — tell the housing policy of a front page from good news on the same topic — is left to the next steps.

The ranking. A round over two or three groups finds 60 to 80 candidates, and a person reads the first twenty. Each candidate is scored in $[0, 1]$ from what the round already holds, without fetching a page or calling a model:

$$s = 0.6\,(1 - e) \cdot \mathit{impact} \;+\; 0.6\,e \cdot \mathit{topic} \;+\; 0.25 \cdot \mathit{record} \;+\; 0.15 \cdot \mathit{reliability},$$

where *impact* is how much nearer the candidate comes to a description of positive impact than to the nearest off-mission one; *topic* is how well it fits the groups asked for; *record* is the share of its source's newest items the screen kept in this round; *reliability* is the source's rating; and $e \in [0, 1]$ is the *topic emphasis*, the person's own trade between positive impact and topic, 0.6 by default. The weights were measured on 117 candidates labelled before any score existed, where the score separated stories worth offering from the rest with an AUC of 0.78 (95% interval 0.67 to 0.87); the default emphasis on ten labelled one-group rounds, where it is the value at which the first ten places hold only the group asked for (Section 7.2). Two rules act on the order rather than the score: a story several outlets carry is offered once before any story is offered twice, and a candidate nearer another group's topics than the round's is listed after every candidate that fits. The best twenty are shown, forty on request.

The AI selection. On request, a language model scores every shown candidate from 0 to 10 for positive impact, from its title and summary, against a written definition: a real change that improves things for people or the planet beyond those in the story, backed by something checkable; low for sport, celebrity, lifestyle, opinion, crime, conflict and disasters.

It is told the round's topics, not the emphasis. Its picks are the best twenty at 6 or more, ticked on the page with a reason each, and remain the editor's to change. A language model is used here, where admission deliberately has none, because what the publication means by positive impact is a judgement of what changed and for whom, and the cost is one call per ten candidates rather than one per article. A pick still goes through admission and the whole pipeline: the selection decides what to spend analyses on, and replaces no check.

The choice. The editor keeps or changes the proposal and sends at most twenty. The backend, not the page, records whether the choice was the person's, the model's or both, and how many of the model's picks were kept, dropped and added. A round can also be chosen *blind*: the editor sends a choice first and the model is run on the round afterwards, so that its agreement with the editor is not inflated by the editor having seen its picks (Section 6.1).

4.5 Enrichment

Every article that comes out of the extraction cascade (Subsection 5.2.7) is turned from plain text into an enriched record by a stack of language-processing steps. Each runs in the article's language, detected from stopword frequencies, with English and Spanish word lists for every heuristic.

- **Keywords** by YAKE, an unsupervised statistical method that needs no training data [5], with the article's language's stopwords.
- **Entities** by GLiNER, a named-entity recogniser that takes the entity types as input rather than fixing them at training time [44]: people, organisations, places, countries and others. The model reads at most 384 tokens, so a long article is read in sentence-bounded chunks; until 16 October it was silently truncated to its opening (Section 5.3).
- **Candidate claims**: each sentence is scored by the presence of structural signals of a checkable fact — named entities, figures, dates, units, reporting verbs such as "announced" or "confirmed", quotations, a minimum length — and kept above a threshold. The score measures whether a sentence has the *form* of a verifiable fact, not whether it is important; a sentence must also end as a sentence, which removed headlines, captions and fragments cut at an abbreviation.
- **Topics**: the article's embedding is compared with descriptions of 23 topics, organised in five groups. Each topic gets a confidence, and a topic counts when its confidence clears a threshold (0.35). A softmax over the same similarities is also computed, but over 23 near-equal cosines it is close to uniform and carries no signal; nothing downstream uses it.
- **Sentiment** by a multilingual XLM-RoBERTa classifier [4], in full precision: an int8 version was measured on 336 real texts and rejected because it changed the label in 8–14% of them, mostly from neutral to positive, which would have quietly loosened the admission gate.

- **Editorial quality**: readability, objectivity, constructiveness, hopefulness, social impact and inspirational value, from curated word lists per language and standard readability formulas.
- **An embedding** of the whole article by BGE-M3, a multilingual embedding model [6]. The same 1,024-dimensional space underlies duplicate detection, topic classification, the mission screen, claim deduplication and evidence ranking.

The three transformer models run in the inference service (Section 4.2). Each step fails differently by design: if entity extraction fails, the claim scorer loses one weak signal and carries on; if sentiment or embeddings fail, the article fails, because a silently neutral sentiment or a zero vector would corrupt an admission decision without anyone noticing.

4.6 The Fact-Checking Pipeline

Fact-checking is not one true-or-false step but a chain of seven stages (Figure 4.3), each more expensive than the one before it. Any stage can stop or downgrade the result, and no later stage can undo a strong rejection by an earlier one. The governing principle is to fail closed: when in doubt, the system leans towards "not verified" rather than towards a confident verdict that may be wrong, because a false sense of certainty costs more than caution. The only point where a language model takes part is bounded to a closed classification that must cite its sources, and its output is then checked by rules that do not depend on the model's own judgement.

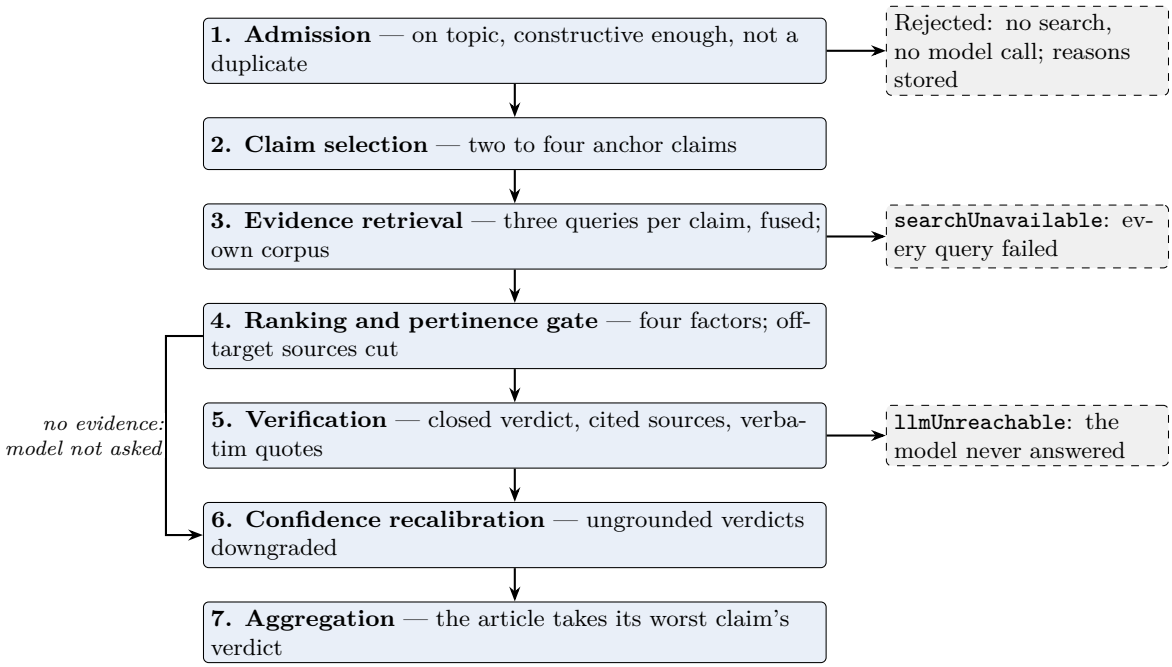


Figure 4.3: The seven stages. Stages 2–6 run per claim, and claims run concurrently; each claim records the furthest stage it reached. The dashed boxes are outcomes reported apart from the verdicts.

Stages 1 and 7 work on the article; stages 2 to 6 work on each claim, in that order and never overlapped, since each feeds the next. A claim whose verdict leaves stage 6 in doubt can,

when the run enables it, go back through stages 3 to 6 once more with a second, wider search (Subsection 4.6.7). What the evaluation measures is the claim-checking part, stages 3 to 6, through the same entry point the single-claim endpoint uses (Section 6.4).

4.6.1 Admission Filter

Before anything is spent on verification, the article must pass three independent checks. They judge the *article* — its topics, its tone, its neighbours in the vector store — and none needs evidence, a search or a language model, which is why they come first and are the cheapest stage.

- **Topic relevance**: at least one topic's confidence must reach a minimum (0.35 by default). No topic has to dominate, since an article can legitimately touch several.
- **Positive impact**: a weighted average of constructiveness, inspirational value, hopefulness, positive sentiment, objectivity and social impact, in $[0, 1]$, must reach a minimum (0.1 by default). Hard floors on objectivity and negative sentiment, which would reject an article whatever its average, exist behind a single switch and are off by default: the floors on constructiveness and inspiration rejected too many real articles, and now only lower the score and are recorded as reasons.
- **Not a duplicate**: the article's embedding is compared with its nearest stored neighbour. Below a relatedness threshold (0.80) the two are unrelated; between it and a duplicate threshold (0.90) they are *related*, a useful signal that does not disqualify; above it the article is a duplicate. A stored article with the same URL never counts: it is the same article from an earlier run, and a re-analysis must not reject itself.

All three always run, even after one fails, so a rejection names every reason. A rejected article is stored with its reasons and is never searched or sent to a model. Every threshold can be overridden for one run (Section 5.5).

4.6.2 Claim Selection

Not every candidate claim of the enrichment stage is worth checking. Two controls, both in the embedding space, reduce them to the few that carry the article:

- **Semantic deduplication**: when two candidates state the same fact in different words (similarity above 0.92), only one survives.
- **An anchor band**: the remaining candidates are ranked by how load-bearing they are for the article's thesis — how central they are to its headline and lead, how much they share its main subjects, how specific they are — and between two and four are kept. The ceiling stops an unusually dense article from multiplying the cost of verification; an article that yields fewer than the minimum is reported as such rather than padded.

Sentences that read as opinion or interpretation are dropped before the ranking: no evidence can confirm or refute "a hopeful step for the sector". The selector's precision — the share

of its proposals a person accepts as checkable claims — is measured by the daily labelling batches, which ask the annotator to label or reject every claim it proposes, with a reason (Section 6.3).

4.6.3 Evidence Retrieval

The failure the design prevents. An early version built one query per claim from its entities, figures and dates. A Spanish claim that the Coyote's mail-order ACME purchases stood for post-war American consumerism was searched as "coyote ACME Estados Unidos Segunda Guerra Mundial", which every page explaining the Greek word *acme* matches; those pages were the best embedding match, the model was shown three of them, and the claim came back `FALSE` at 83% confidence. Each stage had behaved as designed, and none could produce the honest answer, which was `UNVERIFIED`.

Three queries, fused. Each claim is now searched with a planned set of three queries:

- an **anchor** query — entities, quoted figures and dates — for precision on the subject;
- a **proposition** query — the claim's own content words plus two anchors — for what the claim asserts, which the anchor query lacks;
- a **refutation** query — the anchors plus words that would contradict the claim, per language — because every query built from a claim is phrased affirmatively and could otherwise only confirm it.

The three run concurrently and their ranked lists are fused by reciprocal rank fusion [8], which orders candidates by how much the queries agree rather than by which query ran first, and each candidate records which queries found it. When a claim lost its subject on being separated from its article ("the programme trained 30 experts"), the headline words that the article's own keywords agree on are restored to its queries, and the article's own URL is dropped from the results, so an article cannot confirm itself.

Two sources of candidates. The queries go to SearXNG, which consults a closed list of engines measured one by one — general web engines and, at half weight, four academic indexes (arXiv, Crossref, Semantic Scholar and PubMed) that keep answering when the web engines rate-limit. In parallel, the system's own corpus of analysed articles is searched by embedding, always excluding the article itself.

A funnel. Reading a page in full is the most expensive operation of the pipeline, so candidates are first pre-ranked cheaply on their title and snippet, with the same semantic and lexical split the ranking uses, and only the best eight are fetched through the extraction cascade. Evidence pages never escalate to the headless browser.

Search unavailable is not a verdict. If every query of a claim fails — SearXNG unreachable, or answering empty while reporting its engines down — the claim is marked

`searchUnavailable` and stops here with a note that the web was never asked. Its verdict is still `UNVERIFIED`, but it no longer looks like a judgement about the claim, and the evaluation sets it apart. Before giving up, a query SearXNG cannot answer is sent once to a second, keyless route.

4.6.4 Evidence Ranking and the Pertinence Gate

The fetched evidence is re-ranked by a score of four independent factors:

$$\mathit{relevance} = 0.45 \cdot \mathit{semantic} + 0.20 \cdot \mathit{lexical} + 0.20 \cdot \mathit{recency} + 0.15 \cdot \mathit{reliability}.$$

- **Semantic**: the embedding similarity between the claim and the evidence.
- **Lexical**: the share of the claim's own terms the evidence contains, anchors counted double. It is the counterweight to the semantic factor, which scores a page *about the same subject* as highly as a page that addresses the claim — exactly how the ACME etymology pages won.
- **Recency**: an exponential decay with a half-life of a year, so old evidence loses weight gradually rather than being discarded.
- **Reliability**: a rating per domain, from the source catalogue. Only the 37 configured domains have a real rating; any other domain gets a neutral 0.5 and is flagged as unrated, so the default is never presented as a rating.

The weights must sum to one, and the backend refuses to start if they do not: the configuration template once shipped weights summing to 1.2, and nothing noticed. The best five sources per claim are kept, at most one per domain.

The pertinence gate. In the same step, each source gets a second score that leaves recency and reliability out:

$$\mathit{pertinence} = 0.5 \cdot \mathit{semantic} + 0.5 \cdot \mathit{lexical}.$$

A recent, reliable page about something else is still about something else, and letting those two factors lift it would repeat the mistake the gate exists to stop. A source whose pertinence is below a threshold (0.25 by default) is cut *before the model sees it*, with a reason that names the figure: it cannot be cited, counted towards the evidence minimum or treated as corroboration. The threshold is deliberately low — a floor against off-target retrieval, not a relevance ranking — because cutting real evidence turns a checkable claim into `UNVERIFIED`, another way of being useless. It is a per-run threshold, unlike the weights, because it is the knob that trades a confident wrong answer for an honest one. Its default was reasoned, not fitted: no labelled set existed when it was set.

4.6.5 Verification with a Language Model

The model is given the claim, the article's headline and opening (so that it reads the claim in its article's sense), and the ranked evidence, numbered. It must return a structured answer:

- a verdict from a closed set of five — `TRUE`, `PARTIALLY_TRUE` (the central assertion holds, a detail does not), `MISLEADING` (technically true but deceptive, or sources of equal standing conflicting), `FALSE` and `UNVERIFIED`;
- a confidence and a short explanation;
- the numbers of the evidence it used; and
- for each source, its stance — *supports*, *contradicts* or *unrelated* — with the verbatim passage it relied on.

It is told to answer `UNVERIFIED` when the evidence is empty or does not bear on the claim, never to assert what the evidence does not support, and to mark as *unrelated* a source about a different event even when it shares a place, a date or a figure with the claim.

Reducing the task to a closed classification with cited, quoted evidence — instead of a free-form verdict — turns the question of whether the model hallucinated into something that can be checked mechanically in the next stage. Any quote that does not appear verbatim in its source is discarded: an invented quotation is worse than none. The model is served locally by default (`llama3.2:3b`, a 3-billion-parameter model chosen for its footprint, not by a benchmark), and any OpenAI-compatible provider can take its place by configuration.

If the model cannot be reached after its retries, the claim is marked `llmUnreachable` and reported apart from the verdicts, as a failed search is. If there is no evidence at all, the model is not asked: its answer would be discarded by the next stage anyway, and the call is the slowest step of the pipeline.

4.6.6 Confidence Recalibration

This is the most important safeguard of the system, and the reason a modest model can be used without compromising the reliability of the result. Its rules do not depend on the model's own judgement of itself, which is known to be poorly calibrated [16].

1. **No evidence, no verdict.** Below the evidence minimum the verdict is forced to `UNVERIFIED` with confidence zero, and the model is never asked.
2. **Confidence is recomputed.** The final confidence is 0.7 times the model's confidence plus 0.3 times an evidence quality score computed independently: the mean relevance of the retained evidence (half of it), the share of that evidence the model cited (three tenths) and the amount of evidence against the maximum expected (one fifth).
3. **An ungrounded verdict is downgraded.** A definitive verdict that cites nothing, although evidence was available, becomes `UNVERIFIED` with its confidence capped at 0.4. Citing only sources the model itself called *unrelated* counts as citing nothing.
4. **A contradicted detail makes it partial.** A `TRUE` verdict while some source contradicts the claim becomes `PARTIALLY_TRUE`: the central fact holds and a detail does not, which is the ordinary shape of a real verification.

5. **One outlet is not confirmation.** A `TRUE` that rests on fewer than two independent domains keeps its verdict with its confidence capped at 0.6.

The citation rules are independent of the model: they protect against the hallucinations of any model, present or future, without retraining or prompt changes. They were tested against a fake model that bluffs in 83 ways — citations out of range, repeated or of the wrong type, fabricated or stitched quotes, verdicts outside the set, confidences of 5 or of infinity, malformed answers — which found and fixed six ways through (Section 5.9). Three remain, pinned as known failures because each needs a policy rather than a fix: a source the model calls both supporting and unrelated still grounds its citation; a `FALSE` whose every cited source supports the claim stands; and a `TRUE` whose every cited source contradicts it becomes only `PARTIALLY_TRUE`.

4.6.7 The Second Look

The first search for a claim is planned from the claim alone: who and what it names, what it asserts, and its refutation (Subsection 4.6.3). When those three queries find nothing usable the claim ends `UNVERIFIED`; when they find something thin, the model answers with low confidence. In both cases the model's own explanation usually says what was missing — "no source reports the 2024 figure", "the sources describe a different programme". The second look turns that sentence into a second search.

1. **Follow-up queries.** The model is given the claim, the first verdict, its explanation and the titles of the sources it had, and asked for at most three queries, in the claim's language, aimed at what was missing: the original study or report, an official figure, the organisation's own announcement, an independent report of the same event. A model that writes none gets the claim itself as its query, a sentence the first search never sent whole.

2. **Only new pages.** The queries go through the same search, with the same concurrency and fusion, and only pages the first pass never saw, on domains it did not already have, are kept; the article itself never is.

3. **One judgement over both.** The first pass's evidence and the new pages are ranked, gated, judged by the model and recalibrated together, by the same code as the first pass. The second verdict replaces the first, which stays on the claim with the reason the second look was taken and the queries it sent, so a reader can see that the verdict came from a second search.

It fires only when a run enables it, and only on a verdict in doubt: `UNVERIFIED`, or a confidence below a threshold (0.5 by default). It never fires on an outage — with no search or no model, a second search meets the same outage — and never twice. It follows the iterative retrieval of FIRE [43] and the question generation of AVeriTeC's baseline [33], reduced to one extra round: on a processor without a graphics card each round costs a search, a handful of page fetches and two model calls.

Whether it should be on is not assumed. It is decided on the tuning half of the custom set (Subsection 6.3.8) by the tuning loop of Subsection 6.4.6, and measured on the test half.

4.6.8 Article-Level Aggregation

An article's verdict is neither an average nor the verdict of its most important claim: it is the worst of its claims' verdicts, in a fixed order of severity,

$$\texttt{TRUE} < \texttt{PARTIALLY_TRUE} < \texttt{UNVERIFIED} < \texttt{MISLEADING} < \texttt{FALSE}.$$

`PARTIALLY_TRUE` sits below `UNVERIFIED` because a claim whose detail is off has at least been checked. The article's confidence is the mean of its claims' confidences. An article is publishable when it was admitted and its verdict is neither `FALSE` nor `MISLEADING`: `UNVERIFIED` does not block publication, because with a small local model it is the common outcome, and the reader view shows each claim's own verdict.

Known limitation. Because `UNVERIFIED` outranks `TRUE`, a single claim without enough evidence decides the article's verdict however many others were confirmed, and the mean confidence mixes confidences of different verdicts. Separating "how much could be checked" from "what was found" is left open (Section 9.3). The reader view works around it for the reader: a claim whose search or model failed reads "not checked" and is counted apart, and an article none of whose claims was judged is shown as not checked rather than as unverified.

4.7 Concurrency Design

Checking one article used to be sequential end to end: per claim, the searches one after another, the page fetches one after another, some twenty embedding calls one at a time and one model call, then the next claim. A four-claim article spent minutes waiting, and none of that waiting was on the processor. Three changes, in the order they were worth making, removed most of it: the embedding calls were batched; a claim's queries and page fetches, and the web search and the corpus search, were run in parallel; and claims were run in parallel with one another.

The design rests on four rules.

1. **A claim's own stages stay sequential.** Retrieval feeds ranking, ranking feeds the model, and the model's answer is what recalibration works on. The concurrency is between claims, never inside one.

2. **Limits belong to the resource.** Fan-out multiplies: three articles times four claims times eight page fetches is 96 simultaneous requests from one machine, enough for the search engines behind SearXNG to start refusing it. The real ceilings are therefore one semaphore per external service — two concurrent searches, one inference call, two model calls by default (one on a CPU), eight page fetches, two browser pages — applied inside the client that talks to each service. They are process settings, not per-run thresholds: a

per-request override would let one caller decide how hard the process leans on a service every other caller shares.

3. **A permit is held only around a single call.** Holding one while waiting on work that needs the same resource is the bounded-pool deadlock, in which every worker holds what every other is waiting for.

4. **Results come back in input order.** The evidence list's positions are what the model cites by number and what the response, the cache and the lake record. Results in completion order would silently point every citation at a different source — a wrong answer, not a crash. A test pins it.

Threads were chosen over asynchronous code because everything the pipeline calls — the HTTP clients, trafilatura, the embedded vector store, the model client — is synchronous, and the workload is waiting on input and output, for which threads are adequate. Progress events from several claim threads are serialised in one place, so the code that receives them keeps a single-threaded contract, and each event carries the index of its claim, so a client can follow claims whose events interleave.

The speed-up was measured against a simulated network at the declared ceilings: 3.4× on a four-claim article with a fast model, and 1.16× with the production CPU model at one call at a time, where the model takes 180 of the 184 seconds and nothing else is left to overlap. The simulation's 184 s matches the production stack's real run of the same article.

4.8 Interface Design

The interface serves three audiences, and its pages are grouped by them.

The newsroom. Five pages follow the editor's work in order, with a step bar on each:

1. **Discover**: pick one to three topic groups, set the balance between positive impact and topic relevance, and read the ranked candidates of a round, each with its place, score and the parts of its score. The AI selection can be asked for; its picks arrive ticked with a score and a reason, and stay editable. At most twenty are sent to analysis. The round is in the page's address, so a reload resumes it.

2. **Analyze**: paste article URLs and follow each analysis as it happens; claims appear as rows the moment they are selected and fill in with their own verdicts as they finish.

3. **Live**: a second screen that follows any run from any client — the queries sent, every source found and the engines behind it, the rating each received, and the verdict.

4. **Write**: start a new article from a publishable one, follow it while it is written, and review, rewrite, edit, approve and publish it (Section 5.10).

5. **Reader**: what a reader would see — the published articles first, then the checked news.

The lab. One page per stage, for inspecting and debugging it in isolation: a single claim to a verdict with the model's answer before recalibration; the enrichment of a URL or a text; the text corrector; the scraper's request statistics, source health and time to reception; a health check of every source; the graph, with its schema, a read-only query console and related articles; and the evaluation's state — the labelled set against its targets, the claim selector's precision and the harness's runs with their intervals.

The reader. The public reader view lists the articles the pipeline marked publishable, filtered by topic and language. An article shows each claim's verdict and confidence, the model's reasoning, and its sources with their stance, the quoted passage, the site's reliability (or "unrated") and why each was found. A claim whose search or model failed reads "not checked" and is counted apart from the verdicts. Only the article's lead and a link are shown, never its body or a scraped page (Section 8.2.1).

The design follows two rules. Uncertainty is shown, not hidden: every score carries its parts, every rating says whether it is real or a default, every outage says so. And the interface is a client of the same API the evaluation uses, so what a person sees is what is measured.

Chapter 5

Development

5.1 Technology Stack

Table 5.1: The technology stack and why each piece was chosen.

Layer	Technology	Why
Backend	Python 3.12, FastAPI, `uv`	The NLP and scraping libraries are Python; FastAPI gives typed request models and background jobs; `uv` locks dependencies reproducibly
Frontend	Next.js 14 (App Router), TypeScript	Server-side API proxies keep the API key out of the browser; a hand-written CSS design system, no UI library
Services	Docker Compose	The same stack in development, evaluation and production
Search	SearXNG (self-hosted)	Free, no key, and a closed list of engines chosen and measured one by one
Extraction	trafilatura [3], BeautifulSoup, Playwright (optional)	A cascade from cheapest to most expensive (Subsection 5.2.7)
Entities	GLiNER (`gliner_small`) [44]	Entity types given at run time, no training; the smallest of the three models
Sentiment	`twitter-xlm-roberta-base-sentiment` [4]	Multilingual, covers English and Spanish with one model
Embeddings	BGE-M3 [6]	Multilingual, 1,024 dimensions; one space for every similarity in the system
Keywords	YAKE [5]	Unsupervised, per-language stopwords
Language model	`llama3.2:3b` through Ollama	About 2 GB, fits the development GPU; OpenAI-compatible, so replaceable by configuration
Vector store	Qdrant (embedded)	No server to run; one collection of article embeddings
Graph	Neo4j Community	A property graph with a query language; read-only console for exploration
Infrastructure	Terraform, Google Cloud, Caddy	One virtual machine running the same compose stack; HTTPS and a site password in front

The language model. The default model was `llama3.1` (8 billion parameters, about 4.9 GB) until September, when it was replaced by `llama3.2:3b` (about 2 GB). The change

was made for resources — the smaller model fits the development GPU and a CPU-only virtual machine — not because it was measured to be better. Measuring verifier models is the purpose of the model benchmarks (Section 6.6); until then the choice is a resource decision and is reported as such.

No linter. The repository has no linter or formatter configured; the decision was recorded rather than left implicit. The rules that are costly to break are enforced as tests instead (Section 5.9).

5.2 News Acquisition: Discovery and Extraction

Every article the system analyses, and every web page it reads as evidence, is obtained by the acquisition layer described in this section. Its job is narrow and adversarial at the same time: find the articles a configured news source has published recently, and turn any URL into clean article text with its title, author and publication time, while the servers on the other side are slow, inconsistent, occasionally hostile to automated clients, and never under the system's control. The layer is implemented in `backend/src/services/scraper/` and is shared by every part of the pipeline that needs a page.

5.2.1 Overview

Acquisition is two steps, composed by the `Scraper` class (Figure 5.1):

1. **Discovery** (`DiscoveryService`) takes a configured source and returns the URLs of its recent articles, together with whatever the source says about each one before it is opened: a title, a summary and a publication time.
2. **Extraction** (`ExtractorService`) takes one URL and returns the article: its body, title, author, date and, when the page states it, the publication time to the minute.

The two steps are used separately. Discovery feeds the candidate rounds from which a person or the AI chooses what to analyse (Section 4.4), the one-shot ingestion endpoint and the daily labelling batch. Extraction is called for every chosen candidate, for every URL posted to the analysis endpoint, for every web page read as evidence for a claim, and by the tools that check the sources' health. Each call states its *purpose* (`article`, `evidence`, `enrichment`, `ingestion`, `discovery`, `source_check`, `probe`, `labelling`), which decides how much the call may spend and under which heading it is counted.

Four principles run through the implementation, each introduced after a measured failure that is cited where it is described:

- **Cheapest first.** Discovery and extraction both try their strategies in order of cost, and only go to the next one when it could plausibly succeed where the previous one failed.
- **Every fetch is guarded.** Every URL is checked against an address-based guard before any request is sent, and again at every redirect.

- **Everything is counted.** Every page wanted is recorded with its outcome, the strategies tried and the requests actually sent, so a source that stops working shows up as such rather than as an absence of news.
- **Sources are configuration, not code.** Adding a source means adding a YAML file; nothing in the code names a particular outlet.

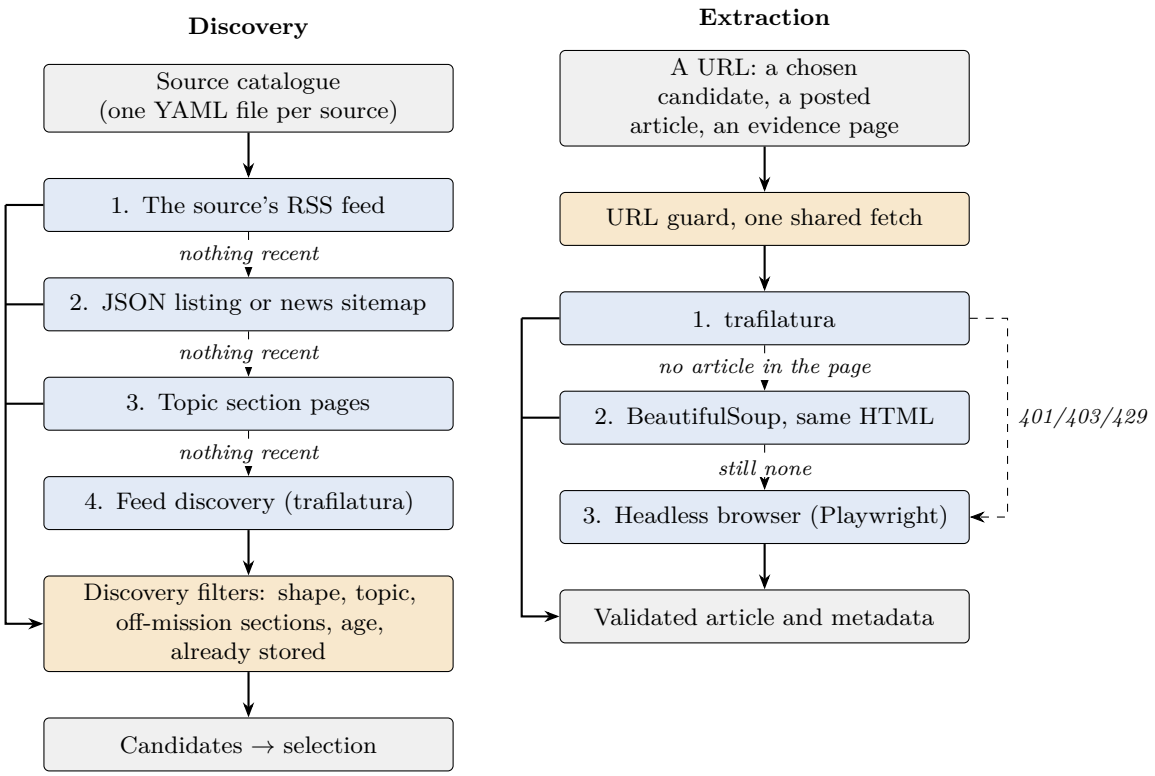


Figure 5.1: The acquisition layer. Solid arrows are the normal path; dashed arrows are escalations, taken only when the previous step could not help. A missing page, a timeout or a guard refusal stops the extraction at once.

5.2.2 The Source Catalogue

Each news source is one YAML file in `backend/data/sources/`, loaded by `SourceRepository` into a `NewsSource` model. Only an identifier, a name and a base URL are required; the fields that matter to acquisition are listed in Table 5.2.

Composition. The catalogue holds 37 sources, 21 in English and 16 in Spanish: 27 news outlets, four public bodies (NASA, the European Space Agency, the World Health Organization and UN News), four academic publishers (MIT News, *Nature*, The Conversation and Yale Environment 360) and two blogs. Three of them (Good News Network, Positive News and Reasons to be Cheerful) publish only positive news. A fourth solutions outlet, The Optimist Daily, was added on 9 October so that the health group had a source that chooses solutions stories; it is not treated as positive-only, because its feed also carries lifestyle tips. Appendix B lists them all.

Table 5.2: Fields of a source that acquisition reads.

Field	Use
`rss_url`	The feed read first by discovery.
`language`	`en` or `es`; selects the keyword lists and section names used to filter its links.
`groups`	The topic groups the source has real sections for (Section 5.2.5).
`reliability_index`	Editorial reliability in $[0, 1]$, used when its pages are ranked as evidence and when its candidates are ranked.
`enabled`	Whether the source is ingested at all.
`requires_javascript`	Sends its articles to the browser first (Section 5.2.7).
`positive_editorial`	The outlet publishes only positive news; the mission screen does not apply to it.
`metadata.selectors`	Optional CSS selectors for the title, body, author and date, for layouts no generic parser understands.

Enabled and disabled. 33 sources are enabled, 20 in English and 13 in Spanish. A disabled source is not ingested, probed or recognised from a posted URL, but it is still read by the reliability map, so its pages keep their rating when they are found as evidence. That is the mechanism for rating a domain without ingesting it, and it is used for two different reasons. The fact-checkers Newtral and Maldita are disabled because a fact-check quotes the claim it debunks, and claim extraction would take that quotation for the article's own claim. Agencia EFE and Reuters are disabled because they refused every request from 23 September to 6 October 2026 (HTTP 403 and 401 respectively).

One source per domain. Reliability and the recognition of posted URLs are both looked up by domain, so a second file on the same domain (BBC Mundo on `bbc.com`, for instance) would silently override the first. A test of the repository fails on any such pair.

How a source is admitted. The catalogue grew from 12 to 36 sources on 28 September 2026, and every addition passed the same test before its file was committed: its feed had to produce article links, and two of those articles had to extract through the real cascade. Seven candidates failed it: NPR, Phys.org and El Confidencial (the feed worked, the articles would not extract), ScienceDaily, DW Español and CSIC (no article links in the feed) and the Associated Press (HTTP 403). RTVE's dead feed was replaced the same day. The 37th, The Optimist Daily, passed the same test on 9 October.

5.2.3 The Shared Fetcher and the URL Guard

One fetch per page. Every HTTP request of the acquisition layer goes through one class, `Fetcher` (`services/scraper/fetcher.py`). It sends a single `GET` with a 20-second timeout and a User-Agent that names the system (`InspiringNewsBot/1.0`), and returns the page as the server sent it. The HTML parsers of the extraction cascade share that page: before this, each strategy fetched for itself, so falling back to a second parser doubled the requests sent to exactly the sites that were already the hardest to scrape. Only a strategy that needs something plain HTML cannot give, a rendered page, goes back to the network.

The URL guard. All fetching happens on the server, and the URLs come from outside: the analysis endpoint takes them from its caller, and evidence retrieval from whatever the search engine returned. Without a check, the service would be an HTTP proxy into its own network. In the development stack that network contains the graph database and the search engine, and on a cloud host the instance metadata endpoint is one request away. The guard (`services/scraper/url_guard.py`) is therefore applied to every URL before it is fetched, and it is deny-by-address rather than allow-by-domain: an allow-list of news domains would defeat the purpose of an endpoint that accepts any article the user found, while what must never be reachable is the infrastructure, which is a question of address space. A URL is refused when:

- its scheme is not `http` or `https`, or it has no host;
- its port is one of twelve ports used by internal services and never by a public web page (SSH, SMTP, SMB, PostgreSQL, MySQL, Redis, Neo4j's two ports, Elasticsearch, Memcached, MongoDB and Telnet);
- any address its host resolves to is private, loopback, link-local (where cloud metadata services live), reserved, multicast or unspecified. The name is resolved by the guard itself rather than trusted, because a public name can have the address `127.0.0.1`, and a name that resolves to both a public and a private address is refused.

A host that does not resolve at all is still refused, but counted as a connection failure rather than as a block, since a mistyped or dead domain is not the guard protecting anything. An explicit list of allowed hosts, empty by default, exists for staging servers on a private network, so that nobody has to switch the guard off to reach one.

Redirects. An HTTP library that follows redirects by itself would walk straight past the guard: a public URL can redirect to `127.0.0.1`. The fetcher therefore follows redirects itself, one hop at a time, checking each new location with the guard, and gives up after five.

Outcomes. A failed fetch is classified once, in the same way for discovery and for extraction, so that a 403 on a feed and a 403 on an article are counted alike. Table 5.3 lists the outcomes; only `ok` produced an article.

Table 5.3: Outcomes of an extraction attempt.

Outcome	Meaning
`ok`	An article was extracted.
`too_short`	Parsed, but shorter than the run's minimum body length: a paywall stub, a cookie wall, a gallery.
`no_content`	Fetched, but no article text was found.
`http_error`	The server answered 4xx or 5xx (the status is kept).
`timeout`, `connection_error`	The server did not answer, or could not be reached.
`blocked`	Refused by the URL guard; never sent.
`unavailable`	The strategy cannot run here (no headless browser installed).
`error`	Anything else.

5.2.4 Discovery Strategies

Discovery tries up to four strategies per source, cheapest first, and stops at the first that returns recent links (Table 5.4). Every attempt is recorded under the purpose `discovery`, so a feed that starts failing appears on the monitoring page next to the articles it no longer delivers.

Table 5.4: Discovery strategies, in the order they are tried (since 9 October 2026).

Strategy	Tried when	Gives per link
1. RSS feed	Always, if the source has a feed URL.	time, title, summary
2. JSON listing or news sitemap	The feed has nothing from the last 30 days, and the source names one.	time, title
3. Topic section pages	Still nothing recent: the source's own section paths, or those its homepage links to, or guesses.	title
4. Feed discovery	Still nothing: a feed is searched for on the feed URL and then on the homepage.	URL only

The RSS feed. The feed is downloaded through the shared fetcher, so it is guarded and has a timeout (the feed library's own HTTP client has neither), and is then parsed with `feedparser`. Each item keeps its publication time (in UTC), its title and a summary reduced to 400 characters of plain text: what the candidate list shows and the AI selection reads before any article is opened. On 4 October 2026, all 1,248 URLs from the 29 feeds that answered carried a time.

JSON listings and news sitemaps. Two sources' feeds froze: the World Health Organization's newest item was from February 2026 and CNN's feed stopped in 2024. A source can name a JSON listing (WHO's news API, about thirty items a week) or a Google News sitemap (CNN's, 268 articles of the last two days, with titles and times), read only when its feed has nothing recent and handed to the same filters in the feed's own shape.

Topic section pages. Four configured feed URLs were found to answer 404 (National Geographic, Reuters, RTVE and SINC), and their homepages advertised no other feed, so those sources produced nothing although their sites were up. Every news site files its articles under sections, and the sections this project wants are the ones its topic configuration already names. The third strategy (`strategies/topic_pages.py`) reads those section pages as HTML and keeps the article links on them:

1. first, the section paths the source's own configuration names per topic group (Smithsonian's Environment candidates had all come from a guessed `/food/` before it named its own);
2. then the sections the homepage itself links to (a short path, at most three segments, ending in a topic's section name), so that nothing is spent on a guess;
3. then guesses, the base URL followed by a section name, one per topic, with at most six

section pages per source in all, and none when the homepage already refused the request (EFE and Reuters refused every guess in the same way they refused the homepage).

Spanish sources also get Spanish section names (`/ciencia/`, `/salud/`, `/cultura/...`), kept in a separate table on purpose, because the English table also decides whether a feed link looks like an article, and that behaviour was already settled on the English feeds. A link is kept when it is on the source's host or one of its subdomains (NASA's science sections live on `science.nasa.gov`), is not itself a section, and passes the same article-shape test as feed links (Section 5.2.6). Each link carries the headline it is linked with on the section page (94 of ABC's 96 links, 59 of France 24's 59); only a link with no text gets a title built from its slug, marked as such. A section page gives no summary.

Feed discovery, last. `trafilatura`'s feed discovery looks for a feed on the configured feed URL and then on the homepage. It was the second strategy until 9 October; run for real it did worse than section pages — ABC's homepage feeds gave a Culture round 38 untitled links from every section, and WHO's dead feed 21 items from 2024 and 2025 — so it is now tried last, drops off-mission and live links, and holds a narrowed Spanish source to a topic word in the link's slug.

Stale feeds. A feed whose items are all older than the age limit counts as empty, so the next strategy runs. RTVE's feed has carried only items from June 2022 since then; until this rule existed, a Culture round listed two of them.

Concurrency. Sources are discovered four at a time: feeds are other people's servers and several are slow (two timed out in the check of 23 September), and four at once keeps a round to seconds without leaning on anyone. Results come back in the order the sources were given, whatever order they finish in.

A regression found in use. From 4 October, when discovery started asking every strategy for publication times, the topic-page strategy inherited the RSS strategy's feed reader and read the dead feed again instead of crawling the sections: National Geographic, Reuters and SINC found nothing for a day. It was fixed on 5 October, with a test that runs the strategy through `DiscoveryService` as discovery does rather than in isolation. The same bug was found again on 9 October in the step beside it: `trafilatura`'s feed discovery had inherited the feed reader too and had not run since 4 October, which is how it came to be measured and moved last.

5.2.5 Discovery by Topic Group

Until 5 October 2026, ingestion read every enabled source and sent every new article it found to a full analysis: a scrape, an enrichment and one language-model call per claim, for each. Discovery is now scoped to what the editor asks for.

Five groups. The 23 topics of the topic classifier are organised into five groups: society, science, environment, culture and health, the same groups over which the custom validation set is balanced (Section 6.3). They are defined once, in `src/config/topics.py`. A round asks for groups rather than topics because several topics are covered by only one or two sources, so a topic picker would often return nothing; and it asks for at most three, because four of five is close to everything, which is what scoping exists to avoid.

Which sources a group reads. Each source file lists its `groups`: a decision taken by hand from what the source actually publishes, not derived from its free-form tags, so a generalist outlet names only the groups it has real sections for. Tests hold every enabled source to at least one group, and every group to at least one English and one Spanish source, so no choice of groups returns a list in one language only; an unknown group name is refused when the file is read. Table 5.5 shows the coverage of the enabled sources.

Table 5.5: Enabled sources per topic group and language. A source may cover several groups.

Group	English	Spanish
Society	9	9
Science	11	11
Environment	12	3
Culture	4	5
Health	11	7

Only the sources that name a requested group are read, and only for the topics of the requested groups: the section names searched, the keywords matched and the links accepted all follow from those topics.

What it saves. The discovery experiment (`python -m src.evaluation.experiments discovery`) measures each setting against a baseline that reads every source for every topic. On 5 October 2026, with two candidates per source, Environment alone read 14 of the 34 sources then enabled (0.41×) and 153 of 1,536 links (0.10×), in 10 s against 15 s; Culture and Health together read 23 sources (0.68×) and 596 links (0.39×). Feeds change during the day, so settings are compared within one run, and repeated on several days for an interval (Section 6.5).

The table also shows the catalogue's weak spot: Environment has three Spanish sources against eleven English ones, so an Environment round is mostly in English.

5.2.6 What Discovery Keeps

A feed or a section page lists far more than articles worth offering: section indexes, live blogs, opinion columns, sport, items from years ago and articles already analysed. Every link is filtered before any article page is fetched, which is the cheapest point at which to say no. Table 5.6 summarises the rules; each was added after a concrete case, given in the table.

Table 5.6: Rules applied to discovered links, and the case behind each.

Rule	What is dropped, and why
Article shape	A link must look like an article: a date in its path (`/2026/09/23/`, `/20260923/`), a slug of at least four words, or an `/article/` or `/story/` segment. The previous English section-name patterns had kept 0 of El Mundo's 26 feed items, 5 of El País's 149 and 0 of NASA's 10.
Off-mission sections	Sport, celebrity, horoscopes, lotteries, betting, weather, crime (`/sucesos/`) and El País's lifestyle magazine (`/icon/`), as whole path segments, so that `/sports-health/` (fitness) stays. Trigger: a match report scored above an analysis of Chinese industry leaving fossil fuels on the positive-impact score (0.34 against 0.28).
Live coverage and opinion	Live pages are streams of updates, not articles; a column's claims are interpretation (`/opinion/`, `/commentisfree/`).
Index pages	`/categoria/`, `/tag/`, `/especiales/`, `/autor/...`, whose slugs read like headlines.
Section navigation	On a section page every link contains the section, so a link there must look like an article by its own shape: BBC's section pages had given `/culture/music` as a candidate.
Outside the section	A section page's links outside its own section are dropped when it has any inside it: ABC's `/cultura/` page linked the day's election stories.
Another topic's section	A link filed under a section of a topic not asked for, unless the item mentions a topic that was: elDiario files wind-power records under `/economia/`, and those stay.
Topic keywords	English items must match a requested topic's section name or keywords in their title, summary or categories. Spanish items, only when the round is scoped, must contain a Spanish topic word in their title or categories (Spanish summaries of general feeds mention renewables or the political "clima" in campaign speeches).
Age	Older than 30 days, by the feed's time or, without one, by a date in the link (including ABC's slug timestamps). Undated links stay.
Already stored	The URL's host and path are already in the data lake, so tracking parameters do not make an old article new.

Keywords and language. The topic keywords are English. Matched against a Spanish feed they kept 5 of El País's 149 items at random, so Spanish feeds were at first not filtered at all, and an Environment round then listed El País's and elDiario's election coverage. A Spanish keyword list is now read when, and only when, a round is narrowed to some topics, and only against an item's title and categories. Measured on the live feeds on 5 October 2026, Environment kept 18 of El País's 159 items and 6 of elDiario's 109, nearly all on topic. When every topic is asked for (the one-shot ingestion endpoint and the labelling batch), Spanish feeds are not filtered, and topic is left to the admission filter, which rejects before any language-model call.

The age limit. Thirty days was chosen from a sample of 6 October 2026: 38 of 363 dated candidates were older (24 of them from the World Health Organization, back to 2025), while a 14-day limit would also have cut nine MIT News research stories. A link with no feed time can still be dated by its path, which is how ABC's mortgage and income-tax calculators, stamped in May, stopped being offered as news in October. That date is used only for the age check: a day is not a publication time. The labelling batch has its own discovery and is not cut by this limit; it prefers articles from the last 60 days instead.

How many per source. At most `perSource` new articles are taken from each source per run (three by default); the rest are reported as deferred rather than silently lost. Before that cap, the mission screen reads up to five times as many of each source's new articles and leaves out those nearer politics, crime, disasters, war, celebrity, sport, markets or service pages than any topic, so that a source's slots go to what passes. The screen, the ranking of the candidates and the choice among them are part of selection, described in Section 4.4.

5.2.7 The Extraction Cascade

`ExtractorService` (`services/scraper/extractor.py`) tries three strategies in order of cost (Table 5.7).

Table 5.7: The extraction cascade.

Step	Strategy	Cost
1	trafilatura	One HTTP request.
2	BeautifulSoup	None: it parses the HTML step 1 already fetched.
3	Headless browser	A second request, a full render with JavaScript, seconds; never for evidence pages.

When to escalate. A failed step only passes the page on when a different strategy could plausibly succeed (Table 5.8). A dead source therefore costs one request per URL, not one per strategy.

Table 5.8: What happens after a failed step.

Outcome of the step	Next
No article in the page, too short, parse error	The next strategy; an HTML parser gets the same page.
HTTP 401, 403 or 429 (a bot wall or a rate limit)	Skip the HTML parsers, which would be refused the same way; go to the browser.
Any other HTTP error (404, 500…), timeout, connection error, refused by the guard	Stop: no strategy can fix a page that is not there.

Step 1: trafilatura. The main extractor, chosen because it isolates the article body from navigation, comments and advertising on most news layouts without any per-site configuration. On the 23 September 2026 check of two articles per readable source it produced the article for every source that answered.

Step 2: BeautifulSoup. Written for the layouts trafilatura misses, it looks, in order, at:

1. the CSS selectors in the source's own file, for layouts nothing else guesses;
2. what the page declares about itself: the JSON-LD `NewsArticle`, whose `articleBody` often holds the full text behind a teaser, and the meta tags;
3. the usual containers (`article`, `[itemprop=articleBody]`, `main`), and finally whichever element holds the most paragraph text of its own.

On the same pages it extracted comparable text, but on a CNN live blog it picked a much smaller block, which is why it comes second.

Step 3: the headless browser. Some pages build their text with JavaScript, and some servers refuse clients that do not behave like a browser. The third strategy renders the page with Playwright and Chromium, and then reads the rendered HTML with the same two parsers, so there is one notion of where an article is, not a third one written against the browser's API. It is restricted in four ways:

- **Never for evidence.** Evidence pages are read by the handful for every claim and already fall back to their search snippet, so rendering them would multiply the cost of the most expensive step by the most frequent purpose for very little. The browser is reserved for articles, ingestion, the enrichment page, the labelling batch, and the health checks that ask what ingestion would get.
- **Guarded like every other fetch.** The start URL is checked before a browser is launched, every request the page makes goes through the URL guard, and the redirect chain is checked after the page lands; content that arrived by way of a forbidden address is discarded.
- **Capped.** At most two renders run at once in the whole process (`BROWSER_MAX_CONCURRENCY`), one Chromium per render, because the browser's synchronous objects are bound to the thread that created them and extraction runs on many threads.
- **Optional.** The browser is an optional dependency, installed in the Docker image and usually absent on a development machine. Without it the attempt is `unavailable`, and the extraction keeps the previous step's outcome with a note that it was not escalated, so the statistics still say why the page failed.

It was checked live on 23 September 2026: a page whose text is built by JavaScript has 29 characters of text in its HTML; trafilatura's result is refused as too short, BeautifulSoup finds nothing, and the browser renders 1,097 characters, for two requests and 11 seconds on a cold start. An ordinary NASA page never gets past trafilatura: one request, no browser.

Routing by source. A URL posted for analysis arrives without a source, so an El País article used to be treated as a generic web page. The extractor now matches every URL against the enabled sources by domain (the source's host or a subdomain of it, the most specific match winning), so that the source's selectors and browser flag apply and the stored article carries the real source. Setting `requires_javascript` puts the browser first, because the two cheap steps are known to fail for that site and would each cost a request to find that out again; they still run after it, for when no browser is installed. The flag is meant to be set from evidence rather than guessed: the monitoring page flags a domain whose every article came from the browser. No source has needed it so far, and El País, the case it was built for, answers 403 to the headless browser as well.

Validation. An extraction is accepted only when its body reaches the run's minimum length (500 characters by default, a per-run threshold like every other tunable of the pipeline), which turns paywall stubs, cookie walls and photo galleries into `too_short` rather than into articles with nothing to check.

5.2.8 Metadata and Publication Time

Metadata from the same page. A strategy can find the text of an article and still miss its title, author, date, summary or lead image. When that happens, the missing fields are read from the same HTML with BeautifulSoup, JSON-LD first and then meta tags, at no extra request, and never overwriting what the winning strategy found. The rule came from an incident: trafilatura 2 leaves the title, author and date out unless asked for its metadata, and the strategy's tests mocked the call, so every article in the data lake had been analysed with an empty title, and every retrieval fallback built on the headline was dead. One test now runs the extractor for real. On the probe of 25 September 2026, 53 of the 60 sampled articles extracted, all of them with a title, 52 with a date and 44 with an author.

Publication time. Both extractors reduce the publication time to a date, and discovery originally threw away the feed's time, so the delay between publication and reception could only be measured in days. Since 4 October 2026 three times are kept:

- **The feed's time** for each discovered URL (Section 5.2.4).
- **The page's own time**, to the minute, when the page states one (`article:published_time`, the JSON-LD `datePublished`, or a `<time datetime>` element). A bare date is not a time, and neither is exactly midnight, which is how date-only sites write it. This costs one more parse of the page when trafilatura won, so it is done only for articles that are stored, not for the dozens of evidence pages a claim reads. Six of seven live articles sampled that day stated their time to the second.
- **When ingestion first saw the URL**, recorded by host and path in the lake's statistics (`lake/stats/sightings.json`), deferred URLs included.

Section 5.2.10 describes the report built from them.

5.2.9 Reading Evidence Pages

The same extractor reads the web pages that evidence retrieval finds for each claim (`services/fact_checker/retrieval/scraper.py`), rather than a second scraping stack. A search result has no configured source, so it is extracted with a generic placeholder source (neutral reliability, never stored), and the extractor still matches its domain against the catalogue, so a hit on a configured outlet is read with that outlet's selectors. Three things differ from article extraction:

- **No browser and no publication time.** Evidence is the most frequent purpose and already has a fallback, so the expensive steps are left out (Section 5.2.7).

- **A failed page keeps its search result.** When a page cannot be read, the evidence keeps the title and snippet the search engine returned, and the claim is judged on that rather than losing the source.
- **Concurrent, and in input order.** A claim's pages are fetched at once instead of one after another; this was the pipeline's longest step apart from the language model, with one slow host holding up the others for no reason. A process-wide ceiling of eight concurrent fetches (`SCRAPE_MAX_CONCURRENCY`) is held around each fetch only, because several claims and several articles reach this point at once. The pages come back in the order they were requested, not the order they finished: the list is ranked, cut and handed to the language model as a numbered list that it cites by index, so completion order would silently point every citation at a different source (Section 4.7).

How many search results are read per claim is itself a per-run threshold (`evidence_fetch_candidates`, eight by default), because widening recall is the first thing to try when a claim comes back `UNVERIFIED` for lack of evidence.

5.2.10 Monitoring the Sources

A source that stops delivering does not raise an error: ingestion simply finds nothing new. Three tools make the state of every source visible, from cheapest to most thorough, and a fourth measures how quickly articles arrive.

Request statistics. Every page wanted is one entry, not every strategy attempt: its final outcome, the strategy that produced the article, every strategy tried, the HTTP requests actually sent (none for a guard refusal, one for trafilatura and BeautifulSoup together, two once the browser steps in), the purpose and the domain. The counts are thread-safe, persisted in the lake, and shown on the `/scraper` page, where "which step does the work for this source" and "why did it fail" can be read per domain.

The source probe. The statistics only know about pages someone happened to ask for. The probe (`POST /scraper/probe` and the "Source health" panel) asks on purpose, for each enabled source: its feed (both feed steps), its topic section pages (always, since how many links they add is part of the answer), and a real extraction of a sample of the links found, split between the two. It classifies each source as:

- **up**: the feed gives links and at least half of the sample extracts;
- **degraded**: articles can be had, but not the normal way: a dead feed rescued by section pages, or under half the sample extracting;
- **down**: nothing extracts.

It also sends one query per language to the search engine and reports which engines answered, since evidence retrieval depends on them. A probe costs ten to fifteen requests per source, so

it runs only when asked, on a background thread (a second request while one is running is refused), behind the storage API key, and keeps its last report in the lake.

The source check. The probe answers "is the source up"; the check (`POST /sources/check`, the `/sources` page) answers "does every configured page still give a complete article". It runs discovery and then extracts a few links through the same cascade as ingestion, browser included, stores and analyses nothing, and includes disabled sources. A source is **working** when every sample extracted with a title, author and date, **partial** when some failed or one lacked those fields, and **broken** when discovery found nothing or no sample extracted.

Time to reception. The freshness report (`GET /scraper/freshness`) joins the three times of Section 5.2.8 with the lake's own records and reports, per article and per source, four delays in hours: **seen** (publication to first sighting: the feed's delay and the system's polling), **queue** (first sighting to fetch: the system's backlog), **reception** (publication to fetch) and **available** (publication to the reader view). Only precise times are used; an article dated only by day is reported in days, apart. A negative delay (a wrong time zone, a rewritten feed time) is counted rather than averaged in. Because ingestion runs only when someone asks for it, "seen" currently measures mostly the time until the next round; queue and available are the pipeline's own.

5.2.11 Verification Against the Live Sources

Unit tests show that the code does what it was written to do; only the real sources show whether that is enough. The acquisition layer was checked against them at each stage of its construction (Table 5.9).

Table 5.9: Checks of the acquisition layer against the live sources (2026).

Date	Sources	Result
23 Sep	12	Discovery went from 4 sources producing links to 9. One NASA article went from discovery to stored analysis in 52 s, and the next run reported it as already stored.
24 Sep	12	Source check: 6 working, every sample with title, author and date; 6 broken (five discover nothing; El País gives 147 links and answers 403 to every article, headless browser included).
25 Sep	12	First probe: 6 up, 3 degraded (dead feeds rescued by section pages: 113, 115 and 64 links, 6 of 6 articles each), 3 down (EFE 403, Reuters 401, El País 403 on articles). Feeds 395 links, section pages 1,272 more; 53 of 60 articles extracted.
29 Sep	34	Probe: 29 up, 4 degraded, 1 down; 156 of 165 sampled articles extracted (95%). Feeds 1,314 links, section pages 3,604 more.
4 Oct	29 feeds	All 1,248 discovered URLs carry a publication time.
6 Oct	2	EFE and Reuters disabled: in the container's statistics since 23 September, 21 and 24 attempts, none delivered.

Three observations follow from these runs.

Section pages are not a fallback of last resort. On both probes the section pages found several times more links than the feeds (1,272 against 395, then 3,604 against 1,314). They

are still tried last, because a feed gives a time, a title and a summary for every link, and a section page gives none of them.

What fails is access, not parsing. Every article that extracted on 25 September was extracted by trafilatura, the first step. The failures were refusals (403, 401) and dead feeds, which is why the cascade spends its effort on deciding when to stop rather than on more parsers. Of the 29 September failures, El País gave 2 of 5 (the other three answered 403, and that host had no Chromium to escalate to), Reuters refused everything, and Positive News' two failures were section pages taken for articles, not failures of the site.

Diagnoses can be wrong in instructive ways. The four dead feeds of 23 September had first been read as "malformed XML": the feed library was parsing the HTML of the 404 page. Classifying the outcome of the fetch before parsing it (Section 5.2.3) is what made the real cause visible.

5.2.12 Testing

The acquisition layer has 181 unit tests in `backend/tests/services/scraper/`, part of the suite that `./scripts/check.sh` runs. None of them reaches the network: pages are served by fake fetchers, the HTTP client and the browser are replaced by stand-ins, and name resolution is patched for the URL guard. Table 5.10 lists what each module covers.

Table 5.10: Unit tests of the acquisition layer, by module.

Module	What it holds
Discovery	The order of the strategies, the age limit and stale feeds, topic and language filters, the article shape, and the section-page strategy run through the discovery service.
Extractor	The cascade's stop and escalation rules, routing by domain, metadata filling, and the requests counted.
Strategies	trafilatura and the shared fetch (redirects followed hop by hop through the guard, a redirect loop abandoned, each failure reported as its outcome), BeautifulSoup (selectors, JSON-LD, densest block), the browser step (guard on every request and redirect, unavailable without a browser), and the section-page crawl.
URL guard	Schemes, ports, private and reserved addresses, names resolved rather than trusted, every resolved address checked, and the allow-list.
Probe and check	The up, degraded and down rules; working, partial and broken.
Statistics	Counting per extraction and per domain, and persistence.

Two properties of the catalogue are also enforced as tests rather than conventions: no two sources share a domain, and every enabled source names at least one topic group while every group has an English and a Spanish source. The strategy tests that mocked trafilatura's call are the reason the missing-title incident went unnoticed (Section 5.2.8); one test now runs it for real and checks that the page's title comes back.

5.2.13 Limitations

Refusals are not worked around. El País answers 403 to most article requests, the headless browser's included; EFE and Reuters refused everything for two weeks and are

disabled. The system does not rotate addresses or disguise itself to get past a refusal: a source that does not want automated clients is treated as unavailable. The cost is coverage, and it falls on Spanish news agencies in particular.

`robots.txt` is not read. The scraper names itself in its User-Agent, reads a few articles per source per round, caps its concurrency per external service, and runs only when someone asks for it, but it does not consult a site's `robots.txt` before fetching. Section 8.2.1 discusses what that means for the collection of third-party content.

Section pages give less. A candidate found on a section page has no summary and no publication time. The AI selection scores such candidates low for lack of information, and when the link carries no date in its path either (CNN's and National Geographic's section links), the age limit cannot apply to it. Dating them would cost a request per candidate before anything is chosen.

The guard has two known gaps. The address is checked before the request, so the window between the guard's name resolution and the HTTP client's own (DNS rebinding) is not closed. And in the browser, a forbidden redirect hop is detected only after it has been requested: its response is discarded, but the request was made. Closing either needs a proxy in front of the fetchers.

Unused paths. No source has `requires_javascript` set, the browser is usually absent on a development machine, and the browser-based discovery strategy (`strategies/playwright_discover.py`) is still an unwired placeholder. The browser step has been verified on test pages, not yet on a source that needs it.

Hand-written vocabularies. The section names, the off-mission sections and the Spanish topic words are lists maintained by hand from the cases met so far; a new outlet with different conventions will need entries of its own.

Duplicates by address only. An article already in the lake is recognised by its host and path, so the same story from two outlets, or under two different addresses, is acquired twice; grouping versions of one story happens later, when candidates are ranked (Section 4.4).

5.3 The Inference Service

Why the three transformer models were moved out of the backend is explained in Section 4.2. This section describes how the move was made.

Adapters, not a rewrite. The entity extractor, the sentiment analyser and the embedding service kept their exact public signatures and now delegate to an HTTP client. Every call site was checked before the change: nothing downstream — the enrichment pipeline, the claim

extractor, the topic classifier, the corrector, the retrieval and ranking chain — ever touched a model object, only the methods that return results. Nothing downstream changed.

A model server that knows nothing of the domain. The service loads the models and returns entities, sentiment scores and vectors. Topics, thresholds and scoring stay in the backend: the topic classifier still owns the topic vocabulary and the cosine arithmetic, and only fetches its vectors over HTTP. Tuning therefore stays inside the backend's configuration and lineage instead of being split across two deployable units. The three models share one service because they share a dependency stack, a warm-up and a resource pool.

Failure differs by component, on purpose. If entity extraction fails it returns nothing, and the claim scorer, which treats entities as one weak signal among several, carries on. If sentiment or embeddings fail they raise, and the article fails: sentiment is a core input of the admission decision and embeddings feed duplicate detection, ranking and topic classification, so a silent default would corrupt decisions without a trace.

Readiness. The service reports healthy only once all three models are loaded; the models warm up in a background thread so the server can answer "not ready" honestly instead of blocking its own start. The compose file holds the backend back until then, which turns the old first-request stall into one explicit wait at start-up. The start period is 900 s, measured rather than guessed: a cold model cache took about eleven minutes to become healthy.

Pinned versions. The service pins the exact library versions the backend had resolved. Left open, the resolver chose a new major version of `transformers`, which broke GLiNER's tokenizer loading outright.

Where the CPU time went. A profile of the service on the development laptop's CPU (16 October) found three costs that bought nothing, and one error. The ranker's eight evidence pages were embedded as one padded batch, which on a CPU took 9.4 s against 6.8 s one at a time (7.3 s against 4.2 s with the mixed lengths it really sends), with identical vectors; the service now embeds one text at a time unless configured for a GPU. Enrichment embedded each article twice, once for storage and once for the topic classifier, at 3.0 s a time for a 5,700-character article; the classifier now reuses the vector. And the claim extractor sent one entity request per sentence, 37 for that article; one batched request finds the same entities in 1.5 s instead of 2.7 s. The error was GLiNER's input limit: a whole article was truncated to its first 384 tokens with only a warning, so the article-level entities came from its opening alone — 5 found where its sentences hold 52. Long texts are now read in sentence-bounded chunks. Together, enrichment of that article went from 17.2 s to 14.1 s while reading all of it.

One trap left by the split. The backend's configuration still names the sentiment and embedding models, but the names are documentary: the models load from the inference service's own configuration. Only the embedding dimension, which sizes the vector collection, is still read by the backend, and a test checks it against the live service.

5.4 Fact-Checking Implementation

The design of Section 4.6 is the result of incidents as much as of plans. Each one below was reproduced, fixed and pinned by a test, and each changed the design.

A re-analysed article was a duplicate of itself. Every extraction gives the article a new identifier, and the duplicate check skipped a stored match only when the identifiers were equal. A second analysis of the same URL — a forced refresh, or any changed threshold, since thresholds are part of the cache key — found its own earlier copy at similarity 1.0 and was rejected. Its quieter twin: the corpus search had no exclusion at all, so the earlier copy came back as evidence for its own claims. Both now exclude the article's URL inside the vector query, and the store keeps one point per URL.

The title was never extracted. trafilatura's second major version leaves the title, author and date out unless asked for them. Every article in the lake had no title, so everything built on the headline — restoring a claim's lost subject among it — was dead. The strategy's tests had mocked the library, which is why nobody noticed; one test now runs it for real.

A claim was checked outside its article. A Spanish sentence about August prize contests, from an article about a street game, came back `TRUE` at 84% confirmed by the Mexico City marathon's prize money. The fix restores the article's subject to the claim's queries and gives the model the headline and the opening, with the rule that evidence about a different event is unrelated even when it shares a place, a date or a figure.

No call to the model without evidence. Below the evidence minimum the verdict was already forced to `UNVERIFIED`, yet the model was still called: on 25 September, 23 of 24 claims spent the slowest step of the pipeline on an answer that was thrown away. The call is now skipped, and so is the progress event that told the interface the model was being asked.

The model was retried behind the system's back. The client retried once, and the SDK underneath retried timeouts twice more by default, so a slow local model could hold one claim for six times the timeout and then report it unreachable. The SDK's own retries are off, and a provider that was never reached raises an error that becomes the claim's `llmUnreachable` flag, instead of returning nothing as a malformed answer would.

The model's judgement of a source is enforced. The verifier already asked the model, per source, whether it supports, contradicts or is unrelated to the claim, but a citation of a source the model had just called unrelated still counted. Since 30 September it does not: a definitive verdict resting only on such sources falls to `UNVERIFIED` with a note saying why, at no extra model call.

Guardrails tested against a bluffing model. A fake model — raw text behind the real client — was made to bluff through the real verifier and recalibration in 83 ways. Six got

through and were fixed, each with a test that failed before the fix: repeated citations inflated the confidence (a one-source `FALSE` went from 0.70 to 1.00); a citation of `true` cited source 1; a null citation list, or valid JSON that was not an object, failed the whole article; a quote stitched from a headline and its body passed as verbatim; and a per-run evidence minimum of zero sent a claim with no evidence to the model.

Search failures look like verdicts. The most consequential finding of the project is not in the code: on 25 September, 68 of 69 search queries came back empty, because the engines behind SearXNG were rate-limiting, challenging or timing out, and every affected claim came back `UNVERIFIED`, indistinguishable from a real one. The response was a closed engine list measured one by one (dead engines removed, four academic indexes added at half weight, a three-second timeout), half as many concurrent searches, each repeated query sent once with non-empty answers cached for ten minutes, the `searchUnavailable` state, and a keyless second route. On 1 October the cause of part of the rate-limiting was found in the stack's own health check, which had sent a real search to every engine every five seconds since September — about 17,000 queries a day per engine from an idle stack. It now asks the engine's own health endpoint. In three runs that day every search answered.

5.5 Storage, Journal and Per-Run Thresholds

The write path. The analysis service writes the raw layer after extraction, the processed layer after enrichment, and the exploitation layer — rewriting the processed record with the report — after verification (Section 4.3). Each write is fail-soft on its own: a failure is reported as its own event and does not stop the next layer, which only loses its parent link. A storage error never discards a result that already cost a scrape, an enrichment and a model call per claim. Writes go to a temporary file and are moved into place atomically, because they run in the background thread pool. The storage sits behind a five-method interface, so moving the lake to a document database or object storage is one class, not a pipeline change.

The journal. The lake keeps results; what a run does on the way — the queries, every source returned, the rating each got — used to exist only in memory, so a crash lost exactly what was needed to understand a bad verdict. Every job event is now appended to a file per job as it happens. A job a restarted process no longer holds is still answered from its file, and a file with no final line is reported as interrupted rather than left looking as if it might still be running.

The analysis cache. A repeat request for the same URL is answered from a cache instead of re-running the pipeline. Two rules keep it sound. Its key includes the run's *effective* thresholds, not only those that differ from the defaults: otherwise changing a default would serve answers computed under the old value. And every entry is stamped with a schema version that must change whenever the response changes shape, because the cache never expires and never rewrites an entry; the rule was broken in the week it was written, which is

why it is now a test that builds a real response and compares its keys with the shape recorded for the current version.

Per-run thresholds. Every tunable threshold lives in one module, as a set of effective values for one run that default from the environment, and a request may override any of them for that run only. An override outside its range, or a misspelt one, is refused when the request is made rather than when the job runs. Thresholds are passed to each component per call, because the components are long-lived objects shared by every request; the earlier practice of copying a setting into a class attribute froze it at import time, so overrides had no effect, and the tests that seemed to cover them reassigned the attribute directly. A test now fails on any such copy. The lake records the overrides of every run, so a stored verdict stays reproducible. The two groups of scoring weights are the declared exception: they are set by the environment only, because each group must sum to one and a caller setting one weight alone would break that.

The graph sync. The graph is written after the lake, in one transaction per article, with short timeouts so that a stopped database costs seconds rather than a minute per run. Hand-labelled facts reach it only through a synchronisation that replays, in order, the source catalogue, the labelled facts and the newest verified run per URL from the lake. Every write merges on the schema's keys, so the sync can be run any number of times. The labelling tool imports nothing from the backend and does not write the graph itself.

5.6 Frontend

The frontend is a Next.js 14 application in TypeScript with the App Router. Its pages are those of Section 4.8; this section describes the three mechanisms they share.

API proxies. The browser never calls the backend. Each page calls a route of the frontend's own server, which adds the API key and forwards the request. A proxy forwards only the paths it knows: a catch-all that forwarded anything would be an open proxy with the key attached. The job and statistics routes must not be cached. Next.js caches server-side requests by default, and before that was turned off every one-second poll after the first was answered with the same stale response: the interface stayed on the first phase of a finished job, and after a backend restart kept answering for a job the backend no longer knew.

The polling hook. An analysis runs as a background job; the page polls its state every second. The interval is created only after the first poll returns, and only if the page has not been left in the meantime, otherwise an interval would leak and poll a finished job forever. Both rules are held by comments, not tests: the frontend has no test runner, and its typecheck is the only automated gate.

One fold for the trace. The analyzer page and the Live screen show the same per-claim trace — queries, sources, ratings, verdict — by folding the job's events with one shared

function. Since claims run concurrently their events interleave, so the fold reads each event's claim index first, and falls back to the claim's text for runs journalled before the index existed. The phase stepper works from the *set* of phases seen rather than their order, which is why the concurrent pipeline did not break it.

The reader view was measured at phone width (375 px) in light and dark themes with no horizontal overflow, and its animations are off for readers who ask for reduced motion.

5.7 The Labelling Tool

The custom validation set (Section 6.3) is labelled with a tool built for it: one Python file and one HTML page, using only the standard library, so that labelling needs nothing installed and none of the system's services running. It listens on the local machine only and accepts writes only as JSON, so another website open in the same browser cannot post facts into the repository through it.

One file per fact. Every saved fact is its own JSON file, numbered in the order of saving, with X-Fact's keys in X-Fact's order followed by the custom set's own. Every hand-verified fact therefore appears in the repository, and in a diff, on its own; a test holds the shared keys to the X-Fact files so that the harness scores both sets with the same code. When the set is complete, a `join` command re-validates every file and merges them into one JSON Lines file; it refuses, and writes nothing, if any file breaks a rule.

The daily batch. The *Today* tab asks the backend for about ten articles drawn at random, seeded by the date, from what the sources published in the last 30 days: one per source, languages alternated, never one already labelled or proposed, none that the mission screen leaves out, preferring the topic groups the balance table is short of. Each comes with the one to three claims the pipeline's own claim selector would check. *Label* fills the form from the article and claim; *Skip* records why the claim is not worth labelling — an opinion, a prediction, a trivial statement, a fragment, a duplicate — and the share labelled is the claim selector's precision.

The rules on save. The tie-break rules that can be checked mechanically are refused on save: a verdict other than `UNVERIFIED` needs an evidence link; a claim backed only by its protagonist's own source must be `UNVERIFIED`; evidence dated after the article is refused. A verdict-by-group table shows the 25 cells against their targets while labelling.

The blind review. The *Review* tab shows 20% of the facts, chosen by a hash of their file names, without their first label, note or own-source flag, and reports observed agreement and Cohen's κ on the first label, so that correcting a fact afterwards cannot inflate it.

Synthetic refuted claims. A `mutate` command builds a `FALSE` fact from a natural `TRUE` one by substituting a number, a date, an entity or a relation, keeps the source's article, date

and evidence, and records the provenance; the tool lists any mutation whose source has since been relabelled (Subsection 6.3.7).

The tool has 32 tests, run by the project's check, which write to a temporary folder and never to the real set.

5.8 Deployment

The production compose file. Production runs the same services as development with a layer of its own: only the API and the interface are published, and only on the loopback interface; every secret is required; containers restart unless stopped and rotate their logs; the language model is a container of its own; and memory caps come from measurements. On 1 October a full analysis ran inside the development stack (75 s) and inside the production one (184 s, the model on a CPU). The production run found that the development limits for the model — two concurrent calls, a 30 s timeout — turned 3 of 4 CPU verdicts into timeouts; production sets one call at a time and 180 s. The X-Fact pilot later found the same for inference on a CPU (four claims at once pushed embedding requests past their 30 s timeout), and production now also runs two claims at once with a 120 s inference timeout, as the evaluation runs do.

Where it runs. One Google Cloud Compute Engine virtual machine, an `e2-standard-4` (4 vCPU, 16 GB) in Madrid, runs the whole compose stack. It was sized from the production stack's measurements (about 9.6 GiB resident) and from the model's time per claim on a CPU. Cloud Run was ruled out: the embedded vector store takes an exclusive file lock, jobs live in memory, work runs in background threads, and about 6 GiB of models must stay loaded, so each service would have needed a redesign.

Infrastructure as code. Terraform creates the project's APIs, a service account for the machine that can read only its four secrets and write logs and metrics, the four secrets in Secret Manager, a network of its own whose only open port is HTTPS, the machine itself with a boot script, a daily disk snapshot kept seven days and, when an address is given, an uptime check and a search-health alert by email.

The front door. Caddy serves the site over HTTPS only, with a password on every page generated by Terraform (only its hash reaches the machine), because the interface's proxies attach the API key. The reader view is the exception and is public. With a domain, Caddy obtains a certificate; without one, only the health endpoint is public and the site is reached through an authenticated tunnel.

Checked without an account. The Terraform is applied to Floci, a local Google Cloud emulator, by one command of the project's check: 32 resources are created, the generated password opens the rendered hash, and a second plan is empty. Floci runs no guest, so the machine's boot has never been executed; a first real deployment needs a billed project and is

described step by step in the repository. Two defects were found on the way: shell scripts checked out on Windows with CRLF line endings, which would have broken the machine's boot script, and a password hash whose dollar signs were expanded by the shell.

Observability. Logs are one JSON object per line; a metrics endpoint in Prometheus' format counts stages, outcomes, verdicts and empty or failed searches; and a health endpoint asks every dependency. On Google Cloud an agent ships both to Cloud Logging and managed Prometheus.

Models in memory. A writer model beside the verifier changes that sizing: a 7–9-billion-parameter writer at the 16k-token context a draft needs takes about 6–7 GB loaded, beside the verifier's 3 GB, on a machine already about 9.6 GiB resident. Three measures keep it within memory, each covering what the one before cannot. In production, Ollama holds one model at a time and unloads the idle one to load the other; since the backend sends one model call at a time anyway, the swap is paid only when a draft follows a verdict or the other way round, and the model container's memory cap follows the largest model rather than the sum. Every model the configuration names — verifier, writer and reviewer, and the writer's long-context variant — is pulled before the backend starts, and only the verifier is required, so a missing writer cannot stop verification. On the development machine a script brings up only the services an activity needs (analysing, writing, evaluating or labelling), stops the others — the writer needs no graph database, the evaluation harness neither the graph nor the API — and keeps exactly one model in memory, refusing to load one when the machine does not have its size and a margin free. On 19 October that machine, which has no graphics card, ran two evaluation runs at once with the verifier on its processor, each model call taking 40 to 75 seconds; the writer models were downloaded meanwhile and tried only afterwards, because a second model loaded between the runs' calls would have evicted the verifier on every one.

5.9 Testing and Quality Assurance

One command defines "done". `./scripts/check.sh` runs the backend tests, the inference service's tests against the real models, the frontend's typecheck and the labelling tool's tests. On 16 October 2026, after the writing flow was merged, it passed with the inference service reachable, so the model tests ran rather than skipped: 1,760 backend tests (one skipped, a check Windows cannot make, and 3 expected failures), 13 inference tests, the typecheck and 32 labeller tests; the two slow tests over the stored corpus passed as well. Separate targets run what needs more than a laptop: the slow tests over the whole stored corpus, the graph against a real Neo4j, and the cloud infrastructure against the emulator.

Rules as tests. The rules most costly to break are enforced by a test module rather than kept as conventions: no setting frozen into a class attribute; every per-run tunable in the thresholds module; the cache's schema version matching the response's shape; no test that calls itself at import (which once aborted collection and took down the whole suite); no test that asserts against a local configuration value; a progress event for every pipeline stage;

English and Spanish lexicons complete; no test writing into committed data; concurrency ceilings kept out of per-run thresholds; and no model library imported by the backend. When one fails, either the code or the declared exception changes, deliberately.

Fakes with contracts. The fact-checker's collaborators that reach the network — the search engine, page fetches, the model — are replaced in tests by shared fakes, and each fake has a contract test that pins its signature to the real class, so a fake cannot drift out of step with what it stands in for.

The full pipeline, end to end. An integration test runs the real analysis from a URL: the real extraction cascade over a saved page, real enrichment over the inference service, real admission on a temporary vector store, and the real fact-checker, with only the search engine, the page fetches and the model faked. Its ten tests check every per-claim stage in order, that cited sources survive ranking, that a claim with no evidence is `UNVERIFIED` without a model call, that the article cannot corroborate itself, the three lake layers with their lineage and the response shape. It ran, rather than skipped, with the inference service reachable: ten passed in 26 s.

The guardrail stress test. 83 scenarios of a fake model bluffing through the real verifier, plus three strict expected failures for the bluffs that still get through (Subsection 4.6.6).

The harness tested like the system. Every metric of the evaluation harness is checked against values worked out by hand on a small fixture — accuracy, per-class precision and recall, macro-F_1 over the classes the gold labels contain, coverage, selective accuracy, Cohen's κ and when it is undefined — together with the bootstrap's determinism under a seed, a paired bootstrap that returns exactly zero for identical runs, and resuming after a crash. A wrong metric is a wrong thesis.

Continuous integration. A GitHub Actions workflow runs three of the check's targets on every push and pull request: the backend, the frontend and the labelling tool. The model, corpus, graph and cloud targets stay local, and a test holds the workflow to the check's list of targets.

Load. A load test of the synchronous analysis endpoint on a live stack succeeded for four concurrent analyses (median 156 s) and degraded at six (median 196 s, one beyond the client's 240 s timeout), with every per-service ceiling holding; sustained load on the job endpoint remains unmeasured.

What is not tested. The frontend has no test runner and no end-to-end tests; two of its rules are held only by comments (Section 5.6). A real model trying to bluff is untested: the stress test uses a fake one.

5.10 Writing, Reviewing and Publishing an Article

The newsroom's last steps were built on 15 October 2026, outside the feature freeze of 14 October by the author's decision; until then writing from verified sources was listed as work for "only if time is left" (Table 3.3). After an article has been analysed and judged publishable, a person can ask for a new article on the same story: the system finds news related to it, writes a draft from those sources and the fact-checks in Catalan, Spanish or English following the person's indications, reviews it, rewrites it if its metrics fall short, and publishes it once a person approves it — today only to the system's own public reader.

One draft, one store. Every stage reads and writes one draft record, kept as a JSON file per draft beside the lake. Its body cites its sources as `[n]`, numbered once and never reordered, so a citation always points at the source it was written from. A draft's editorial status is *draft*, *approved* or *published*; long steps run in the background, one at a time per draft, and a person's edit sends an approved draft back to *draft*.

Writing, in a closed world. Only an article the reader publishes can be written from: analysed, admitted, and not judged `FALSE` or `MISLEADING`. Related news comes from two routes at once — the internal corpus, by the analysed article's own stored vector, and the open web, by its headline through the same metasearch engine the fact-checker uses — filtered to one source per domain and none from the article's own outlet. The analysed article is always source `[1]`; then the related news; then only the evidence a verdict was actually based on, each excerpt taken around the passage the verdict quoted. One call to the language model writes the draft, under rules that keep it in a closed world: only the numbered sources and the fact-checks, a citation on every factual sentence, no number, name, date or quote the sources lack, and the verdicts deciding what may be stated — a `FALSE` or `UNVERIFIED` claim never as fact. Text in a source is material, never an instruction. Checks without a model then repair what can be repaired (citations to sources that do not exist are removed) and log what needs a judgement: uncited sentences, runs copied from the analysed article, numbers no source contains.

Reviewing against what the writer was given. The review scores nine metrics from 0 to 100: the title and the subtitle, and the corrector's seven (Section 6.6.2). Unlike free text, a draft can be held to its sources, so the model is given each source's excerpt and the verdicts and asked to check every factual sentence against the source it cites; deterministic checks then bind the scores where a small model reading a long prompt is unreliable — a factual sentence with no citation caps the hallucination score, a citation to no source lowers factual consistency. A second call proposes a correction, which is offered to the person only if it adds no figure, keeps every citation on its statement, cites no new source, stays in the draft's language and keeps a plausible length. Catalan is supported with its own complete word lists and a named stand-in for readability, since no Catalan reading-ease formula was found to cite.

Rewriting towards targets. When metrics fall below their targets — 85 for factual consistency and hallucination, 80 for grammar, lower for style and SEO, reasoned rather than fitted — the person can ask for a rewrite guided by the failing metrics. Each round is a closed-world call over the draft's own sources, the same checks as the correction, and a new review; a rewrite that breaks a check is not kept, and any earlier text can be restored.

Publishing, with the AI disclosed. An approved draft is published as a snapshot built from an allowlist of fields — never the instructions, the review or any source's excerpt, which is somebody else's text — so the public page shows what a person approved until they publish again or take it down. Every published article says, before its text, that it was written by an AI model, names the model, and says that a person approved it. The EU AI Act's Article 50(4), applicable from 2 August 2026, asks for such a disclosure for AI-generated text published to inform the public, with an exemption for text under human editorial responsibility; whether this site meets the exemption has not been judged, and disclosing does not depend on it. Social channels are listed with what each would need (both publish media, not text) and are not built.

What was measured. The writer was run directly, outside the stack, with `llama3.2:3b` on four drafts of a sample article in Catalan and Spanish: none of the four cited a source; three stated as fact a claim the fact-checks had left `UNVERIFIED` (the planted `FALSE` one was not stated); and the Catalan had errors a reader sees at once. A fifth run repeated it. The review, on two seeded drafts, caught a claim judged `FALSE` and the missing subtitle, but missed both spelling errors and a wrong figure, and its correction changed a fact and moved a citation — which is how the figure and moved-citation checks came to exist. The rewrite checks stopped four of five rewrites, three of them for pinning a source onto the false claim; they cannot see a fact a rewrite leaves out. Two practical findings followed: the 3-billion-parameter model is fit to verify claims quickly on a CPU and not to write or correct an article, and Ollama silently cuts a long prompt to its default 4,096-token context, so the writer's model needs its own larger context. On 19 October two openly licensed writers were tried the same way: Qwen3.5 at 4 and 9 billion parameters (Apache 2.0), served at a 16k-token context and with their reasoning mode switched off, since both otherwise think before every answer. Both cited a source for every sentence or all but one, filled the metadata and stated neither the unverified nor the planted false claim; on a processor alone a draft took about three minutes with the smaller and four to six with the larger. Their Catalan differed: the smaller made slips a reader notices, the larger wrote "litigi" (a lawsuit) for *liti* (lithium) throughout. The smaller is the recommended writer (Subsection 7.6.2); no draft has yet been written through the running stack.

Chapter 6

Experimental Design

6.1 Research Questions and Metrics

Each research question of Section 1.5 is answered by a fixed set of measurements, chosen before the results were known. Table 6.1 summarises them; the rest of this section explains the choices that are not standard.

	What is measured	**Metric**	**Data**
RQ1	The work discovery saves, and how well it orders what it finds	Sources and links read against the every-topic baseline; precision of the first k candidates shown; AUC of the ranking score; share of the first k on the topics asked for	Live candidate rounds, each candidate labelled worth offering or not
RQ2	Verdict quality, and what retrieval contributes to it	Accuracy, macro-F_1, per-class precision and recall, Cohen's κ against gold, with bootstrap intervals; the same with no evidence and with the annotator's own evidence; evidence recall and MRR; the stage each error comes from	The custom set (Section 6.3), natural and synthetic claims apart; X-Fact as an external benchmark
RQ3	Agreement between a person and a model making the same decision	Cohen's κ and the confusion matrix; for selection, precision, recall and Jaccard of the model's picks against the editor's	Annotator against the AI labelling pass; editor against the AI selection (blind rounds); annotator against the pipeline
RQ4	The share of the journalist's work done at a stated quality	Claim-selector precision; coverage and selective accuracy of the verdicts; time per article, system against person	All of the above, joined per article

Table 6.1: Research questions and the measurements that answer them.

RQ1 — a retriever of news, evaluated as a ranked list. The candidates of a discovery round are a ranked list shown to an editor, so they are evaluated as information retrieval

evaluates one: by the precision of its first k positions and by how well its score orders relevant above non-relevant items [20], with "relevant" meaning *worth offering*: constructive, on one of the topics asked for, and an article rather than a page of another kind. The work saved is counted directly, as the sources and links a scoped round reads against a round that asks for every topic.

RQ2 — three conditions, so that retrieval has a number. The verifier is run three times over the same claims: with no evidence at all (the model's own knowledge), with the evidence the pipeline retrieves, and with the annotator's reference evidence given directly. The first gap measures what the open-web retrieval contributes; the second, what better retrieval could still add. This is the decomposition FEVER and AVeriTeC use when they report verification with gold and with retrieved evidence [33, 38]. Retrieval itself is measured against the annotator's evidence — whether its links, or their domains, are among the candidates, the ranked sources and the cited ones — and each wrong verdict is attributed to the stage of the four-stage architecture [17] in which the trace shows it went wrong. Natural and synthetic claims (Subsection 6.3.7) are reported apart, and evidence published after a claim is flagged rather than counted [14].

RQ3 — agreement, with the annotator as the ceiling. Where a person and a model take the same decision on the same input, their agreement is measured with Cohen's κ [7], read with the usual caution about fixed thresholds [1, 23]. Three such decisions exist in the system: which candidates to publish (editor against the AI selection, with blind rounds so that the editor cannot be anchored by the model's picks), which label a claim deserves (the annotator against the AI labelling pass), and the verdict (the annotator against the pipeline). The annotator's agreement with their own blind re-label is the ceiling against which the model's agreement is read; recent work reporting that language models can match or exceed crowd annotators on some labelling tasks [13] is the hypothesis being tested, not an assumption.

RQ4 — extent, not replacement. The system's answer to RQ4 is a profile, not a verdict: for each step of the journalist's work, the share the system performs without a person and the quality at which it does so. The verifier's coverage (the share of claims it answers with a definitive verdict) is reported with its selective accuracy (the accuracy on those claims), because a system that abstains often and is right when it answers takes over a different share of the work than one that always answers.

6.2 Selection and Justification of the Evaluation Dataset

In order to quantitatively evaluate the performance of the fact-verification system proposed in this work, a reference dataset (*benchmark*) is required against which standard classification metrics (precision, recall, and F_1) can be measured relative to a verified *ground truth*. This section describes the process followed to select such a dataset, justifying why the most widely cited benchmark in the automated fact-checking literature — FEVER [38] — was discarded

in favour of X-Fact [18], and how this decision is directly motivated by the architecture of the developed system.

6.2.1 The Closed-Retrieval Paradigm: FEVER

FEVER (*Fact Extraction and VERification*) [38] is, to date, the most influential dataset in automated fact-verification research. It consists of 185,445 claims generated through the controlled alteration of sentences extracted from Wikipedia, each subsequently labelled by human annotators — without knowledge of the original sentence — as `SUPPORTED`, `REFUTED`, or `NOT ENOUGH INFO`. For each supported or refuted claim, the sentence or set of sentences from Wikipedia constituting the necessary evidence for that verdict is additionally recorded.

The aspect of FEVER most relevant to this work is its **closed-domain retrieval paradigm**: both the claim and its correct evidence originate from a fixed, bounded corpus — a static dump of English Wikipedia — and systems evaluated against this benchmark typically index that same corpus to perform evidence retrieval. The original paper itself characterises the task as the combination of three subtasks evaluated jointly: document selection, sentence selection within those documents, and final verdict classification, all operating over the same closed corpus.

Direct multilingual extensions of FEVER also exist, such as XFEVER, which automatically translate (via DeepL) the original claim-evidence pairs into other languages, including Spanish. However, these extensions inherit the same structural limitation as the original dataset: the *correct* evidence remains defined as a specific sentence within a closed, translated corpus, rather than as any valid evidence available on the open web.

6.2.2 Why FEVER Is Not Suitable for This Work

The system developed in this thesis — described in detail in Chapter 4 — does not perform verification against a closed corpus. Its fact-checking pipeline (Section 4.6) is organised into seven sequential stages:

1. **Admission**: the article must be on one of the publication's topics, constructive enough, and not a duplicate of one already stored; otherwise nothing further is spent on it.
2. **Claim selection**: the article is decomposed into candidate sentences, and the two to four most load-bearing checkable claims are kept.
3. **Evidence retrieval**: three queries per claim are sent to SearXNG, a self-hosted metasearch engine that queries several search engines over the open web in real time, beside a search of the system's own corpus of past articles.
4. **Ranking and the pertinence gate**: the retrieved sources are ordered by relevance to the claim, and those that share its subject without addressing it are cut.
5. **Verification**: a language model (`llama3.2:3b` by default, run locally through Ollama) reasons over the ranked evidence and returns a verdict that must cite the sources it

rests on.

6. **Confidence recalibration**: the verdict and its confidence are checked against the evidence independently of the model, and an ungrounded verdict is downgraded.

7. **Aggregation**: the claims' verdicts are combined into the article's.

This architecture — referred to throughout this work as the **proposed method** — therefore performs **open-domain retrieval**: no fixed corpus exists against which claims are compared; instead, evidence is searched for *on the fly* on the live web at verification time, exactly as a human fact-checker would.

Evaluating this system against FEVER or its multilingual variants would introduce a serious methodological mismatch, for two reasons:

- **Invalid-evidence bias.** The system could locate, through SearXNG, evidence that is perfectly valid and sufficient in a source other than the predefined Wikipedia sentence, and still be penalised for not matching the expected evidence verbatim. The resulting metric would systematically underestimate the system's true performance.

- **Partial evaluation of the architecture.** Since FEVER fixes the source corpus in advance, evaluation against this benchmark would measure almost exclusively the *verification* stage, leaving *evidence retrieval* and *ranking* unevaluated — precisely the stages that constitute the most original, and also the most fragile, contribution of the system, as discussed in Chapter 7.

For these reasons, FEVER/XFEVER was discarded as the primary evaluation benchmark: not out of convenience or expected performance, but because the dataset's own design assumes a (closed) retrieval paradigm incompatible with the (open) retrieval paradigm underlying the proposed system.

6.2.3 X-Fact as an Open-Retrieval Benchmark

As an alternative, **X-Fact** [18] was selected: the largest publicly available multilingual dataset for the verification of naturally occurring, real-world claims. X-Fact contains 31,189 claims across 25 languages — including Spanish — drawn from professional fact-checking organisations and labelled by expert fact-checkers using a fine-grained veracity scale (not limited to FEVER's supported/refuted/not-enough-info triad).

The decisive structural difference with respect to FEVER is that X-Fact's claims are **real-world claims** made in public discourse (statements by public figures, news headlines, viral posts), not synthetic sentences derived from Wikipedia, and their verification — both by the original human fact-checkers and by the baseline models proposed by the authors — relies on evidence retrieved via search engines over the open web, not against a predefined closed corpus. The authors themselves report that their best baseline model, which combines the claim text with metadata and evidence retrieved through a search engine, achieves an F_1 of approximately 40%, positioning X-Fact as a challenging benchmark even for systems that

already operate under the same open-retrieval paradigm as the method proposed in this work.

This alignment of paradigms — open retrieval in the evaluation dataset and open retrieval in the evaluated system — is what makes X-Fact a methodologically coherent choice: it allows the claim-checking half of the pipeline (evidence retrieval, ranking, verification and recalibration, jointly) to be measured, not only its final reasoning stage, and situates the results within the framework of the baselines Gupta and Srikumar [18] report for their in-domain test set, of which Spanish is one of twelve languages. (The paper gives no per-language figure, so the comparison of Section 7.1.3 is with the whole test set.)

6.2.4 Domain Limitation and a Custom Validation Set

Nevertheless, X-Fact does not fully capture the nature of this work's application domain either. Its claims originate predominantly from political discourse and controversial public statements, while the developed system is applied to constructive news about science, the environment, health, culture and society, from the positive-news outlets, general newspapers, public bodies and academic publishers of its source catalogue (Section 5.2.2). Their claims are mostly reports of studies, projects, figures and events, often sourced to the organisation the story is about, and they sit beside interpretations of those facts that no evidence can settle — a mix virtually absent from X-Fact, whose claims were chosen by fact-checkers precisely because someone disputed them.

For this reason, the quantitative evaluation in this work relies on **two complementary sources**: X-Fact [18], as a standardised benchmark enabling comparison of the proposed method against the state of the art in multilingual, open-retrieval fact verification; and a custom validation set, built from claims extracted from real articles of the configured sources and labelled by hand (Section 6.3), designed to measure the system on the claims of the domain it was built for — the real use case, and one that no existing academic benchmark covers.

6.3 Construction of the Custom Validation Set

Section 6.2.4 argued that X-Fact [18], although methodologically aligned with the open-retrieval architecture of the proposed method, does not cover its application domain: X-Fact is dominated by political statements, whereas the system is applied to positive-news articles about science, the environment, health, culture and society, whose claims mix checkable facts with interpretive readings of them. This section describes how the complementary, hand-labelled validation set was built: its size and stratification, the record format, the label vocabulary, the annotation procedure, the tie-break rules fixed *before* labelling began, the protocol used to verify the consistency of the labels, and the division into a tuning and a test half.

6.3.1 Size and Stratification

The set targets **150 claims**, with 100 as the minimum below which per-stratum counts stop being informative. With 150 claims, a 95% confidence interval on an accuracy near 50% is approximately ±8 percentage points ($1.96\sqrt{0.25/150} \approx 0.080$), which is sufficient to separate the models compared in Chapter 7 when they differ substantially, and insufficient to rank models that differ by a few points. That limit is stated here rather than discovered later.

The claims are stratified along two axes, so that neither a single verdict nor a single subject dominates the metrics:

- **Verdict**: the five labels of the system's own `Verdict` enumeration (Table 6.4).
- **Topic**: each claim carries one of the 23 topics of the system's topic classifier (`src/config/topics.py`). Because 23 topics over 150 claims leaves six or seven claims per topic, too few to balance on, balance is sought over the five *topic groups* into which those topics are already organised: society, science, environment, culture and health.

Five verdicts by five groups gives 25 cells, with a target of six claims each (30 per verdict and 30 per group). Language (English or Spanish) and claim type (factual, numerical or interpretive) are recorded and reported, but not targeted.

The targets are a goal, not a quota. Positive-news articles are predominantly accurate, so the `FALSE` and `MISLEADING` cells are expected to be the hardest to fill from the outlet's own articles. A cell that remains short is reported with its real count rather than padded with claims from outside the domain, and the evaluation reports macro-averaged F_1 alongside accuracy for that reason: under class imbalance, accuracy rewards a system that predicts the majority verdict.

Scope. The set is meant to hold the claims the system would check, so its articles must be ones the system would admit. Before the discovery stage screened candidates by mission, the daily labelling batch also drew party politics, armed conflict, crime, sport, a product review, a puzzle and a news roundup. On 14 October 2026 the facts of those articles — 39 facts from 17 articles — were moved out of the set by the same criteria the system applies to keep such stories out: the off-mission subjects and non-article formats of its mission screen. They are kept, with the category that excluded each, and can be restored; they are not evaluated.

The distribution, as it stands. Table 6.2 gives the set on 14 October 2026. The prediction above held, more strongly than expected: not one of the 109 natural claims is `FALSE`. With that column empty, a system that always answered `TRUE` would score 83% accuracy, and the set could not test the one thing a fact-checker exists for. The `FALSE` column is therefore filled with synthetic refuted claims, built as FEVER and SciFact built theirs and reported apart from the natural ones (Subsection 6.3.7). The `MISLEADING` and `UNVERIFIED` cells stay short and are reported with their real counts.

	Society	Science	Environment	Culture	Health	Total
TRUE	10	23	21	14	23	91
PARTIALLY_TRUE	0	2	2	1	2	7
MISLEADING	1	0	0	0	0	1
UNVERIFIED	3	2	5	0	0	10
FALSE (synthetic)	6	6	6	6	6	30
Total	20	33	34	21	31	139

Table 6.2: The custom set on 14 October 2026, by label and topic group: 109 natural claims (74 in English, 35 in Spanish) and 30 synthetic refuted ones (21 and 9). Of the natural claims, 31 were labelled by the annotator and 78 by an AI labelling pass awaiting the annotator's review (Subsection 6.3.4).

6.3.2 Record Format

Every record reproduces, key for key and in the same order, the format into which X-Fact was converted for this work (Section 6.2.3), so that the harness that scores the system against X-Fact scores it against this set without modification. The shared keys are listed in Table 6.3; the custom set appends its own keys after them, which a reader of the X-Fact format ignores.

Key	Content in the custom set
Shared with X-Fact	
language	en or es.
site	Domain of the outlet in which the claim was read (X-Fact records the fact-checking organisation here).
claimant	Who makes the claim: a person, organisation or the outlet itself.
claim	One atomic, checkable assertion, as the article states it.
claimDate	Publication date of the article.
reviewDate	When the label was last set.
labelRaw	The label in the annotation guide's vocabulary (Table 6.4).
label	The label in the system's Verdict vocabulary.
referenceEvidenceLinks	URLs of the evidence on which the verdict rests.
split	Always test: no claim is ever used to train or tune on.
Added by the custom set	
id	The record's file name (fact001, fact002, ...), in the order of annotation.
topic	One of the 23 classifier topics; its group is derived.
claimType	factual, numerical or interpretive.
sourceTier	Tier of the strongest evidence used (Rule 4, Section 6.3.5).
onlyOwnSource	Whether only the organisation the story is about backs the claim (Rule 1).
evidenceDate	Date of the newest evidence used (Rule 3).
articleUrl	The article the claim was taken from.
annotatorNote	The reasoning behind a close call, and the rule that decided it.
review	The blind second label, when the record is in the verification sample (Section 6.3.6).

Table 6.3: Record format of the custom validation set. The first ten keys are those of the X-Fact records used in this work, in the same order.

As in X-Fact, the evidence links are kept for inspecting a disagreement, not fed to the system: the system retrieves its own evidence from the open web at verification time, and handing it the annotator's sources would measure the reasoning stage alone, which is precisely the flaw for which FEVER was discarded (Section 6.2.2).

6.3.3 Label Vocabulary

X-Fact preserves each fact-checker's original label (`labelRaw`) next to a normalised one (`label`); the custom set does the same. The annotation guide names its labels after AVeriTeC [33], the real-world, open-web claim verification benchmark whose four classes — *Supported*, *Refuted*, *Not Enough Evidence* and *Conflicting Evidence/Cherry-picking* — are defined by the evidence available rather than by an absolute notion of truth, which is the stance this work also takes. A fifth label, *partially supported*, is added because the system's own `Verdict` distinguishes a claim whose central assertion holds while a detail does not, the outcome in which real verification most often lands. Table 6.4 gives the correspondence.

`labelRaw`	`label`	**Definition**
supported	`TRUE`	The evidence supports the claim as stated.
partially supported	`PARTIALLY_TRUE`	The central claim holds; a detail does not.
conflicting evidence/cherrypicking	`MISLEADING`	Sources of equal standing conflict, or the claim is true but selected to mislead.
refuted	`FALSE`	The evidence refutes the claim.
not enough evidence	`UNVERIFIED`	There is not enough independent evidence to decide either way.

Table 6.4: Annotation labels, after AVeriTeC [33], and the system verdicts they map to. The X-Fact mapping used in this work sends X-Fact's *partly true/misleading* to `MISLEADING` and its *complicated/hard to categorise* to `UNVERIFIED`, so both sets are scored on the same five classes.

6.3.4 Annotation Procedure

Claims are taken from articles published by the positive-news and science sources the system ingests, and neither the articles nor the claims are chosen by the annotator. Each day a batch of about ten articles is drawn at random from what the sources published in the previous thirty days, which keeps small the gap between the evidence admissible under Rule 3 and the present-day web the system searches. Before the draw, the candidates pass the same screen that keeps the system's own discovery to the publication's mission: pages that are not articles (podcasts, photo galleries, roundups, media advisories, shopping) and stories the publication never covers (party politics, crime, sport, celebrity) are left out, so the set is drawn from what the system would itself analyse. The draw takes at most one article per source, alternates the two languages, excludes every article already labelled or previously proposed, and prefers articles whose predicted topic belongs to an under-filled topic group. It is seeded with the date, so the same set of candidate links yields the same draw. For each article, the system's own claim extractor and anchor-claim selector propose between one and three claims, which are therefore the claims the system would itself verify.

The annotator either labels a proposed claim or rejects it with a recorded reason: an opinion or interpretation, a prediction, a checkable but trivial statement, a fragment of a sentence, or a duplicate. Those rejections are kept, and the share of proposals accepted is reported as the precision of the system's claim selection as judged by the annotator. Each record holds one atomic claim, kept in the article's own wording; a proposed sentence that asserts two things is split into two records, following the claim decomposition practice of real-world fact-checking

datasets [17]. The annotator then searches for evidence independently of the system, records the links on which the verdict rests, and assigns the label under the rules of Section 6.3.5. The system's verdict is never shown during annotation.

Records are entered through a small web form built for this purpose and kept apart from the system itself: a single program using only the Python standard library, so that labelling depends neither on the system's services nor on any installed package. Its fields are exactly the keys of Table 6.3, and it enforces the rules that can be checked mechanically at the moment a record is saved: a verdict other than `UNVERIFIED` requires at least one evidence link; a claim backed only by its protagonist's own source must be labelled `UNVERIFIED` and the source named; and an evidence date later than the article's publication date is rejected. The same form shows the running count per verdict and topic group against the targets of Section 6.3.1, which is how balance is steered during annotation rather than assessed after it.

Each record is stored as its own file (`fact001.json`, `fact002.json`, ...) in the project's version-controlled repository. Every verified claim is therefore individually visible, and every later correction is recorded as a change to that one file, which leaves an audit trail of the annotation. Once the set is complete, the files are validated again and merged, in order, into a single JSON Lines file in the X-Fact format, which is what the evaluation harness reads. The per-record files remain the source, and the merged file is derived from them.

AI-assisted labelling, disclosed. To keep pace with the daily batches, a share of the claims was first labelled by a large language model (Claude, by Anthropic) working through the same form, under the same guide and rules, with the evidence it relied on cited in the note. Every such record says so at the start of its note ("AI label ... not yet checked by the human annotator"), so it can be told from the annotator's own at any time. None of these labels counts as gold until the annotator has reviewed it and kept or changed it; the agreement between the AI pass and the annotator's review is not discarded but reported, since it measures exactly what RQ3 asks (Section 6.1). The language model used for this pass is not the one the system uses to produce verdicts, so the gold set is not labelled by the model it evaluates; it is, nonetheless, labelled in part by a model, which is why the review is a condition of the set and not an option.

6.3.5 Tie-Break Rules

The decisions that cause the most hesitation during annotation are also those on which a single annotator is least consistent with themself over time. The following four rules were therefore fixed before labelling began and are applied without exception; the `annotatorNote` records which rule decided a close call.

Rule 1 — The organisation's own source. Positive-news claims frequently originate in the press release of the very NGO, company or institution the article is about. A claim for which the *only* available evidence is that organisation's own communication (press release, report, website or social media post) is labelled *not enough evidence* (`UNVERIFIED`), and

the source is named in the note. An organisation reporting on itself is not independent confirmation, and a claim the system could only confirm by reading the claimant's own release is not one it has verified.

Rule 2 — Numeric tolerance. A figure is considered supported when it lies within ±10% in relative terms of the figure in the source (an article's "30%" against a source's "28%" is supported), or when it is a reasonable rounding of it ("nearly 3,000" for 2,870). Beyond that margin, the claim is *partially supported* if the figure is a detail of an otherwise correct claim, and *refuted* if the figure is the claim.

Rule 3 — Date of the evidence. Only evidence available on the article's publication date is admissible, as in AVeriTeC [33], which restricts evidence to documents published before the claim to avoid temporal leakage. A claim that could not be checked when it was published and was confirmed later remains *not enough evidence*; the verdict does not change. Evidence that did not yet exist cannot have made a claim well-founded when it was made, and fact-checking evaluations that ignore this condition have been shown to overstate what a system could do in practice [14].

Rule 4 — Hierarchy of sources. Sources are ranked as follows: primary source (a report, official statistic or peer-reviewed study) > reference media outlet > press release > social media. When sources conflict, the higher tier prevails; when sources of the same tier conflict and none is decisive, the claim is labelled *conflicting evidence* (`MISLEADING`). The tier of the strongest source used is recorded in `sourceTier`, which also allows the evaluation to be broken down by how well-sourced each claim was.

6.3.6 Verification of the Labels

The set is labelled by a single annotator, so inter-annotator agreement cannot be measured. What can be measured is **intra-annotator agreement**: whether the guide, applied by the same person at a different time, produces the same label. Re-evaluating a sample is a requirement of the protocol, not an option: without it there is no way to tell a consistent label from one that would have come out differently on another day. Once labelling is complete, 20% of the *test half* (Subsection 6.3.8) is labelled a second time — the claims the reported numbers rest on:

- **The sample is not chosen by the annotator.** It is the first $\lceil 0.2\,n \rceil$ records of the test half in the order of a cryptographic hash (SHA-256) of their identifier, which is fixed in advance and independent of content: 14 of the 70 test claims, listed in the split's record (`split.json`, `recheck`).
- **The second pass is blind.** The review view shows the claim, its source article, dates, topic and evidence links, but not the first label, the `onlyOwnSource` flag or the annotator's note, all of which reveal it.

- **It takes place at least a week after the first pass**, so that the second label is produced by the guide rather than recalled.

Agreement is reported as observed agreement and as Cohen's κ [7], which discounts the agreement two labellings would reach by chance given how often each used every label; this matters here because the label distribution is expected to be uneven. Following the conventions reviewed by Artstein and Poesio [1], $\kappa \geq 0.6$ ("substantial" on the scale of Landis and Koch [23]) is taken as the threshold for the guide to be considered consistently applicable; below it, the rules behind the disagreements are revised and the affected strata relabelled.

Disagreements are resolved against the guide and, where the first label proves wrong, the record is corrected. Agreement remains measured on the *first* label, which the record keeps beside the second: correcting a record after its review must not turn a disagreement into an agreement.

The same requirement, applied to the system. A label can be inconsistent; so can a verdict. The pipeline searches a web that changes between two runs, through engines that answer differently from one request to the next, and a local model at temperature zero still varies. The same 14 claims are therefore also evaluated a second time by the system, with the same configuration, and the two runs are compared verdict against verdict (Subsection 6.4.7). The annotator's self-agreement and the system's run-to-run agreement are both reported as Cohen's κ on the same claims, so the two consistencies can be read side by side.

6.3.7 Synthetic Refuted Claims

The natural claims contain no refuted one (Table 6.2). Constructive outlets rarely publish a claim the evidence contradicts, and the claims they do get wrong are more often wrong in a detail (`PARTIALLY_TRUE`) than in their substance. A verification benchmark without a refuted class cannot measure whether a system detects false information, so the `FALSE` class is built, as the benchmarks this work follows built theirs.

The precedent. FEVER obtained most of its refuted claims by *mutating* claims extracted from Wikipedia: annotators paraphrased, negated, generalised or specified them, or substituted an entity or a relation with a similar or a dissimilar one, and the mutated claims were then labelled against the evidence [38]. SciFact had expert annotators write the negation of claims supported by a scientific abstract, so that the same abstract refutes them [41]. Both methods share the property that matters here: the evidence that supports the original claim is the evidence that refutes the mutated one, so the refuted label rests on a source already checked, not on a new search.

What was learned from the precedent. Mutation leaves traces in the wording. In FEVER, negation words such as "not" and "only" were frequent enough in refuted claims that a model reading the claim alone, without any evidence, reached an accuracy close to that of evidence-based systems [34]. A benchmark with that artefact measures a model's sensitivity

to wording, not its verification. The procedure below avoids it by design.

The procedure.

1. **Source.** Only a natural claim labelled `TRUE`, whose note states the value the evidence gives, is mutated; each source is mutated once.

2. **Substitution only, of what the claim asserts.** One element is replaced: a number, a date, an entity or a relation (*raised* for *lowered*, the overall prize for the young photographer's category). Negation is not used. The replaced element must be what the claim asserts, not a detail of it: under the guide's numeric rule (Subsection 6.3.5), a wrong detail of an otherwise correct claim is `PARTIALLY_TRUE`, not `FALSE`. Where the claim states figures that must agree with each other, they are changed together, so that the claim does not refute itself.

3. **Plausible values.** The substitute is chosen to be plausible without the evidence — two million new cases of breast cancer a year is roughly the world's figure, not Latin America's; the first Lagrange point lies as far from Earth as the second — so that knowledge of the world, or the evidence, and not the form of the claim, is what refutes it.

4. **Everything else unchanged.** The mutated record keeps its source's article, date, topic, claim type, source tier and evidence links, and names its source and the kind of substitution (`mutatedFrom`, `mutation`). The labelling form refuses a mutation whose source is not a natural `TRUE` claim, or whose text is unchanged, and lists any mutation whose source is later relabelled, since its label was only valid by construction from that source.

5. **Stratified, and reported apart.** Thirty refuted claims were made, six per topic group, nine of them in Spanish; twelve substitute a number, eight a date, six an entity and four a relation. The evaluation reports every metric for natural and synthetic claims separately (the `origin` breakdown), as well as together, because a system can handle substituted claims well and the claims the world produces badly.

What the synthetic class can and cannot show. It shows whether the system notices when a claim contradicts evidence it can find, which is the core of verification. It does not show how often real constructive news is false, nor the kinds of error real articles make; the natural claims answer those questions, and the X-Fact benchmark supplies refuted claims the world actually produced [18]. Like the AI-assisted labels of Subsection 6.3.4, the thirty mutations were proposed by a language model and are reviewed by the annotator; the blind re-label (Subsection 6.3.6) draws from them as from any other claim, which also tests whether a person reading the evidence reaches `FALSE` where the construction says so.

6.3.8 The Tuning and Test Halves

The system is not trained on the set: it searches the open web when a claim is checked, and nothing it learns from a labelled claim is stored. What can still be fitted to the labels is everything the author decides by looking at results — a threshold, the trigger of a second

search (Section 4.6.7), a prompt. Reporting on the same claims that guided those decisions would inflate the result, so before any run over the set it was divided once, on 19 October 2026, into a **tuning half** (`train`) and a **test half** (`test`). Every decision is taken on the first; the second is read once, for the numbers this chapter reports.

Half and half. The plan first set aside a third for tuning. Because the pipeline learns nothing from the claims, the risk the split guards against is limited to the few decisions above, and the author chose an even split: the tuning half then holds enough disagreements between system and label to decide on, and the test half still holds 70 claims.

How the halves were drawn. Three rules, applied by the labelling tool (`labeller/app.py split`) rather than by hand:

- **Claims travel with their article.** Claims of one article share its evidence, and a synthetic refuted claim (Subsection 6.3.7) is its source claim with one element changed and the same evidence. Split apart, tuning on one would be tuning on the other; the 139 claims form 59 such groups, and a group is never divided.
- **Each half holds its share of every stratum**: of each verdict in each language, and of each topic group. Groups are placed one at a time, the group holding the rarest stratum first, on the side whose targets its claims fall furthest short of — iterative stratification [35], with groups in place of single claims. A first draw balanced verdict and language only, and put all 21 culture claims in the test half; adding the topic group as a second dimension corrected it.
- **Ties are broken by a hash** (SHA-256) of the group's first identifier, so no step depends on a choice, and the assignment is frozen: a claim keeps its side when it is relabelled, and a claim added later joins its article's side, or the side its stratum is short of.

Table 6.5 gives the result. The single `MISLEADING` claim fell in the test half, and the two Spanish `UNVERIFIED` claims belong to one article and fell in the tuning half; strata of one or two claims cannot be shared.

	Tuning	**Test**
Claims (articles)	69 (28)	70 (31)
`TRUE`	45	46
`PARTIALLY_TRUE`	3	4
`MISLEADING`	0	1
`UNVERIFIED`	6	4
`FALSE` (synthetic)	15	15
English / Spanish	48 / 21	47 / 23
Society / science / environment / culture / health	11 / 17 / 16 / 10 / 15	9 / 16 / 18 / 11 / 16

Table 6.5: The two halves of the custom set, frozen on 19 October 2026 (`backend/data/evaluation/split.json`).

The re-check sample. The fifth of the test half that is labelled a second time (Subsection 6.3.6) is drawn from it by the same hash, and recorded with the split: 14 claims. The same 14 claims are also run through the system a second time (Subsection 6.4.7), so the consistency of the labels and the consistency of the system are measured on one sample.

6.3.9 Limitations

- **One annotator.** Self-agreement measures the consistency of the guide's application, not its validity: a systematic misreading applied twice agrees with itself.
- **Selection by the system.** Articles are drawn at random and claims are proposed by the system, which removes the annotator's tendency to pick the most checkable cases, but ties the set to the claims the system's selector favours. A claim the selector would never propose cannot enter the set, so the evaluation measures verification on the system's own claim choice, not on an independent sample of all check-worthy claims. The first six records were labelled before the daily draw existed and were chosen by hand; they are identified as such (no batch refers to them) and reported separately.
- **Temporal asymmetry.** Rule 3 restricts the annotator to evidence available at publication time, whereas the system searches today's web, without a date restriction. A system verdict informed by later evidence can therefore be scored as an error; such cases are identified from the evidence dates and reported separately rather than silently absorbed into the error rate.
- **Size.** As computed in Section 6.3.1, differences of a few points between models are within the uncertainty of a 150-claim set.

6.4 Evaluation Harness

Every verdict-level number in this thesis is produced by one tool, the evaluation harness (`backend/src/evaluation/`). It runs the pipeline's own claim-checking stage — the one behind the single-claim endpoint, not a copy of it — over a labelled set in X-Fact's format, which both the X-Fact files and the custom set share, and scores the results against the labels:

```
uv run python -m src.evaluation.cli run --dataset <set> --model <model>
uv run python -m src.evaluation.cli report --run <run directory>
```

Two gaps in the pipeline were closed before it produced a single number. The claim-checking stage took no language, so Spanish claims would have been searched with the English word lists; and it took no article context, so a custom-set claim could have been confirmed by its own article. Both are now optional arguments: the harness passes each row's language, and for the custom set the article's URL, which the retriever excludes. X-Fact rows get no context: the harness gives the pipeline nothing from the gold row that production would not have.

The model is an argument of the run and the provider is configuration only, so the same command measures the local model and a hosted one. The internal corpus is a snapshot

copied into the run at its first start, so a run does not change as the live store grows.

6.4.1 Record Format and Run Layout

One record per claim. Each claim's result is one line of a JSON Lines file, with every field present even on an error, so a report never has to ask whether a key exists. A record holds the gold row's fields, so a report needs nothing else; the verdict and confidence after recalibration and the model's raw answer before it; the stage the claim reached and whether its search or model failed; the queries sent, with their kind; *every* candidate source seen, once, with the stage that cut it and why; the ranked evidence in the order the model saw it, with each factor of its score, its stance, whether it was cited and the quote; the full trace of progress events with timestamps; the latency of each stage; the model calls with their tokens; and the provenance — model, provider, thresholds, harness version and code commit. No scraped page body is stored.

Runs are keyed, so the same command resumes. A run lives in a directory named by a key derived from the dataset's hash, the model, the effective thresholds, the scoring weights, the corpus mode and the harness version: changing any of them starts a new run, while the same command continues the old one. The commit is recorded on every record but not keyed, which would discard a run on every commit. Editing a label changes the dataset's hash, so two gold sets are never mixed in one run.

Resuming. Claims already scored are not run again; errors are retried; a torn last line from a crash is cut and its claim re-run; and claims whose search or model was unreachable can be re-run on request, since those are outages, not results. Every record is flushed and synced to disk as it is written. During the X-Fact pilot this rule absorbed an interruption of the container runtime and two native crashes, re-running exactly the claims each had lost (Section 7.1). Each session of a run records its own settings — timeouts and concurrency — a fix the pilot prompted.

What is committed. Raw runs are not committed: they hold third-party titles and quotes. The report of a run — its metrics in JSON, a readable report and a copy of the run's manifest — is committed, and those are the numbers this document quotes.

6.4.2 Metrics and Confidence Intervals

What is reported apart. Each claim gets one outcome, the first that applies: *error* (the harness or the pipeline crashed), *search unavailable* (no web query was answered), *model unreachable*, or *scored*. Only scored claims enter the metrics. The others are listed by identifier and never counted as errors of the system: on 25 September, 68 of 69 searches came back empty, and folded into the error rate that would have measured the search engine's uptime. On top of the outcome, two flags:

- **Temporal leak**: a cited source was published after the claim (the annotation guide's Rule 3). Undetermined when the claim or the source has no date.

- **Verdict leak** (X-Fact only): a ranked source is on the fact-checker's own site, so the verdict may have been read rather than reached.

Every metric is reported on all scored claims and again without the flagged ones. An answer the model gave in no usable format is scored as the `UNVERIFIED` it became and counted per model.

Verdict metrics. Over the five verdicts:

- **accuracy**, and **macro-averaged** F_1 over the classes present in the gold labels — a class with no gold support is shown with its predictions but not averaged in — with per-class precision, recall, F_1, support and predicted count, and the confusion matrix;
- **Cohen's** κ against the gold labels [7], the same statistic as the annotator's self-agreement, so that the system and the annotator are compared on one scale;
- **coverage** — the share of claims given a definitive verdict rather than `UNVERIFIED` — and **selective accuracy**, the accuracy on those claims, the standard pair for a classifier allowed to abstain [12].

Intervals. Every statistic comes with a percentile 95% bootstrap interval over claims [9]: 10,000 resamples, seeded (2026 by default) and with the seed in the report, records ordered by identifier first so that the same run gives the same interval however its claims finished. Two runs are compared with a *paired* bootstrap of their difference over the claims both scored, alongside how many claims only one of them got right; a run compared with itself gives exactly zero, which a test holds.

Breakdowns. By language for both sets; by topic group, claim type, source tier and origin (natural or synthetic) for the custom set; by fact-checking site for X-Fact. Each with its size, accuracy and interval, macro-F_1, coverage and selective accuracy.

Cost. Each record carries its model calls, tokens and the provider's answer time, measured inside the production client, so the report gives the median and 90th-percentile latency per claim, tokens per claim and per second, and the price per hundred claims from the price table — "n/a" without a price, never zero.

6.4.3 Retrieval Evaluation

RQ2 asks what retrieval contributes. The harness answers in two ways.

Against the annotator's evidence. For each claim with reference links, it checks at three depths — every candidate seen, the ranked evidence the model was shown, and the evidence it cited — whether a reference *link* was found, and whether a reference *domain* was; and it computes the reciprocal rank of the first hit in the ranked list. Links are normalised with the retriever's own function, imported rather than copied, so "found" means what the pipeline means by it. For X-Fact, links on the fact-checker's own site are removed from the references

and counted, since they are the verdict. A miss is not proof of failure — another source can be as good as the annotator's — which is why domain hits are reported beside link hits. The cut reasons of references that were found and then cut show where they were lost.

Without references. For any set: the share of claims with any ranked evidence, candidates and ranked sources per claim, the share cut at ranking, distinct and rated domains, the share from the internal corpus, retrieval and ranking time, queries per claim, and which query kind — anchor, proposition or refutation — found each ranked source. All are means over claims with bootstrap intervals, by set and by language.

Strategies. A retrieval strategy is a run with its own thresholds, corpus, engines or code, named by a label that enters the run key. A retrieval-only mode stops before the model, so a strategy costs only its search and fetch time, and a comparison command sets strategies side by side, each paired against the first. This is what the planned ablation (Table 3.3) uses: switching a component off on the same frozen claims, in one window, rather than comparing versions of the code that cannot be re-run on today's web.

6.4.4 Stage Attribution

Each scored claim whose verdict differs from its gold label is attributed to one stage of the architecture [17], the first that matches in Table 6.6. The rule is applied twice, because the reference links are the better witness but not every row has them — an `UNVERIFIED` gold label needs none.

Table 6.6: How a wrong verdict is attributed to a stage. The first row that matches decides.

Stage	Reference-based	Trace only
Retrieval	No reference link or domain among the candidates	No candidates at all
Ranking	A reference among the candidates, none ranked (cut by the funnel, the same-domain rule, the pertinence gate or the cap)	Candidates, but nothing ranked
Reasoning	A reference is ranked, and the model's raw verdict differs from gold	Something ranked, and the model's raw verdict differs from gold
Aggregation	The raw verdict equals gold and recalibration changed it	The same

Both columns are reported, with counts, shares and claim identifiers, and the claims on which they disagree are listed. Under the reference rule, a wrong verdict resting on sources other than the annotator's counts as retrieval: the decisive source was never found. Under the trace rule the same claim counts as reasoning: the model misread what it was shown. Both are true statements about the claim, and the disagreement between them is itself informative. Claims set apart (search or model unreachable, errors) are never attributed. Everything is read from fields the records already hold.

The rule is not yet a module of the harness. For the X-Fact pilot it was computed by a script with this table (Section 7.1); building it into the report is one of the P0 items of Table 3.3.

6.4.5 Journalist-Effort Metrics

RQ4 asks what share of the journalist's work the system takes on, at what quality. Four measures answer it.

- **Claim selector's precision**: claims labelled over claims labelled plus claims rejected, over every daily batch, in total, by rejection reason and by language. The first six facts, chosen by hand before the daily batch existed, are excluded. *Computed today* by the labelling tool and the evaluation page.
- **Time per fact**: the time from opening a fact's form to saving it, counting only the seconds the page is visible, reported as median and interquartile range. *Designed, not built*: the labelling tool does not yet record it.
- **Usable without re-checking**: the share of scored claims whose verdict equals the gold label, is definitive, cites at least one source with a stance and has neither leak flag. Its counterpart is **confidently wrong**: a definitive verdict that differs from gold, which is what a journalist who trusted the system would publish. *Designed, not built.*
- **Effort estimate**: the usable share times the median time per fact, in minutes saved per hundred claims, beside the confidently-wrong share as the cost of trusting it — stated as an estimate, with its assumptions.

For the steps before verification, the same question is asked of discovery and selection: how many sources and links a scoped round reads against an every-topic round, how many of the first twenty places are worth offering, and how far the AI selection agrees with the editor (Section 6.1).

6.4.6 The Tuning Loop

On the tuning half of the custom set, and only there, every claim goes through a loop after its first verdict (`run -loop`). The loop asks of each disagreement with the gold label whether a second attempt fixes it, and if not, whether the claim or the system is at fault:

1. **Verify.** The claim is checked as the pipeline checks it. If the verdict equals the gold label, and the pipeline's own trigger would not fire, the claim is done.
2. **Look again.** Otherwise the claim gets a second look (Subsection 4.6.7). If the second verdict equals the gold label, the claim is *fixed*: the evidence was on the web, and the first search did not find it.
3. **Check the claim itself.** If it still disagrees, the claim is judged once more, with no search, against the annotator's own evidence links — the pages the gold label was set from, the "oracle" evidence condition of FEVER and AVeriTeC [33, 38]. If on those pages the model reaches the gold label, the error is the *retrieval*'s: the decisive source exists and the search never found it. If it gives the same verdict the pipeline gave, the model and the label disagree on the same evidence, and a person decides which is wrong (*adjudicate*); a third verdict is a *reasoning* error.

The outcomes are reported with the claims behind each, and the *adjudicate* list goes to the annotator's review of the AI-assisted labels (Subsection 6.3.4): a gold set an LLM helped to label is the first thing the loop can find wrong. The loop uses the gold label to choose which claims to look at again, so its accuracy is an upper bound, not a score. What decides whether the second look is turned on is the other number it reports: the accuracy obtained when the second verdict replaces the first wherever the pipeline's own trigger fires, without gold, paired over the same claims, with the claims it fixes and the claims it breaks. Only that rule, chosen on the tuning half, is run on the test half.

6.4.7 Run-to-Run Consistency

The fifth of the test half that the annotator labels twice (Subsection 6.3.6) is also run through the system twice, with the same model, thresholds and corpus snapshot, at a later time. The two runs are compared over the claims both scored:

- **Verdict agreement and Cohen's κ** between the runs, with percentile bootstrap intervals, and the confusion of one run's verdicts by the other's — on the scale of the annotator's self-agreement, so the two can be read side by side; and the same for the model's raw verdict before recalibration.
- **Where a change came from**: the overlap of the two runs' ranked evidence for each claim (Jaccard over normalised addresses). A verdict that changed on the very same evidence is the model's own variation; one that changed with the evidence comes from the web, which changed between the two runs.
- **Whether it matters**: each run's accuracy against the gold labels on the same claims, and the paired difference — two runs can disagree on a claim and both be wrong.

6.5 Experiments on the Stages Before the Verdict

The harness measures what the fact-checker concludes. A second tool measures what decides which articles reach it at all — discovery, the selection step, admission and the topic classifier — which is where the answer to RQ1 and part of RQ3 lie. One command runs each experiment:

```
uv run python -m src.evaluation.experiments <experiment> [options]
```

and writes a JSON result and a readable report to a timestamped folder under `backend/data/evaluation/experiments/`. Intervals are percentile 95% bootstrap intervals with a fixed seed and 2,000 resamples; an AUC is the probability that a random positive outranks a random negative, 0.5 being a coin.

Two further studies were run once rather than as commands, to choose values that are now constants of the code: the mission screen's margin (527 live candidates read by hand) and the ranking's weights (117 candidates labelled before any score was computed, with every reading kept so other weights can be tried without embedding anything again). Their data are committed with the experiments.

Table 6.7: The experiments on the stages before the verdict.

Experiment	Question	Measures
`discovery`	How much does scoping to topic groups save, and what does it still find?	Sources read, attempts, links, candidates by language, seconds, each against the every-topic baseline
`selection`	How far does the editor agree with the AI selection?	Precision, recall and Jaccard of the AI's picks against what was sent; AUC of its score; assisted and blind rounds apart
`blind-ai`	The same, without the editor having seen the AI's picks	Runs the AI on a round already sent
`admission`	What do the admission thresholds admit?	Admission rate, and precision and recall against the editor's choice, over a grid of topic and impact minimums; AUC of the impact score
`topics`	How often is the topic classifier right?	Top-1 topic, gold in the top three, top-1 group, share with no topic
`emphasis`	What does weighing the topic over positive impact change?	Re-scores recorded rounds at each emphasis; their first ten and twenty against labels

Labels. The candidate labels these experiments are scored against — clearly positive, worth offering, not worth offering, and whether a page is an article at all — were assigned from each candidate's title, summary and link. The labels of the 6 and 9 October rounds were assigned by the AI labelling pass (Claude) and have not yet been checked by a person; the P1 items of Table 3.3 require the author to label at least two rounds and to run ten blind selection rounds before RQ1 and RQ3 are answered. Feeds change through the day, so settings are compared within one run and repeated on several days for an interval.

6.6 Model Benchmarks

6.6.1 Verification Models

The verifier is compared across models on the custom set (Table 3.3, P1): the local baseline, `llama3.2:3b`; two openly licensed local models of the same family at two sizes, `qwen3.5:4b` and `qwen3.5:9b`, with their reasoning mode switched off; and, if time allows, one hosted model reached through an OpenAI-compatible endpoint. Every other part of the pipeline is held fixed — thresholds, weights, engines, corpus snapshot — so the runs differ by the model alone, and each run's key records that they do, the reasoning mode included: a hybrid model told not to think and the same model thinking answer differently under one name.

Chosen on one half, reported on the other. Since the split of Subsection 6.3.8 exists, the models are compared on the *train* half, and the one chosen is then run once on the *test* half to report it; the test half chooses nothing. The rule was written before any candidate's result was read: a candidate replaces the baseline when its accuracy gain's paired 95% interval over the claims both runs scored lies above zero; when the interval includes zero, the faster model is kept. With 69 claims a paired interval of roughly ten points is expected, so only large differences separate models. Two runs of the same model already differ — the run-to-run consistency of Subsection 6.4.7 and Table 7.7 — so a small gain is not read as one.

What is set side by side. Each model is one harness run. The models are set side by side, one row each, with accuracy and macro-F_1 and their intervals, coverage, selective accuracy, Cohen's κ, invalid outputs, claims set apart, tokens per claim and per second, the cost per hundred claims, and the **response time**: the seconds a claim takes at the median and the 90th percentile, and the model's own answer time per call. A run's report breaks the claim's time into its stages — the claim's entities, evidence retrieval, ranking, the model's verdict, and a second look where one was taken — with each stage's share. The model's span includes waiting for its turn at the model; the answer time does not; the gap between them is a busy model, not a slow one. Response times compare only between runs made alone on the machine, and each model is run alone.

The same models are repeated under the evidence conditions of RQ2 — no evidence, retrieved evidence, the annotator's evidence — only for the chosen one, so that the cost of the comparison stays within the plan.

6.6.2 Writing Models

The text corrector (Section 4.8) judges a text on seven metrics; two of them, readability and coverage of verification, are computed deterministically, and five — grammar, factual consistency, search-engine optimisation, hallucination and style — come from one call to the language model. The benchmark measures a model as the *editor* it is in that role, by running the corrector's own call, prompt included, and refusing to compare runs made with different prompts. It answers none of the four research questions and was moved to "only if time is left" on 14 October; it is described here because it is built and tested.

Planted defects, not opinions. A human score for how good a text's search optimisation is would be an opinion per text; a planted defect is a fact. Six short news texts were written for the set, three in English and three in Spanish, each in a clean version and five versions with exactly one defect aimed at one metric: agreement, spelling and tense errors (grammar); a figure contradicting an earlier one (factual consistency); an invented authority and sweeping claims (hallucination); a vague headline with the subject gone from the lead (search optimisation); and a casual, repetitive, exclamatory register (style). Each defect version against its clean text is a pair whose right answer is known — the targeted score should go down — which makes 30 pairs, 15 per language.

The rubric , in the order a model is judged:

1. **Format compliance** as a gate: the share of texts for which all five metrics came back usable. A model below 95% is out, whatever it scores when it complies; a missing or non-numeric score is never averaged as zero.

2. **Detection**: per pair, 1 when the defect version scored lower on the targeted metric, 0.5 on a tie, 0 when higher, so a model that gives every text the same score sits at chance (0.5); with a bootstrap interval over pairs, per metric and per language.

3. **Defect named**: whether the targeted metric's summary names the defect.
4. **Off-target drift**: the mean change on the four metrics a defect was not aimed at; lower is better.
5. **Consistency**: the spread of a text's score across repeats.
6. **Calibration**, read rather than ranked: the mean score of the clean texts.
7. **Cost and latency**, to choose between models the rubric cannot separate.

The local baseline is compared with the other local candidates, and with one hosted provider if time allows, each paired against the baseline over the pairs both scored. With 30 pairs an interval of about ±15 points is expected, so only large differences will show. Two defects of the corrector were found while building the benchmark and fixed: a null list of issues lost all five metrics, and a missing score counted as a judgement of zero.

The draft reviewer, on seeded drafts. The benchmark judges free text; the reviewer of the writing flow (Section 5.10) judges a draft against the sources it was written from, and proposes a correction that is offered only if it passes the checks. It is tried apart, on the two drafts seeded with defects on 15 October, one English and one Catalan (the Catalan one as a test keeps it, the English one rebuilt from the record of that day), with the same two sources and fact-checks: in English a spelling error ("it's" for "its"), a figure the cited source contradicts (3 million euros where it says 1.2), a claim the fact-checks judged `FALSE` stated uncited, an eight-character title and no subtitle; in Catalan a spelling error ("esta" for "està") and the same `FALSE` claim. A defect counts as found when an issue the *model* raised names it, read from its answer before the deterministic checks bind the scores; each draft is reviewed twice per model. Counted too: the seconds and tokens per call, whether every metric parsed, and whether the correction was offered or why not. Issues the model invented are read by a person. The script is `scripts/try_reviewer_model.py`.

6.7 Experimental Setup

Hardware. Two machines are relevant. Every run reported in this thesis was made on the development laptop: an Intel Core i7-12700H, 16 GB of RAM and an NVIDIA RTX 3050 Laptop GPU with 4 GB, on which about 80% of the 3-billion-parameter model fits, the rest running on the processor; the inference service runs on the processor. The production target is a 4-vCPU, 16 GB virtual machine with no GPU, on which the same model takes about 45 s per claim against about 6 s on the laptop. Latency figures are therefore reported with the machine they were measured on, and a CPU deployment must use the production model limits (one call at a time, a 180 s timeout).

Models. The verifier is `llama3.2:3b` (Q4_K_M) served by Ollama (in production with a 4,096-token context and one request at a time); the inference service runs GLiNER

(`gliner_small`), the `twitter-xlm-roberta-base-sentiment` classifier and `BAAI/bge-m3`, with pinned library versions. Every record names the model and the provider that answered.

Runs in containers. Evaluation runs are made inside the development stack with an extra compose layer that mounts the evaluation data and the code (so a harness fix applies on resume without a rebuild) and the repository read-only, so that every record carries the commit it was made on. The layer also sets the CPU limits the pilot needed: a 180 s model timeout, one model call at a time, a 120 s inference timeout and two claims at once, since four concurrent claims on CPU inference pushed the ranker's embedding calls past the 30 s default.

Thresholds and weights. The X-Fact runs overrode no threshold; their effective values, written in full into each run's manifest, were those of Section 4.6: pertinence 0.25, ranking weights 0.45 / 0.20 / 0.20 / 0.15, pertinence weights 0.5 / 0.5, confidence weights 0.7 / 0.3, at most five sources per claim, one piece of evidence required for a verdict and two independent domains for full confidence in a `TRUE`. One value came from the environment's configuration rather than the declared default: the duplicate threshold was 0.96 instead of 0.90. Admission is not exercised by the harness, which checks claims, so it did not affect these runs; it is recorded because the manifest records it. A run's thresholds are hashed into its key.

Provenance. Each report states the commits its sessions ran on, the dataset's hash, the thresholds' hash, the harness version, the corpus mode and point count, and the bootstrap seed. Before every live run the health endpoint and the source probe's search check are read and recorded, because a run's verdicts mean nothing if the search engines refused it.

The split. Before any run on the custom set, the set is to be split once, stratified by label and language, into about one third for tuning and two thirds for testing (Table 3.3, P0): every threshold is tuned on the tuning part only, and every reported number comes from the test part.

[To write: the frozen split: its date, its seed and its counts by label and language.]

Chapter 7

Results

This chapter reports what has been measured as of 16 October 2026, question by question, and says plainly where a measurement has not been made yet. Two kinds of result are distinguished throughout: those measured against a person's judgement, and those measured against labels assigned by the AI labelling pass and not yet reviewed by the annotator, which are indications until that review is done (Section 6.3.4).

7.1 Pilot on X-Fact

The first run of the full pipeline against labelled data: X-Fact [18], the open-retrieval benchmark chosen in Section 6.2.3. It served two purposes. It shook out the evaluation harness against live services before the custom set is settled, and it places the system beside the numbers Gupta and Srikumar publish for their benchmark. The second purpose turned out to be the harder one: thirteen years separate some of the claims from the web the system searched, and that distance changes what an X-Fact score means.

7.1.1 What Was Run

Nothing was trained. The verification model is a zero-shot language model reading evidence that the pipeline retrieves from the open web at the moment of checking, so there are no weights to fit and no X-Fact claim was used to tune any threshold. What the pilot measures is the pipeline's own claim-checking stage (the one behind the single-claim endpoint), run through the evaluation harness of Section 6.4 over X-Fact rows and scored against their labels.

Two sets were run on 13 October 2026:

- **The pilot**: 64 claims sampled from the English and Spanish rows of X-Fact, eight per language and label (fewer where a label is missing), preferring development and test rows. Its 24 English claims come from PolitiFact and are all training rows, because English has no evaluation split in X-Fact; its 40 Spanish claims come from Chequeado (31 training, 2 development, 7 test). The label distribution is flat by construction.

- **The Spanish test split**: all 165 Spanish rows of X-Fact's in-domain test set (α_1), extracted in file order by a deterministic script. It is the only part of the benchmark the paper reports on that this work can reproduce, and it covers one of α_1's twelve languages.

Configuration. `llama3.2:3b` (Q4_K_M) served by Ollama on the development laptop (Intel i7-12700H, 16 GB RAM, an NVIDIA RTX 3050 Laptop GPU with 4 GB, of which about 80% of the model fitted, the rest running on the CPU); the inference service (entities and embeddings) on the CPU in its container; SearXNG with the engine set of the deployment. Thresholds and scoring weights were the defaults, and the internal corpus was a snapshot of the live store (5 points, none of them about the claims). Each claim was checked without its claimant, dates or fact-checker: the claim text alone, as the single-claim endpoint receives it.

Health before the run. The health endpoint answered for all four dependencies (inference 49 ms, SearXNG 36 ms, the language model 42 ms, Neo4j 1,060 ms). The source probe's search check returned 89 results in each language, from Bing, Yep, Wikipedia (English only) and four academic indexes; Google and Brave were refusing SearXNG ("access denied", "too many requests"). A three-claim smoke run returned 15 to 17 web results per claim before the full run started.

7.1.2 Results

Every claim of both sets was scored: claims whose search or model was never reached were re-run until none was left set apart. Table 7.1 gives accuracy and macro-averaged F_1 with percentile 95% bootstrap intervals over claims (10,000 resamples, seed 2026); macro-F_1 averages over the five verdicts the gold labels contain, three for English, whose PolitiFact scale has neither `MISLEADING` nor `UNVERIFIED`.

Set	**n**	**Accuracy (%)**	**Macro-F_1 ($\times$100)**
Pilot, all	64	29.7 [18.8, 40.6]	16.8 [10.4, 23.4]
English (PolitiFact)	24	41.7 [20.8, 62.5]	35.8 [15.7, 55.4]
Spanish (Chequeado)	40	22.5 [10.0, 35.0]	13.0 [6.1, 19.6]
without leak flags	43	23.3 [11.6, 37.2]	15.3 [7.1, 23.8]
Spanish test split (α_1)	165	19.4 [13.3, 25.5]	11.1 [7.9, 14.2]
The same model with the claim alone, no evidence			
Pilot	64	18.8 [9.4, 28.1]	6.3 [3.4, 8.8]
Spanish test split	164	16.5 [11.0, 22.6]	8.4 [4.9, 12.7]
Baselines from the label counts			
Pilot, majority class	64	25.0	8.0
Pilot, uniform random	64	20.0	$\approx$19.5
Spanish test, majority (`TRUE`)	165	30.3	9.3
Spanish test, uniform random	165	20.0	$\approx$19.2

Table 7.1: The pipeline (`llama3.2:3b`) on the X-Fact pilot and on the Spanish rows of X-Fact's in-domain test set, with 95% bootstrap intervals. "Without leak flags" leaves out claims for which a cited source is newer than the claim or a ranked source is the fact-checker's own page (Section 7.1.4).

On the pilot the system is barely distinguishable from chance: Cohen's κ against the gold

labels is 0.07, with an interval (−0.03 to 0.18) that includes zero, and its macro-F_1 is below what a uniform random guess is expected to score on a set balanced by construction. The reason is visible in the confusion matrix: the system answered `FALSE` to 43 of the 64 claims, only 14 of them correctly (16 of the 64 are false), and never answered `MISLEADING`. Asked without evidence, the same model answers `FALSE` to 60 of 64, so this lean is the model's prior showing through the evidence rather than a property of the evidence. The evidence nevertheless helps: paired over the same claims, the pipeline beats the closed-book model by 10.9 accuracy points (interval 3.1 to 20.3) and 10.4 macro-F_1 points (4.5 to 16.8). On the Spanish test split the picture is worse: an accuracy of 19.4%, below the 30.3% of always answering the majority class, a κ of −0.002 (−0.06 to 0.06), `FALSE` for 107 of the 165 claims (24 are false) and `MISLEADING` never, for the second-largest class; and there the evidence buys nothing measurable over the closed-book model, which answers `FALSE` to 155 of 164 claims (3.0 accuracy points, interval −3.0 to 9.1).

Retrieval. For the 38 Spanish claims with reference links (the English rows carry none), the system retrieved none of the annotator's links at any depth, and a reference *domain* among its candidates for 10.5% of claims. It found evidence for 98.4% of the claims, about 18 candidates and 4.7 ranked sources each. Under the trace-only rule of Section 6.4, 44 of the 45 wrong verdicts are reasoning errors; under the reference rule, 27 of the 29 it can judge are retrieval errors. The two rules disagree on 28 of those 29: the model misread what it was shown, and what it was shown was not what the fact-checker had used.

Cost. A claim took 36.5 s at the median (90th percentile 59.9 s): 14.8 s of retrieval, 8.6 s of ranking and a 5.6 s model call of about 2,100 prompt and 180 completion tokens. At two claims at once, the 64 claims take about 18 minutes on the development laptop.

What the pilot found in the tooling. The harness was run against live services here for the first time. It recorded each resumed session's timeouts and concurrency only once, for the first session, which was fixed and is now tested; inference timed out under four concurrent claims on the CPU, and the run was resumed with a longer timeout and two claims at once; a model caught in a repetition loop was recorded as unreachable rather than as a model failure; the check of the working tree that stamps each record with its commit outlasted its timeout over the container's mount and lost one session's commit, which was fixed; and the harness process crashed twice in native code while parsing scraped pages. An interruption of the container runtime and both crashes were absorbed by the resume rule, which re-ran exactly the claims each had lost.

7.1.3 Beside the X-Fact Baselines

Table 7.2 reproduces the baselines of the X-Fact paper [18] from its Tables 3 and 4, read from the published version. Every figure is a macro-averaged F_1 in percent, the mean of four fine-tuning runs (standard deviation in parentheses), over X-Fact's seven labels. The paper reports neither accuracy nor any per-language figure. Its models are multilingual BERT

fine-tuned on 19,079 training claims; "Meta" adds the language, site, claimant and both dates as text, and the evidence models (Attn-EA) read the snippets of the top five Google results for each claim, collected when the dataset was built. There is no uniform random baseline in the paper: "Attn-EA (Random)" is the evidence model given random snippets, and the majority baseline always answers *false*.

Model (X-Fact paper)	**α_1 in-domain**	**α_2 out-of-domain**	**α_3 zero-shot**
Majority	6.9	10.6	7.6
Claim-Only	38.2 (0.9)	16.2 (0.9)	14.7 (0.6)
Claim-Only + Meta	39.4 (0.9)	15.4 (0.8)	16.7 (1.1)
Attn-EA (Random)	37.5 (0.8)	16.3 (0.5)	14.9 (1.2)
Attn-EA	38.9 (0.2)	15.7 (0.1)	16.5 (0.7)
Attn-EA + Meta	**41.9** (1.2)	15.4 (1.5)	16.0 (0.3)
Training augmented with 12,311 English PolitiFact claims (Table 4)			
Claim-Only + Meta	37.1 (2.7)	14.5 (0.5)	14.4 (0.3)
Attn-EA + Meta	38.0 (4.5)	14.7 (2.6)	14.3 (1.9)

Table 7.2: The X-Fact baselines as published (Gupta and Srikumar, Tables 3 and 4): macro-F_1 in percent over seven labels, mean (standard deviation) of four runs. α_1: 3,826 claims in the training languages and sources, Spanish (Chequeado) among them; α_2: 2,368 claims from unseen sources of four training languages; α_3: 3,381 claims in twelve unseen languages.

Table 7.3 sets the system's macro-F_1 beside the paper's closest rows. The Spanish test split is the only row of ours drawn the way α_1 is, and it covers one of α_1's twelve languages; the pilot's English half has no counterpart in the paper at all, since English is not among X-Fact's evaluation languages.

System and set	**Labels**	**Macro-F_1 ($\times$100)**
X-Fact paper, supervised mBERT, mean of 4 runs		
Majority, α_1	7	6.9
Claim-Only, α_1 / α_3	7	38.2 / 14.7
Attn-EA (claim + snippets), α_1 / α_3	7	38.9 / 16.5
Attn-EA + Meta (best), α_1 / α_3	7	41.9 / 16.0
This work, zero-shot `llama3.2:3b`, one run, 95% interval		
Spanish test split (α_1, Spanish only)	5	11.1 [7.9, 14.2]
Pilot, Spanish	5	13.0 [6.1, 19.6]
Pilot, English (not in X-Fact's evaluation)	3	35.8 [15.7, 55.4]
Closed-book (no evidence), Spanish test split	5	8.4 [4.9, 12.7]
Majority, Spanish test split	5	9.3

Table 7.3: The system beside the X-Fact baselines. The two halves differ in label set, training, evidence, sample and year of search; the list that follows says how, and none of the differences is small.

On the one row drawn as α_1 is, the system's macro-F_1 of 11.1 is 27 points below the paper's claim-only model and 31 below its best, above its majority baseline, and below what a uniform random guess is expected to score on the same claims (about 19). Read through the differences listed below, this says that the zero-shot pipeline does not approach a supervised model on X-Fact's own distribution; it does not say by how much the two would differ on equal terms.

The two halves of Table 7.3 are not directly comparable, for reasons that each move the figure by more than its interval:

- **Labels.** X-Fact scores seven labels; this work scores five, with *mostly false* merged into `FALSE`, *mostly true* mapped to `PARTIALLY_TRUE` and *other* dropped. A merged class is easier to hit, and macro-F_1 averages over fewer classes.
- **English.** English rows exist only in X-Fact's training data, keep PolitiFact's own six-point scale (no *partly true/misleading*, no *complicated*) and map PolitiFact's *half true* to `PARTIALLY_TRUE`, whereas X-Fact's master mapping sends *half true* to *partly true/misleading*, which this work maps to `MISLEADING`.
- **Sample.** The pilot is stratified, eight claims per language and label, so its labels are balanced by construction; X-Fact's test sets are not. Only the Spanish test split is drawn as α_1 is.
- **Size.** The paper's figures are means over four runs on thousands of claims, and their standard deviations (0.1 to 4.5 points) measure seeds, not sampling; this work's are single runs over 64 or 165 claims, with intervals 13 to 40 points wide.
- **Supervision.** mBERT was fine-tuned on 19,079 X-Fact claims and learned their label distribution and the claim-only cues the paper itself documents (its claim-only model beats the majority by 31 points on α_1). The model here never saw an X-Fact label, which makes α_3, where the supervised models also face unseen data, the fairer reference despite not containing Spanish.
- **Evidence and its date.** The paper's evidence is the snippets of the top five Google results, collected when the dataset was built; this work's is the open web of 13 October 2026, through the engines that answered SearXNG that day (Google and Brave did not), fetched in full and filtered by the pertinence gate. Section 7.1.4 is about what that changes.
- **Metadata and context.** The best paper model reads the claimant, the site and both dates; the pipeline reads the claim text alone, and Chequeado's claims are quotes stripped of speaker, place and moment.

7.1.4 What the Years Between Claim and Search Do

The English claims were made between 2008 and 2018, and X-Fact recorded no date for the Spanish ones; the dataset was published in 2021; the web was searched in October 2026; and the model's pretraining data ends in December 2023, according to its model card. A label that was right when a fact-checker wrote it is being compared with what the web says years later. Six effects follow. Each is counted on the pilot, with the accuracy of the claims it touches against the rest; at this size those differences are indications, not findings.

Verdict leakage. The fact-check itself is now on the web. PolitiFact's own verdict page was ranked for 9 of the 24 English claims and cited for 8, four of them from PolitiFact's staging server, which search engines index. Chequeado's page was never retrieved for any of the 40 Spanish claims, although every row links it and all 42 of those pages still answer: with

Google refusing the metasearch engine, the engines that answered did not surface Chequeado. The leak inflated less than feared, because the small model read the verdict page as evidence rather than as a ruling: 4 of the 9 leaked claims were right (44%, against 27% for the rest). A stronger model would read the ruling.

Temporal leakage. Of the 20 claims for which it can be decided, 16 cited a source published after the claim. The custom set's annotation guide forbids exactly this (Rule 3), but the pipeline searches without a date limit, and on X-Fact newer evidence is the norm: leaving out the claims with either leak lowers the pilot's accuracy from 29.7% to 23.3%.

Drift. Twenty of the 64 claims tie themselves to their own present, twelve with relative time ("hoy", "now", "este año") and nine with records or rankings ("de la historia", "the first time in history"). Without a claim date, "hoy" means the day of the search: a claim that the Asignación Universal por Hijo "today" reaches four million beneficiaries, true when Chequeado checked it, was judged false on articles from 2026; a 2012 claim about the worst cities for asthma was judged true on the 2024 and 2026 rankings. No accuracy penalty shows at this size, and none can be measured properly without a re-dated, re-labelled copy of each claim.

Link rot. Of the pilot's 305 reference links, the 120 English ones were never available (X-Fact ships PolitiFact's rows without them). Of the 69 Spanish links that are neither Chequeado's nor social-media posts, 14 no longer exist (404, a domain that no longer resolves, or a redirect to a home page) and 4 refused the check; 5 of the 14 survive in the Wayback Machine. The system retrieved none of the annotator's links, so recall measured in 2026 is bounded by what has survived, and by search engines that no longer reach the web of 2017.

Recency of search. Of the 235 ranked sources with a publication date, 53% appeared after X-Fact was built and 23% in 2026, the year of the search, the most frequent year of all. For the dated English claims the median ranked source is 6.8 years newer than the claim. The evidence a fact-checker read at the time is a minority of what search returns now.

The model's memory. Every claim and every fact-check of it predates the model's training cutoff, so the verdicts could be in its weights. Asked about each claim with no evidence, the model reported remembering a fact-check for none of the 64, and answered `FALSE` to 60 of them, scoring below the majority baseline. There is no sign that this 3-billion-parameter model recalls X-Fact's verdicts; a larger or more recent model should be probed the same way before its X-Fact score is read.

Consequence. For benchmark runs on fact-checker datasets, the fact-checkers' own domains should be excluded at retrieval time, through a per-run threshold that is off by default; in production, and on the custom set, a published fact-check is legitimate evidence and must stay available. This is recorded as a recommendation and has not been implemented.

7.2 RQ1 — Finding Constructive News on Chosen Topics

RQ1 asks whether a retriever of the open web, scoped to chosen topics, simplifies a journalist's task of finding constructive news on them. It is answered by two kinds of measurement: the work the scoping saves, and how well the ranked round orders what it finds (Section 6.1). The candidate labels used below were assigned from title, summary and link by the AI labelling pass and have not been checked by a person; the P1 items of Table 3.3 replace them with the author's own labels on at least two rounds, and repeat the discovery measurement on five days.

The work saved. On the first run of the discovery experiment (5 October, two candidates per source), a round scoped to Environment read 14 of the 34 sources then enabled (0.41× the every-topic baseline) and 153 of 1,536 links (0.10×), in 10 s against 15 s; Culture and Health together read 23 sources (0.68×) and 596 links (0.39×). A person reading a scoped round therefore starts from a tenth to two fifths of the links an unscoped reading would offer. One run is not an interval: the planned five days will give one.

The ranking separates what is worth offering. On the 117 candidates of the five rounds recorded on 6 October, labelled before any score existed (62 not worth offering, 33 worth offering, 22 clearly positive), the chosen score separated stories worth offering from those not worth it with an AUC of 0.778 [0.673, 0.868], and clearly positive stories from the rest with 0.806 [0.679, 0.916] (Table 7.4). In the rounds, the first twenty places of a Science and Society round held 8 stories not worth offering in the sources' order and 1 once ranked, and 14 clearly positive instead of 6; a Society round's first twenty went from 15 not worth offering to 6.

Table 7.4: How well each signal separates the candidates of 6 October (AUC, 95% bootstrap interval, 86 candidates the screen keeps). Labels by the AI labelling pass.

Signal	Worth offering vs not	Clearly positive vs rest
The chosen score (story 0.6, record 0.25, reliability 0.15)	0.778 [0.673, 0.868]	0.806 [0.679, 0.916]
The story alone	0.753 [0.647, 0.850]	0.812 [0.692, 0.917]
The source's record in the round	0.684 [0.556, 0.797]	0.664 [0.503, 0.808]
The source's reliability rating	0.486 [0.356, 0.605]	0.425 [0.294, 0.566]
Freshness (newer first)	0.301 [0.195, 0.416]	0.420 [0.269, 0.584]
The AI selection's score (`llama3.2:3b`)	0.650 [0.533, 0.759]	0.755 [0.612, 0.881]

Two signals were kept out or kept small on this evidence. Freshness points the wrong way within the 30-day limit (AUC 0.30): the day's breaking news is mostly the day's politics. The reliability rating carries no signal for the mission (0.49) and has the smallest weight; it is kept because how far a source is trusted is part of what the publication offers. The 3-billion-parameter AI selection separated the stories less well than the ranking on this sample.

Aligning the rounds with the mission and the groups. On 9 October five one-group rounds were run live, then again after each of four changes, and their 324 candidates labelled.

Over the first twenty places of the five rounds (100 places), pages that were not articles went from 13 to 0, stories of another group from 21 to 13, stories not worth offering from 29 to 17, and clearly positive stories stayed at 34. Health, the weakest round, went from 5, 5, 10 and 3 to 0, 2, 3 and 4 on the same four counts. The format rules caught 28 of the 35 non-articles and no article; the group-fit rule told a story of the round's group from one of another with an AUC of 0.874 [0.809, 0.930].

The topic emphasis. Re-scoring every candidate of ten labelled one-group rounds (200 places in their first twenties) at each emphasis (Table 7.5) shows the trade the person controls. At the default of 0.6 the first ten places hold only the group asked for (100 of 100) and the first twenty 194 of 200, against 184 before; the cost is clearly positive stories of the group, 62 down to 53, replaced by on-topic hard news that the AI selection and the person are left to sort. The AUC for a story of the group against another's rises from 0.706 [0.608, 0.797] to 0.825 [0.751, 0.891].

Table 7.5: Ten one-group rounds re-scored at each topic emphasis: counts over their first twenty places (200), and over their first ten (100) in the last column. Labels by the AI labelling pass.

Emphasis	Of the group asked for	Worth offering, of the group	Clearly positive, of the group	Not worth offering	First ten
Before (fixed weights)	184	154	62	36	94
0 (positive impact only)	174	150	61	33	89
0.3	187	154	61	38	94
0.5	192	150	53	45	98
0.6 (default)	**194**	**150**	**53**	**48**	**100**
0.7	194	139	46	60	100
1 (topic only)	195	136	52	63	97

What this answers and what it does not. On the evidence so far, the scoped, screened and ranked round does what RQ1 asks of it: a person reads a fraction of the sources and links, and the first twenty places hold markedly fewer stories not worth offering and stories of other groups than the sources' own order. Three limits keep this from being a final answer. The labels are an AI's, not a person's; the rounds come from one or two days, so the 25 rounds of 9 October are closer to five independent samples than to 25; and order is not supply — a round of thirty with fourteen stories worth offering still shows six that are not.

[To write: the P1 measurements: discovery on five days against the every-topic baseline, with intervals; precision of the first twenty and the ranking's AUC on at least two rounds labelled by the author.]

7.3 RQ2 — Verifying the Claims

RQ2 asks whether the pipeline can verify the claims of the news it retrieves, and how that can be quantified. The quantification is fixed in Section 6.1: accuracy, macro-F_1 and Cohen's κ against the gold labels, natural and synthetic claims apart, under three evidence conditions (none, retrieved, the annotator's), with retrieval recall and the stage each error comes from.

What the X-Fact pilot already shows. The only verification runs so far are on X-Fact (Section 7.1), and they answer RQ2 for political fact-checks rather than for the system's own domain. Four findings carry over.

- **The zero-shot 3B verifier is weak.** On the 64-claim pilot it reached 29.7% accuracy [18.8, 40.6] and a κ of 0.07 whose interval includes zero; on the 165 Spanish test claims, 19.4% [13.3, 25.5], below always answering the majority class.
- **It leans to `FALSE`.** It answered `FALSE` to 43 of 64 pilot claims and never `MISLEADING`; without evidence the same model answered `FALSE` to 60 of 64. The lean is the model's prior.
- **Retrieved evidence helps, a little.** Paired over the same claims, the pipeline beat the closed-book model by 10.9 accuracy points [3.1, 20.3] on the pilot, and by an amount indistinguishable from zero on the Spanish test split (3.0 points [−3.0, 9.1]).
- **Errors are both retrieval and reasoning.** Under the trace rule 44 of 45 wrong verdicts are reasoning errors; under the reference rule, 27 of the 29 it can judge are retrieval errors. The model misread what it was shown, and what it was shown was not what the fact-checker had used — no annotator link was retrieved at any depth.

The pilot also showed how much of an open-retrieval score on an old benchmark is about time rather than verification: 16 of the 20 claims for which it can be decided cited a source newer than the claim, and the fact-checker's own verdict page was ranked for 9 of the 24 English claims (Section 7.1.4). The custom set avoids the worst of this by construction: its claims are weeks old, not years, and its annotation rule restricts the gold label to evidence available at publication time.

On the custom set: the test half. The split was frozen on 19 October (Subsection 6.3.8) and the pipeline run on both halves the same day: `llama3.2:3b` on a processor without a graphics card, the corpus snapshot, every claim scored and none set apart. Every decision — here, whether to turn on the second look — was taken on the tuning half before the test half's report was read. Table 7.6 gives the test half; its headline is the default pipeline, the first pass.

	n	Accuracy [95% CI]	Macro-F_1	κ [95% CI]
All claims	70	52.9% [41.4, 64.3]	0.227	0.195 [0.035, 0.359]
Natural claims	55	45.5% [32.7, 58.2]	0.164	−0.013 [−0.104, 0.078]
Synthetic `FALSE`	15	80.0% [60.0, 100]	—	—
English / Spanish	47 / 23	51.1% / 56.5%	0.279 / 0.286	0.191 / 0.199
Tuning half, for reference	69	44.9% [33.3, 56.5]	—	—

Table 7.6: The custom set's test half, first pass, against the gold labels (`backend/data/evaluation/reports/manual-test/`). Percentile bootstrap intervals over claims, 10,000 resamples.

Three findings. **On the natural claims the pipeline agrees with the labels no better than chance** ($\kappa = -0.01$): it answered `FALSE` to 18 of the 46 `TRUE` claims, the lean towards

FALSE that the X-Fact pilot showed. **It catches the synthetic refuted claims** — 12 of 15 — whose evidence plainly contradicts a substituted figure or date. And **the labels' source matters**: on the 20 natural claims the annotator labelled the pipeline agrees with gold on 6 (30%), on the 35 the AI pass labelled on 19 (54%). The samples are small and the intervals overlap, but the direction is the one a gold set labelled by a language model invites — a second model agreeing with the first — and is why the annotator's review of those labels (Section 7.4) comes before any of these numbers is final.

Retrieval found evidence for every claim (4.9 ranked sources from 21 candidates), but rarely the annotator's: the annotator's link was among the candidates for 14.5% of the claims, ranked for 8.7% and cited for 5.8%; the annotator's domain among the candidates for 34.8%, ranked for 23.2% and cited for 13.0%.

The tuning loop and the second look. On the tuning half every claim whose verdict disagreed with gold — 38 of 69 — was looked at again (Subsection 6.4.6):

- **9 fixed** by the second look: a second search, planned from what the first verdict said was missing, found the evidence.
- **8 retrieval errors**: still wrong after the second look, but right when judged on the annotator's own pages — the decisive source exists and the search never found it.
- **15 for a person to decide**: on the annotator's own pages the model gives the pipeline's verdict again. They are mostly the model's: a TRUE label judged FALSE (5), a synthetic FALSE judged TRUE on its source's evidence (4, the substitution unnoticed), an UNVERIFIED label judged FALSE (3); several explanations restate the claim as supported and conclude FALSE.
- 2 reasoning errors (a third verdict on the annotator's pages), 2 claims whose pages could not be read and 2 without pages.

Had the gold label chosen which claims get a second look, accuracy would have risen from 44.9% to 58.0%. The pipeline cannot use the gold label; its own trigger — a verdict UNVERIFIED or below 0.5 confidence — fired on 15 claims and gained 2.9 points [−2.9, +8.7], fixing 3 claims and breaking 1. That gain does not exclude zero and costs a second look on a fifth of the claims, so the second look was left off by default. On the test half, where the pipeline took its second looks itself, it fired on 17 claims and changed the accuracy by 0.0 points [−4.3, +4.3], one claim fixed and one broken: the decision held. The loop's diagnosis says where the next points are. The model is wrong with confidence more often than it is unsure, so a trigger on doubt cannot find most of its errors; and on the annotator's own evidence it still contradicts 15 labels, most of them correct. The larger error is the 3-billion-parameter verifier's reasoning, which the model comparison (Section 7.6) addresses, not the search.

[To write: Closed-book and oracle conditions on the test half; stage attribution under both rules; all three after the annotator's review of the AI-proposed labels.]

7.4 RQ3 — How Close a Model Comes to a Person

RQ3 compares a person and a language model taking the same decision on the same input, in three places (Section 6.1): which candidates to publish, which label a claim deserves, and the verdict; the annotator's agreement with their own blind re-label is the ceiling. None of the three has yet been measured against a person, and this section says what exists and what is missing.

Labelling: the AI pass against the annotator. 108 of the 139 facts in the custom set — 78 natural claims and the 30 synthetic ones — carry a label proposed by the AI labelling pass, each marked as such in its note. The annotator's review, keeping or changing each, is P0 work. Its outcome is the first RQ3 number: the agreement, as Cohen's κ with its confusion matrix, between a strong language model applying the annotation guide and the person who wrote it. From the review onward the annotator labels each batch before the AI does, so that every new fact adds to that comparison without anchoring.

[To write: κ and observed agreement between the AI pass and the annotator's review, on the 108 facts, with the confusion matrix and the rules behind the disagreements; and the annotator's blind re-label of the hash-chosen 20%, as the ceiling.]

Selection: the AI against the editor. The only comparison so far is indirect: on the 6 October candidates, the AI selection's 0–10 score separated stories worth offering from the rest with an AUC of 0.650 [0.533, 0.759], below the ranking's 0.778, but both were measured against labels the AI labelling pass assigned, not against an editor (Table 7.4). Rounds already sent with the AI's help are *assisted*, and their agreement is inflated, because an editor anchors on a proposal; the number to report comes from *blind* rounds, where the editor sends a choice before the model is run.

[To write: ten blind and ten assisted rounds: precision, recall and Jaccard of the AI's picks against the editor's, and the AUC of its score, blind against assisted.]

Consistency: each against itself. The protocol requires a fifth of the test half to be evaluated a second time (Subsection 6.3.6): 14 claims. The system's half of that requirement is done. The 14 were run through the pipeline again 1 hour 51 minutes after the test run, with the same model and settings (Table 7.7). 13 of the 14 verdicts were the same twice — $\kappa = 0.84$ [0.44, 1.00] — although the evidence was not: the two runs' ranked sources overlapped by a mean Jaccard of 0.58, identically for 5 claims and not at all for 2. The one verdict that changed, `FALSE` then `TRUE` for a claim labelled `TRUE`, changed with entirely different evidence; no verdict changed on the same evidence. Over a short interval, then, the system's inconsistency comes from the web it searches rather than from the model, and it is small next to its disagreement with the labels. With 14 claims the interval is wide, and two hours is not the week the annotator's re-label waits. The annotator's half — the blind second label of the same 14 — is still to be done.

[To write: The annotator's blind re-label of the same 14 claims: observed agreement and κ, beside the

Between two runs of the same 14 claims	n	Agreement [95% CI]	κ [95% CI]
Final verdict	14	92.9% [78.6, 100]	0.837 [0.440, 1.000]
The model's raw verdict	14	92.9% [78.6, 100]	0.837 [0.440, 1.000]
First pass, before any second look	14	92.9% [78.6, 100]	0.811 [0.000, 1.000]

Table 7.7: Run-to-run consistency on the re-check sample (`reports/manual-test-recheck/consistency/`).

system's.]

Verdicts: the pipeline against the annotator. The verifier's agreement with the annotator is the κ of the RQ2 runs on the reviewed custom set (Section 7.3), read against the annotator's self-agreement. Before the review, on the test half it agrees with the 20 labels the annotator set on 6 and with the 35 the AI pass set on 19.

[To write: κ of the pipeline against the reviewed labels, beside the self-agreement ceiling and the AI pass's agreement.]

7.5 RQ4 — How Much of the Work the System Takes On

RQ4 asks, of the three steps joined, what share of a journalist's work the system takes on and at what quality. Its answer is a profile, built step by step from the other questions' measurements and from three of its own.

Choosing the claims. Over the nine daily labelling batches from 29 September to 14 October, the claim selector proposed 214 claims, of which 136 were labelled and 78 rejected: a precision of 0.636 [0.570, 0.701] (bootstrap over claims). It is markedly higher in English (98 of 141, 0.70) than in Spanish (38 of 73, 0.52). The rejection reasons were opinion or interpretation (22), a fragment of a sentence (21), a trivial statement (15), a prediction (8), a duplicate (6) and others (6). Two cautions apply. The batches from early October onwards were worked by the AI labelling pass, so the figure mixes the annotator's decisions with the AI's and will be recomputed on the reviewed batches; and the claim splitter was changed on 9 October, after fragments cut at abbreviations were found, so the later batches measure a slightly different selector. About two in three proposed claims are worth checking; the system takes on the choice of claims, but a person still discards about one in three.

Verifying them. On the X-Fact pilot the verifier gave a definitive verdict for 95.3% of claims (its coverage) and was right on 31.1% of those (its selective accuracy): it almost never abstains, and when it answers it is mostly wrong. On that evidence the 3B verifier cannot take over verification on political claims; whether it can on constructive news, and what share of its verdicts would be usable without re-checking, is the custom-set measurement of RQ2.

[To write: coverage, selective accuracy, the share usable without re-checking and the confidently-wrong share on the custom test split.]

Time. A claim took 36.5 s at the median on the development laptop in the pilot (90th percentile 59.9 s), of which 14.8 s retrieval, 8.6 s ranking and a 5.6 s model call; on the CPU-only production stack a four-claim article took 184 s, with the model 180 of those seconds. A round of discovery takes about a minute; the AI selection about 70 s for 117 candidates with the 3B model. Against these, the time a person spends is not yet measured: the labelling tool does not record time per fact.

[To write: time per fact from the labelling tool (median and interquartile range), against the system's time per claim; the effort estimate of Subsection 6.4.5.]

The profile so far. Finding: the system reads a tenth to two fifths of the links and orders them so that the first twenty places hold few stories not worth offering, on AI labels (Section 7.2). Choosing claims: about two in three of its proposals are checkable. Verifying: not yet at a quality a journalist could rely on with the 3B model, on the only data measured. At every step a person is still needed for the judgement the system is weakest at — hard news against constructive news on the same topic, opinion against fact in Spanish, and whether a verdict can be trusted.

7.6 Model Comparison

7.6.1 Verification Models

The verifier was compared on the train half of the custom set (Subsection 6.6.1), each model alone on the development machine's processor, the Qwen models with their reasoning mode off. Two conditions were run: each model with the pipeline's own search, and each model on the annotator's own reference pages, the same evidence for every model, so that the models differ by their reading alone (Table 7.8).

Train half, 69 claims	`llama3.2:3b`	`qwen3.5:4b`	`qwen3.5:9b`
Accuracy, own search	44.9%	30.4%	not run
Accuracy, the annotator's pages	55.1%	34.8%	56.5%
paired against the baseline	—	−20.3 [−34.8, −5.8]	+1.4 [−11.6, +14.5]
κ, the annotator's pages	0.190	0.168	0.307
Right when it commits	55.6%	39.6%	72.9%
The model's answer, median (s)	21.7	30.6	61.4

Table 7.8: The verifier models on the train half: with each model's own search, and on the annotator's own pages (the oracle condition), where the evidence is the same for every model. Intervals are paired 95% bootstrap intervals over claims.

The baseline stays. With its own search `qwen3.5:4b` scored 14.5 points below the baseline ($[-29.0, 0.0]$); on the annotator's pages, 20.3 points below, an interval that excludes zero. Two runs of the baseline on the same half differ by −2.9 points ($[-10.1, +4.3]$; 87% of verdicts the same, κ 0.76), so both gaps lie beyond the noise of the comparison. The pipeline's guardrails changed only 6 of its 69 answers; the rest is its reading. It answers `UNVERIFIED` when the pages do not repeat the claim and `PARTIALLY_TRUE` for details the claim does not make — one claim was marked partially true because its pages did not mention a larger diagram the

claim never mentioned. `qwen3.5:9b`, on the same pages, was as accurate as the baseline (+1.4 points, $[-11.6, +14.5]$) but far more careful: it left 21 claims `UNVERIFIED` and answered `FALSE` to 6 true ones, where the baseline did to 14, so the verdicts it committed to were right 72.9% of the time against 55.6% (+17.4, $[+2.7, +32.0]$), at three times the model's time. Its own search was not run. By the rule written before the runs — accuracy, then speed — the baseline remains the verifier. The rule leaves one decision open: if a wrong verdict costs more than an honest `UNVERIFIED`, as a fact-checker's purpose suggests, the 9B model is the better verifier, and its run with its own search is the one that would settle it.

What retrieval costs. The same comparison measures RQ2's second gap on the train half: the baseline on the annotator's pages against its own search, +10.1 points ($[-2.9, +24.6]$), macro-F_1 +0.142 ($[+0.002, +0.257]$). Even on those pages it answers `FALSE` to 14 of 45 true claims: the lean the tuning loop found is the model's. The test half's three conditions are left (Table 3.3, P0.5).

7.6.2 Writing Models

The writer. The verifier, `llama3.2:3b`, was measured unfit to write: no citation in four drafts, and an unverified claim stated as fact in three (Section 5.10). Two openly licensed candidates were tried on 19 October with the same sample article, one draft in Catalan and one in Spanish each, on a processor with no graphics card, after the evaluation runs had finished (Table 7.9).

	`llama3.2:3b`	`qwen3.5:4b`	`qwen3.5:9b`
Seconds a draft (ca / es)	17–25 (GPU)	169 / 208	351 / 256
Citations (ca / es)	none in 4 drafts	14 / 26	22 / 21
Sentences with no citation	9–20 a draft	0 / 1	0 / 0
Metadata complete	2 of 4	2 of 2	2 of 2
Withheld claim stated	3 of 4	none	none
Memory loaded, 16k context	—	3.7 GB	6.7 GB

Table 7.9: The writer trial: one sample article, the two Qwen3.5 models with their reasoning mode off, against the verifier's drafts of 15 October.

Both candidates fix what ruled the 3B model out. Read by a person, their Spanish was good, and their Catalan decided between them: the 4B model's had slips a reader notices; the 9B model wrote "litigi" (a lawsuit) for *liti* (lithium) throughout. The 4B model is recommended: twice as fast on a processor, half the memory, and the better Catalan on this sample. One article per language is a trial, not a benchmark, and the Catalan of either needs a person's reading before it is published, which the flow already requires.

The reviewer. The draft reviewer was tried on the two seeded drafts of Subsection 6.6.2, twice each per model (Table 7.10). Only `qwen3.5:9b` found every planted defect — both spelling errors, the contradicted figure, the claim judged `FALSE` — and every one of its answers could be read. The 3B model found no spelling error in four reviews and scored the hallucination metric on an inverted scale; the 4B model was good when it answered, but

one of its reviews was two answers that were not JSON at all. The 9B model still invents a little (twice, that the Catalan subtitle states the false claim), and its English corrections were refused for adding a figure — the check cannot tell a figure put right from a new one, and keeps to the safe side. It is the recommended reviewer: a different model from the writer, as the setting is meant, at about three and a half minutes a review on a processor.

	`llama3.2:3b`	`qwen3.5:4b`	`qwen3.5:9b`
Planted defects found (of 10)	5	6	10
Spelling errors found (of 4)	0	2	4
Metrics parsed (of 28)	24	21	28
Corrections offered (of 4)	0	2	2
Seconds a review	26–58	106–142, 488 once	200–224

Table 7.10: The draft reviewer on the two seeded drafts, twice each: defects found by the model itself, before the deterministic checks bind the scores.

The editor. The writing-model benchmark (Subsection 6.6.2) was run on 19 October for the three local models, one repeat each. The 3B model answered every text usably (100%) and detected the planted defect in 80% of pairs ($[72, 88]$), a defect lowering its metric by only 3 to 46 points. The 4B model's first run fell under the format gate (80%) while detecting 93% ($[82, 100]$) of the pairs it scored: seven of its eight failures were answers that nested each metric inside the one before, valid JSON with every score in it, which the corrector read at the top level only. The reader was corrected, and the 4B model's re-run and the 9B model's run were started; both were stopped unfinished on 19 October and resume where they stopped.

[To write: the 4B model's re-run under the corrected reader and the 9B model's run: format compliance and detection, paired against the 3B.]

7.6.3 Cost and Response Time

Every model here is local and openly licensed: its per-token cost is zero, and its cost is the machine's time. Response time is therefore reported beside quality for every role, measured on the development machine's processor (no graphics card), each model alone on it: two runs at once share the processor, and their times are each other's waiting (Table 7.11).

The model is three quarters of a claim's time (77% for the baseline), and about a third of its span is waiting for the other claim's call: retrieval and ranking together take 14 seconds. A faster verifier is therefore the only large speed-up left, and the candidate tried was slower as well as less accurate. End to end, an article of four claims took 184 seconds in the production stack on 1 October; a draft takes about three minutes and its review about three and a half more, with one model swap between them, since the production stack holds one model in memory at a time (Section 5.8).

For the X-Fact pilot of 13 October, on the development laptop's graphics card, the baseline's model call took 5.6 s at the median (90th percentile 9.5 s), 28.3 completion tokens per second against 5.2 on the processor: the same model is about six times faster on a graphics card, which is the hardware choice the deployment's cost estimate does not yet include.

Role	Model	Median	90th pct.
Verifying a claim, whole	`llama3.2:3b`	72.7 s	94.8 s
evidence retrieval		9.0 s	19.9 s
ranking		5.1 s	6.9 s
the model, with its wait		55.6 s	77.8 s
the model's answer alone		34.0 s	48.9 s
Verifying a claim, whole	`qwen3.5:4b`	109.9 s	130.5 s
Writing a draft	`qwen3.5:4b`	169 s (ca), 208 s (es)	
Reviewing a draft	`qwen3.5:9b`	200–224 s	
Judging a text (benchmark)	`llama3.2:3b`	28.1 s	—
Judging a text (benchmark)	`qwen3.5:4b`	44.6 s	—

Table 7.11: Response time per role on the development machine's processor, each model alone. A claim's stages are the pipeline's own events; the model's span includes waiting for the other claim's call (two claims at once, one model call at a time, as in production).

7.7 The Stages Before the Verdict

Discovery and the candidate ranking, measured on 5, 6, 9 and 13 October, answer RQ1 and are reported in Section 7.2. This section reports the other stages that decide which articles get a verdict.

The mission screen. On 527 live candidates from Culture, Society and Health rounds (6 October), each title and summary was compared with the topics and the off-mission descriptions exactly as the screen does. At a margin of 0.04 it left out 68 of the 484 candidates from general outlets; read by hand, nearly all were elections, crime, accidents, storms, market wire items and celebrity interviews, the doubtful ones four war-history pieces and one story on data-centre regulation. Between margins of 0.03 and 0.04 about 3 in 17 candidates were worth keeping, which is why the line is at 0.04. Widened the same day with descriptions for labour disputes and markets, it left out 115 of 545 instead of 80, all 36 new ones being strikes, protests, finance and election politics, with one story wrongly let back in. The sentiment model was tried as a signal and rejected: it rated an endangered primate's birth 0.89 negative and a lost theatre reborn 0.89 negative, because a problem being solved reads as negative.

Admission. Replaying the admission thresholds offline over the articles of the 41 facts labelled by 5 October, 71% would be admitted at the values of the local configuration that day (topic minimum 0.35, impact minimum 0.3, against a declared default of 0.1 for the latter); at a topic minimum of 0.40, 61% at an impact minimum of 0.1 and 49% at 0.3. Four articles (10%) have no topic at the classifier's floor and are rejected whatever the minimum.

[To write: the admission experiment over at least 100 round candidates the editor chose for or against: admission rate, precision and recall over the grid, and the AUC of the positive-impact score.]

The topic classifier. On the same 41 articles, the classifier's top topic was the gold one 54% of the time [37, 68], the gold topic was in its top three 78% [66, 90], its top group was right 73% [59, 85], and 10% [2, 20] of articles had no topic at the 0.35 floor. Society is the weak group: five of its nine articles were classified as environment. Topics steer admission

and the balance of the set, not verdicts, which is why improving the classifier was de-scoped on 29 September.

[To write: the classifier again on the reviewed custom set (at least 100 articles): top-1, top-3, group accuracy, the no-topic rate and the group confusion; and whether the 0.35 floor stays.]

Time per stage. The X-Fact pilot gives the claim-level split on the development laptop (Section 7.5); the production stack's single analysis gives the CPU's.

[To write: time per stage — enrichment, admission, retrieval, ranking and the model — over the pilot and one ingestion round, on the development GPU and the production CPU, from the metrics endpoint and the job journal.]

7.8 Discussion

The steps before the verdict work better than the verdict. The clearest results of the project so far are about finding the news, not about verifying it. Scoping, screening and ranking a round do what an editor needs from them: a fraction of the links, and first places that hold few stories not worth offering, with signals that cost no model call (Section 7.2). That is consistent with the literature's view that automation is most useful where it assists a step of the work rather than replaces the person [28]. It also rests on AI labels, and the first measurement against a person may lower it.

A small local verifier is not yet a fact-checker. On X-Fact the 3B model is barely above chance and leans to `FALSE` whatever the evidence. Part of that is the benchmark: claims years old, searched on today's web, with the verdict sometimes on the page; part is the label mapping and the stratified sample; and part is the model. The pilot cannot separate those parts, and a stronger model on the same claims, and the same model on constructive news, are the two measurements that would. The design choices of the verifier — a closed label set, forced citation, verbatim quotes, and a confidence recomputed from the evidence — make its failures visible and its right answers checkable, but they do not make a weak model strong.

Retrieval and reasoning fail together. The disagreement between the two attribution rules on the pilot (28 of 29 claims) is the most instructive single number: the model misread what it was shown, and it was never shown what the fact-checker used. Improving either alone would not have fixed most of those claims. That is the case for the oracle-evidence condition of RQ2, which gives the verifier the annotator's own evidence and so separates the two.

Against the state of the art. The supervised baselines of the X-Fact paper reach a macro-F_1 of 38 to 42 on its in-domain test set; the zero-shot pipeline reaches 11 on the Spanish part of it. The difference is large and in the expected direction, and Section 7.1.3 lists why the two are not like for like. AVeriTeC's restriction of evidence to what existed before the claim [33] and Glockner and colleagues' demonstration of leaked evidence [14] are both borne out by the pilot: 23% of the dated ranked sources were from the year of the search, the most frequent year of all.

What the evaluation itself contributes. The harness, the custom set and the experiments are designed so that every number comes with its interval, its provenance and the claims set apart. That is what allowed the pilot to show that one result was a measurement of time and not of verification, and what will make the custom-set results interpretable when they come.

[To write: the discussion of the custom-set results per research question, once measured.]

7.9 Threats to Validity

Internal validity.

- **Labels by a model.** Most labels of the custom set and all the candidate labels of the selection experiments were assigned by an AI labelling pass. Until the annotator reviews them, a result measured against them compares the system with another model. Every such result is marked as such.
- **One annotator.** Self-agreement measures whether the guide is applied consistently, not whether it is right; a systematic misreading applied twice agrees with itself.
- **The selector chooses the set.** The custom set's claims are those the system's own selector proposed, so verification is measured on the system's choice of claims, not on an independent sample of all check-worthy claims.
- **Tuning on the test data.** No threshold has been tuned on labelled data yet; the frozen split is there so that none is tuned on the claims it is reported on.

Construct validity.

- **Synthetic refuted claims.** The `FALSE` class of the custom set is built by substitution. It tests whether the system notices a claim contradicting evidence it can find, not how often or how real constructive news is wrong; it is reported apart.
- **Today's web against the claim's date.** The system searches without a date limit; the annotator may use only evidence available at publication. Newer evidence is flagged, not removed, and on X-Fact it dominated (Section 7.1.4).
- **Label mapping.** X-Fact's seven labels are mapped onto the system's five, which changes what macro-F_1 averages over.

External validity.

- **Size.** With about 100 test claims, a 95% interval on accuracy is roughly ± 10 points: only large differences between models or conditions will show.
- **Two languages, 37 sources, one period.** The results hold for these sources in October 2026, in English and Spanish; the selection experiments come from one or two days of feeds.

- **Search availability.** Results depend on which engines answered SearXNG on the day; on 13 October Google and Brave refused it. Every run records what the health and search checks said before it, and claims whose search failed are set apart, but a partially answering engine set still shapes the evidence.
- **A moving web.** A search cannot be repeated: the same query returns different pages on different days. Strategies are compared on the same days, interleaved, rather than across time.

Chapter 8

Sustainability and Ethical Implications

8.1 Sustainability Report

8.1.1 Environmental Dimension

No model was trained. The largest environmental decision of the project is negative: every model is used as published, from open weights. Training or fine-tuning a language model, even a small one, costs orders of magnitude more energy than running it, and none of the research questions needed it. The verifier is a 3-billion-parameter model rather than an 8-billion one, chosen for its footprint.

The energy of running it. The energy of the system is the energy of the machine it runs on, for the time it runs. That time is measured: about 36 s per claim on the development laptop, of which about 15 s is search and page fetching, 9 s ranking and 6 s the model; and about three minutes per four-claim article on a CPU-only server, almost all of it the model. The power drawn was not measured. As an order of magnitude only, at an assumed 100 W for a laptop under load, the whole X-Fact pilot (64 claims, two at a time, about 18 minutes of wall-clock time) cost about 30 Wh, or about half a watt-hour per claim; the figure should be read as an estimate under that assumption, not as a measurement. Several design choices reduce the work per article: the admission filter rejects an article before any search or model call; the claim selector keeps at most four claims; claims with no evidence are not sent to the model; repeated queries are sent once; and the selection step means only the articles a person or the model chose are analysed at all, rather than everything discovery finds.

The server. The production target is one shared virtual machine, sized from measured memory use, rather than a set of services each sized for a peak; it was never run, so it has consumed nothing. Choosing a region whose electricity has a low carbon intensity is a decision for the real deployment.

8.1.2 Economic Dimension

The cost of the project is its 900 hours of work, a laptop already owned, and no licences (Section 3.5). The cost of running it was a design requirement from the start (Section 1.2): no paid search or model API, so the marginal cost of checking a claim is machine time, and the whole stack fits on one virtual machine of about 115 USD a month. That keeps the system within reach of a small newsroom or a single researcher, and makes every result reproducible without an account with a provider whose model or index may change underneath it.

The trade-off is explicit. A free metasearch engine is refused by the engines behind it more often than a paid search API would be, and a small local model verifies worse than a large hosted one. Both costs are measured rather than hidden: failed searches are reported apart, and the model benchmarks compare the local model with a hosted one on accuracy, latency and price. An organisation that can pay for a search API or a hosted model can switch either by configuration.

8.1.3 Social Dimension

Who benefits. Readers who want news that is constructive and verified get articles whose claims carry a verdict and the evidence behind it, instead of a promise. Journalists and small newsrooms get a tool that reads the sources, screens what does not fit, proposes the claims worth checking and does a first search for evidence — the steps that take most of the time before any judgement is made.

Who could be harmed. Three groups. Readers, if a wrong verdict is presented with more authority than it deserves; the system answers by preferring "not verified", by setting failures apart and by showing every verdict's evidence, but the X-Fact pilot shows that the current verifier is often wrong, and a reader shown a confident `TRUE` cannot tell. Journalists, if a tool like this is used to reduce newsrooms rather than to support them; RQ4 is asked as a question of extent precisely because the results show where a person is still needed. And the sources, whose articles are read automatically; how that is limited is in Section 8.2.

Trust. A verdict that can be inspected is the point of the system: a reader can follow each claim to its sources and see why each was trusted, including when a source's reliability is unknown. Whether that transparency increases readers' trust, or only informs those who already look, has not been studied here.

8.2 Ethics of Data Management and Analysis

8.2.1 Collection of Third-Party Content

The system reads articles that other people wrote and published. What it does with them is limited in four ways.

- **What is stored.** The text of an analysed article is kept in the data lake, which is local and not published, so that a verdict can be traced back to what was read. Evidence

pages are read for the claim they bear on; only their title, a quoted passage and their address are kept with the verdict, and the evaluation harness stores no page body at all. Raw evaluation runs, which hold third-party titles and quotes, are not committed to the repository.

- **What is shown.** The public reader view shows an article's lead and a link to the original, never its body or a scraped page; a test holds that rule. The original publisher keeps the reader.
- **How it asks.** The scraper names itself in its user-agent, asks each source for a few articles per run rather than crawling it, and is run on request, never on a timer. It does not read `robots.txt` (Section 5.2.13), which is a limitation to correct before any public deployment.
- **When it is refused.** A source that refuses automated access is disabled, not worked around: EFE and Reuters, which answered every request with 403 and 401 for two weeks, were disabled on 6 October. The second search route, when challenged as a bot, pauses rather than pretending to be a browser.

Fact-checking organisations' pages are used as evidence when a search returns them, and the two Spanish organisations are rated as sources, but they are not ingested: a fact-check quotes the claim it debunks, and the system would otherwise present that quotation as an article's claim.

8.2.2 Personal Data and Privacy

The system processes no data about its users: there are no accounts, no profiles and no tracking, a consequence of cutting them from the scope. The personal data it does process are the names of people that appear in published news. The entity recogniser extracts them, and the graph stores them as entity nodes linked to the articles and claims that mention them, beside the claimant of each labelled fact.

These are data that a publisher has already made public, processed for journalistic and research purposes, for which the General Data Protection Regulation provides room for exemptions and derogations (Article 85 for freedom of expression and information, Article 89 for scientific research, each subject to safeguards); they are not combined with any other source about the person, and nothing in the system profiles individuals. A public deployment would still need its own assessment of the legal basis. Three precautions follow nonetheless. The graph and the lake are local and not published; the graph's query console is read-only and behind an API key; and an entity is keyed by its type and name only, with no attempt to identify the person beyond what the article says. A claim attributed to a named person carries a verdict about the *claim*, not about the person, and the reader view presents it that way.

8.2.3 Responsibility for Automated Verdicts

An automated verdict that a claim is false can harm whoever made the claim, and one that a claim is true can mislead whoever reads it. The system is built to keep responsibility with people and to make its own contribution inspectable.

- **Failing towards caution.** When the evidence does not decide, the answer is `UNVERIFIED`; a definitive verdict that cites nothing, or only sources the model itself called unrelated, is downgraded; a `TRUE` confirmed by a single outlet keeps a capped confidence.
- **Failures are not verdicts.** A claim whose search or model failed is marked as such, and the reader view says "not checked" rather than showing the `UNVERIFIED` it is stored as. A search outage is never presented as a judgement about a claim.
- **Every verdict shows its work.** The model's reasoning, its answer before the guardrails changed it, each source's stance and quoted passage, and whether each source's reliability is a real rating or a default, are visible to the reader.
- **Text written by a model says so.** An article the writing flow produced is published only after a person approves it, and states before its text that an AI model wrote it, naming the model (Section 5.10).
- **A person decides what is published.** The system proposes and the editor chooses. Its accuracy has been measured on one benchmark and found low, and it is not presented as a fact-checker whose verdicts stand on their own.

8.3 Equity and Bias Analysis

8.3.1 Biases in the Data

- **The source catalogue.** The 37 sources were chosen by hand, and their reliability ratings were set by hand. They are mostly European and North American outlets, plus Infobae (Argentina), The Conversation (Australia) and international bodies; voices from the rest of the Global South, local and community media, and outlets in other languages are absent. Only the configured domains have a real rating; every other domain found as evidence gets a neutral default, flagged as unrated, which favours sources the catalogue already knows when the ranking breaks ties.
- **Two languages.** English and Spanish only; the Spanish side has fewer enabled sources (13 against 20), only three in the environment group, and a claim selector that is markedly less precise in Spanish (0.52 against 0.70).
- **The custom set's balance.** By design it is balanced over five topic groups and five verdicts, but the natural claims contain no `FALSE` and one `MISLEADING`, two thirds of the facts are in English, and science, environment and health hold more facts than society and culture (Table 6.2).

- **One annotator and an AI pass.** The labels reflect one person's application of the guide, and for most facts the proposals of one language model, whose own biases in reading evidence are not known.
- **X-Fact's domain.** The external benchmark is political statements checked by fact-checkers, selected because someone disputed them, from two organisations in the pilot.

8.3.2 Biases in the Processes

- **The mission screen** encodes a view of what the publication does not cover — politics, protests, crime, markets — in nine written descriptions and their margins. It leaves out labour disputes and protests along with party politics, which is an editorial position, not a neutral filter, and it can leave out a constructive story whose words sit near one of those subjects (a prison-to-firefighter jobs programme was read as crime). The three positive-only outlets are exempt, which favours their editors' judgement over the screen's.
- **The ranking and the AI selection** encode a definition of positive impact written by the author. The ranking's weights were fitted on labels an AI assigned; the AI selection scores threat stories on topic highly and candidates without a summary low.
- **The search engines** decide what evidence exists for the system: their ranking, their refusals on a given day, and their coverage of Spanish sources. Google and Brave refused the metasearch engine during the pilot.
- **The models.** The embedding model places stories in a space learned from its training data; the sentiment model reads a problem being solved as negative, which is why it was kept out of the mission screen; and the verifier leans to `FALSE` on political claims whatever the evidence.

8.3.3 Actions Taken to Reduce Them

- **Stratification.** The custom set targets six facts in each of 25 verdict-by-group cells, the daily batch prefers the groups that are short, and cells that stay short are reported with their real counts rather than padded.
- **Rules fixed before labelling.** The four tie-break rules were written before the first label, and those that can be checked are enforced by the labelling tool on save.
- **The blind re-label and κ.** 20% of the facts, chosen by hash, are labelled again blind and at least a week later; agreement is measured on the first label.
- **AI labels disclosed and reviewed.** Every label the AI pass proposed says so in its record, none counts as gold until the annotator reviews it, and their agreement is a reported result.
- **Partitions reported apart.** Natural and synthetic claims, English and Spanish, each topic group, and the claims whose search or model failed are reported separately, so

that a strength in one cannot hide a weakness in another.

- **Every rating says what it is.** An unrated domain is shown as unrated; a screened candidate is listed with the reason and the margin that left it out, so an editor can see what the screen removed.
- **Two languages from the start.** Every lexicon exists in both languages, and a source in a third language fails the test suite rather than being scored with English word lists — a fix for a measured bias, when the same story scored 0.875 for constructiveness in English and 0.0 in Spanish.

8.3.4 Critical Analysis of the Impact

Not every bias above matters equally. Three can change what a reader is shown or the verdict a claim receives, and remain.

- **The screen and the ranking decide what is offered.** A story the screen leaves out never reaches an editor unless they open the list of what was left out; a story ranked below the first twenty is rarely read. Both rest on descriptions and labels written by the author or an AI, so the publication's view of "constructive" is applied at scale before a person sees anything. Showing what was left out and why is a mitigation, not a cure.
- **The evidence decides the verdict.** The search engines' coverage and the catalogue's ratings favour well-known Western outlets, so a claim about a project reported mainly by local or non-English media is more likely to come back `UNVERIFIED`, which reads as less trustworthy even though it is a statement about the evidence, not the claim.
- **Spanish is served less well.** Fewer sources, a less precise claim selector and a worse pilot result in Spanish (22.5% against 41.7% on the pilot, with different label sets) mean that Spanish-language news is checked less reliably.

The biases in the evaluation data, by contrast, affect what this thesis can claim rather than what a reader sees; they are reported as threats to validity (Section 7.9). The most important of them — labels proposed by a model — is also the one the project can remove before the end, by the annotator's review.

8.3.5 Inclusive Language, References and Examples

This document refers to people by role — the annotator, the editor, the reader, the journalist — and uses gender-neutral forms when a person's gender is not known or not relevant ("they", "themself"). Examples are taken from the system's real data, and were chosen to illustrate a mechanism, not a group of people; claims attributed to named people are discussed as claims, without judgement of the person. The figures and tables do not rely on colour alone to carry meaning. The defence will follow the same conventions.

The cited literature reflects the field as it is, which is largely English-language and from European and North American institutions; the system's own limitations in the same direction

are stated in Section 8.3.

8.4 Alignment with the Sustainable Development Goals

The project relates most directly to one goal, and indirectly to two.

- **SDG 16, Peace, justice and strong institutions**, target 16.10: ensure public access to information. Verified, inspectable news on topics people can act on is a contribution to the quality of the information the public has access to; a system that reports what it could not verify, and why, contributes to it more honestly than one that does not.
- **SDG 9, Industry, innovation and infrastructure**: a verification pipeline that runs on open models and free software, on one modest machine, lowers the cost of a capability that is otherwise available only to large organisations.
- **The goals of the topics themselves** — health (SDG 3), climate (SDG 13), education (SDG 4) — are served only indirectly, by making constructive reporting on them easier to find and to check.

The contribution is modest and indirect: the system is a research prototype, its verifier is not yet accurate enough to be relied on, and its effect on the public's access to information has not been measured.

Chapter 9

Conclusions

9.1 Answers to the Research Questions

The answers below are provisional: they state what the measurements made so far support, and each ends with the measurement still needed.

RQ1 — Finding. *Yes, on the evidence so far.* A round scoped to one topic group read 0.41 times the sources and 0.10 times the links of an every-topic reading; its ranking separated stories worth offering from the rest with an AUC of 0.78 [0.67, 0.87]; and after the alignment of 9 October the first twenty places of five one-group rounds held no non-articles and 17 stories not worth offering out of 100, against 13 and 29 before. All of it rests on labels assigned by the AI labelling pass and on one or two days of feeds.

[To write: settle with the author's own labels on at least two rounds and discovery on five days.]

RQ2 — Verifying. *Not yet with the 3B model, on the only data measured.* On X-Fact the pipeline reached 29.7% accuracy [18.8, 40.6] on the stratified pilot and 19.4% [13.3, 25.5] on the Spanish test split, with a κ against gold indistinguishable from zero; retrieved evidence added 10.9 points [3.1, 20.3] over the model alone on the pilot and nothing measurable on the Spanish test split. How it can be quantified is the other half of the question, and that half is answered: by the harness's metrics with intervals, the evidence conditions, retrieval recall and the stage attribution. On the custom set's test half, with the default pipeline, it reached 52.9% [41.4, 64.3]: it caught 12 of 15 synthetic refuted claims, but on the natural claims agreed with the labels no better than chance ($\kappa = -0.01$), calling 18 of 46 supported claims false. A second, wider search for verdicts in doubt, tuned on the other half, changed nothing on the test half; the tuning loop placed most errors in the 3B model's reasoning rather than in the search. The same claims run twice gave the same verdict 13 times in 14.

[To write: the closed-book and oracle conditions on the test half, after the annotator's review of the AI-proposed labels.]

RQ3 — Person and model. *Not yet measured.* The comparisons are designed and their data are being produced: 108 labels proposed by the AI pass await the annotator's review, and the blind selection rounds have not been run. The system's own consistency is measured: 13 of 14 verdicts the same on a second run ($\kappa = 0.84$ [0.44, 1.00]), the one change coming from different evidence.

[To write: κ between the AI pass and the annotator; between the AI selection and the editor on blind rounds; between the pipeline and the annotator; and the annotator's self-agreement as the ceiling.]

RQ4 — The system. *Partly, and the person is still needed at every step.* The system takes on the reading of the sources and a first ordering of what it finds; about two in three of the claims it proposes are checkable (precision 0.64 [0.57, 0.70], lower in Spanish); and its verdicts, on the only data measured, are not yet reliable enough to be used without checking.

[To write: coverage, selective accuracy and the share of verdicts usable without re-checking on the custom set, and time per fact against time per claim.]

9.2 Achievement of the Objectives

[To write: update each status at the end of the project.]

9.3 Limitations

What the evaluation cannot show. The threats of Section 7.9 bound every result. In short: the custom set is small, labelled by one person with an AI pass still to be reviewed, drawn from the claims the system's own selector proposes, and its refuted class is synthetic; the external benchmark is political and years old; and no search can be repeated.

Known limitations of the system.

- **Worst claim wins.** One `UNVERIFIED` claim outweighs any number of `TRUE` ones in the article's verdict, and the article's confidence averages different verdicts.
- **Search is fragile.** Keyless search is refused by the engines behind it at times, and the second route was itself challenged from the development machine. Failures are reported, not prevented.
- **On-point but not on-subject.** The pertinence gate catches a page about something else; it does not catch a page about the same subject making a different claim. That judgement is the model's, enforced but unmeasured.
- **Ratings for 37 domains only.** Every other domain gets a neutral default, flagged as unrated.
- **Thresholds reasoned, not fitted.** The pertinence gate, the ranking and confidence weights and the claim selector's thresholds were set by reasoning, because no labelled set existed; tuning them on the tuning split is left for the end if time allows.

Table 9.1: The specific objectives of Section 1.4 as of 16 October 2026.

Objective	Status	What was done, and what is left
O1 Discovery and selection	Achieved	Discovery by topic group, the mission screen, the ranked round, the AI selection and the editor's choice of up to twenty, all recorded and measured on live rounds. Left: measurement against a person's labels (RQ1, RQ3).
O2 Extraction and enrichment	Achieved	A three-step extraction cascade, the enrichment stack in a separate model service, and a three-layer lake with lineage.
O3 Open-web fact-checking	Achieved	Seven stages with cited, quoted, recalibrated verdicts, failures reported apart, and a trace per claim; tested end to end and against a bluffing model. Its accuracy is the open question of RQ2.
O4 Validation set and harness	Partly	The harness, its metrics, retrieval evaluation and the X-Fact pilot are done. The custom set holds 139 facts (109 natural, 30 synthetic), but 108 labels await the annotator's review, the blind re-label has not been done, and stage attribution and the effort metrics are not yet part of the harness.
O5 Model benchmarks	Not yet	Both benchmarks are built and tested; neither has been run beyond the baseline on X-Fact. The writing benchmark was moved to "only if time is left".
O6 Reader view	Achieved	A public feed and article view with each claim's verdict, evidence and the reason each source was trusted.
Beyond the objectives	Built, not evaluated	Writing, reviewing, rewriting and publishing a new article from an analysed one (Section 5.10); the only model measured as writer, `llama3.2:3b`, is unfit for it.

- **Housekeeping.** Jobs are kept in memory without eviction; the analysis cache never expires and also caches rejections, although verdicts depend on today's web; and URLs are compared as plain strings, so tracking parameters make the same article look new.
- **Never deployed.** The cloud infrastructure has only been applied to an emulator, and the scraper does not read `robots.txt`.

9.4 Future Work

9.4.1 An API-Keyed Search Engine

The problem. Every piece of web evidence the system retrieves comes from search engines queried without API keys: a self-hosted SearXNG instance, which scrapes public engines from one IP address, and, when it cannot answer, DuckDuckGo's HTML results page queried directly. Keyless search keeps the system free of paid services and of credentials, but every keyless route

can be refused by the engine behind it, at any time and for any IP. On 25 September 2026, 68 of 69 queries sent through SearXNG during a measurement came back empty because the upstream engines had rate-limited or challenged the instance, and 23 of 24 claims were left without any web evidence.

What the system does today. Two measures address that failure without an API key, both introduced on 30 September 2026. First, a failed search is reported as such: when every query planned for a claim fails, the claim is marked `searchUnavailable` and its trace stops at the evidence retrieval stage, so the evaluation reports those claims apart from genuine `UNVERIFIED` verdicts. Second, a query SearXNG cannot answer is sent once to DuckDuckGo directly, and a claim is marked `searchUnavailable` only when both routes fail. The second route has a known limit: on the day it was introduced, DuckDuckGo answered every keyless request from the development machine with a human-verification challenge (three of three requests, through both of its HTML interfaces). The fallback recognises the challenge, reports the search as unavailable and stops asking for a few minutes; it recovers evidence only when DuckDuckGo lets the requesting IP through. Both keyless routes therefore share a single weakness: an engine that does not want automated traffic can refuse it.

Proposal. A search engine accessed with an API key, such as the Brave Search API, is rate-limited per key rather than per scraping IP, and answers automated requests by design. Brave also maintains its own index, so its results are not copies of the Google or Bing results that SearXNG already scrapes, which matters for a system that counts independent sources as corroboration. It would be added as one more route in the chain that already exists:

- **Position in the chain.** Asked only when the keyless routes cannot answer, so that normal operation stays free of cost; or, if the evaluation shows the keyless routes fail too often under load, asked first for benchmark runs only.
- **Same shape of result.** Its results are mapped to the same evidence records (URL, title, snippet) and tagged with their route in the existing `engines` field, so the pertinence gate, ranking and verification treat them identically and the evaluation can still tell which route produced each source.
- **The key as configuration.** The key is read from the environment, never committed, and the route is disabled when no key is set, so the system keeps working, keyless, without one.
- **Cost accounted for.** Every keyed query is counted per run, so the cost of a benchmark is reported beside its results.

Why it is not adopted now. An API key introduces a per-query cost under the provider's terms at the time, a credential that every deployment must manage, and a dependence on a commercial service whose terms and quotas may change — which would also weaken the reproducibility of the reported results, since a later reader could not re-run them without the

same service. The system therefore stays keyless, and the keyed route is proposed together with the measurements that would justify it:

Table 9.2: Measurements required before adopting an API-keyed engine.

Measurement	Question it answers
Share of `searchUnavailable` claims	How often do the keyless routes fail?
Evidence recovered by the keyed route	How much of that does a key recover?
Accuracy on the recovered claims	Is the recovered evidence useful?
Sources found by no other route	Does its own index add independence?
Cost per benchmark run	What does the recovery cost?

The first row is available as soon as the evaluation harness runs over the labelled claims; the others require one run with the keyed route and one without, over the same claims.

9.4.2 Further Work

Evaluating the writer. The writing flow is built (Section 5.10) but has been measured only on a handful of drafts with a model shown to be unfit for it. Its evaluation would take a larger, openly licensed writer model with a context of at least 16k tokens; extract each draft's claims with the pipeline's own extractor and verify each against the draft's own sources only, with no web search; and score the share of supported claims [27] and the precision and recall of the citations [11], validated by a hand check. The pipeline's verifier would become the evaluator of its own writer — validated, not assumed. Publishing to social networks would first need a stage that turns an article into media.

Evidence against the age of a claim. Re-running retrieval on the same recent claims after a week and after a month, to put a number on how much evidence a claim gains as it ages — the question that benchmarks of fresh knowledge ask of language models [40] and that the custom set's Rule 1 meets every day, when a new study has only its own press release.

The rest of the product. The recommendation system over the graph, with the verdict as one of its signals, and the mobile application of the wider project (Section 2.4).

Operations. The first real cloud deployment, with a search check from a data-centre address; reading `robots.txt`; evicting finished jobs; expiring the analysis cache; normalising URLs before comparing them; and the graph measured at scale beyond a few thousand articles.

Quality. End-to-end tests of the frontend; a policy for the three bluffs that still pass the guardrails; separating "how much could be checked" from "what was found" in the article's verdict; and the topic classifier, measured on the reviewed set and improved if it matters.

9.5 Final Remarks

[To write: personal assessment of the project, in the author's own words: only the author can write it.]

Bibliography

[1] Ron Artstein and Massimo Poesio. Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4):555–596, 2008. doi: 10.1162/coli.07-034-R2.

[2] Isabelle Augenstein, Christina Lioma, Dongsheng Wang, Lucas Chaves Lima, Casper Hansen, Christian Hansen, and Jakob Grue Simonsen. MultiFC: A real-world multi-domain dataset for evidence-based fact checking of claims. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019. Association for Computational Linguistics.

[3] Adrien Barbaresi. Trafilatura: A web scraping library and command-line tool for text discovery and extraction. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, pages 122–131. Association for Computational Linguistics, 2021.

[4] Francesco Barbieri, Luis Espinosa Anke, and Jose Camacho-Collados. XLM-T: Multilingual language models in Twitter for sentiment analysis and beyond. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference (LREC 2022)*, Marseille, France, 2022. European Language Resources Association.

[5] Ricardo Campos, Vítor Mangaravite, Arian Pasquali, Alípio Jorge, Célia Nunes, and Adam Jatowt. YAKE! keyword extraction from single documents using multiple local features. *Information Sciences*, 509:257–289, 2020.

[6] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024. URL `https://arxiv.org/abs/2402.03216`. Published in Findings of ACL 2024.

[7] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960. doi: 10.1177/001316446002000104.

[8] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi: 10.1145/1571941.1572114.

[9] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993.

[10] Ahmed El-Kishky, Thomas Markovich, Serim Park, Chetan Verma, Baekjin Kim, Ramy Eskander, Yury Malkov, Frank Portman, Sofía Samaniego, Ying Xiao, and Aria Haghighi. TwHIN: Embedding the Twitter heterogeneous information network for personalized recommendation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.

[11] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Singapore, 2023. Association for Computational Linguistics.

[12] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017.

[13] Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120 (30):e2305016120, 2023. doi: 10.1073/pnas.2305016120.

[14] Max Glockner, Yufang Hou, and Iryna Gurevych. Missing counter-evidence renders NLP fact-checking unrealistic for misinformation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Abu Dhabi, United Arab Emirates, 2022. Association for Computational Linguistics.

[15] Lucas Graves and Federica Cherubini. The rise of fact-checking sites in Europe. Digital news project report, Reuters Institute for the Study of Journalism, University of Oxford, 2016.

[16] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330, 2017.

[17] Zhijiang Guo, Michael Schlichtkrull, and Andreas Vlachos. A survey on automated fact-checking. *Transactions of the Association for Computational Linguistics*, 10:178–206, 2022. doi: 10.1162/tacl_a_00454.

[18] Ashim Gupta and Vivek Srikumar. X-Fact: A new benchmark dataset for multilingual fact checking. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 675–682, Online, 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-short.86. URL `https://aclanthology.org/2021.acl-short.86`.

[19] Naeemul Hassan, Fatma Arslan, Chengkai Li, and Mark Tremayne. Toward automated fact-checking: Detecting check-worthy factual claims by ClaimBuster. In *Proceedings*

of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pages 1803–1812, 2017. doi: 10.1145/3097983.3098131.

[20] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418.

[21] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023. doi: 10.1145/3571730.

[22] Mozhgan Karimi, Dietmar Jannach, and Michael Jugovac. News recommender systems – survey and roads ahead. *Information Processing & Management*, 54(6):1203–1227, 2018.

[23] J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977. doi: 10.2307/2529310.

[24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[25] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Singapore, 2023. Association for Computational Linguistics.

[26] Karen McIntyre and Cathrine Gyldensted. Constructive journalism: An introduction and practical guide for applying positive psychology techniques to news production. *The Journal of Media Innovations*, 4(2), 2017.

[27] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Singapore, 2023. Association for Computational Linguistics.

[28] Preslav Nakov, David Corney, Maram Hasanain, Firoj Alam, Tamer Elsayed, Alberto Barrón-Cedeño, Paolo Papotti, Shaden Shaar, and Giovanni Da San Martino. Automated fact-checking for assisting human fact-checkers. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21), Survey Track*, 2021.

[29] Liangming Pan, Xiaobao Wu, Xinyuan Lu, Anh Tuan Luu, William Yang Wang, Min-Yen Kan, and Preslav Nakov. Fact-checking complex claims with program-guided reasoning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Toronto, Canada, 2023. Association for Computational Linguistics.

[30] Eli Pariser. *The Filter Bubble: What the Internet Is Hiding from You.* Penguin Press, New York, 2011.

[31] Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019. Association for Computational Linguistics.

[32] Venu Satuluri, Yao Wu, Xun Zheng, Yilei Qian, Brian Wichers, Qieyun Dai, Gui Ming Tang, Jerry Jiang, and Jimmy Lin. SimClusters: Community-based representations for heterogeneous recommendations at Twitter. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020.

[33] Michael Schlichtkrull, Zhijiang Guo, and Andreas Vlachos. AVeriTeC: A dataset for real-world claim verification with evidence from the web. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. URL `https://arxiv.org/abs/2305.13117`. arXiv:2305.13117.

[34] Tal Schuster, Darsh Shah, Yun Jie Serene Yeo, Daniel Roberto Filizzola Ortiz, Enrico Santus, and Regina Barzilay. Towards debiasing fact verification models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3419–3425, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1341. URL `https://aclanthology.org/D19-1341`.

[35] Konstantinos Sechidis, Grigorios Tsoumakas, and Ioannis Vlahavas. On the stratification of multi-label data. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2011)*, volume 6913 of *Lecture Notes in Computer Science*, pages 145–158. Springer, 2011.

[36] Stuart Soroka and Stephen McAdams. News, politics, and negativity. *Political Communication*, 32(1):1–22, 2015. doi: 10.1080/10584609.2014.881942.

[37] James Thorne and Andreas Vlachos. Automated fact checking: Task formulations, methods and future directions. In *Proceedings of the 27th International Conference on Computational Linguistics (COLING)*, pages 3346–3359, Santa Fe, New Mexico, USA, 2018. Association for Computational Linguistics.

[38] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: a large-scale dataset for fact extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 809–819, New Orleans, Louisiana, 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-1074. URL `https://arxiv.org/abs/1803.05355`. arXiv:1803.05355.

[39] Soroush Vosoughi, Deb Roy, and Sinan Aral. The spread of true and false news online. *Science*, 359(6380):1146–1151, 2018. doi: 10.1126/science.aap9559.

[40] Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, Jerry Wei, Jason Wei, Chris Tar, Yun-Hsuan Sung, Denny Zhou, Quoc Le, and Thang Luong. FreshLLMs: Refreshing large language models with search engine augmentation. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2024.

[41] David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7534–7550, Online, 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.609. URL `https://aclanthology.org/2020.emnlp-main.609`.

[42] William Yang Wang. Liar, Liar Pants on Fire: A new benchmark dataset for fake news detection. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 422–426, Vancouver, Canada, 2017. Association for Computational Linguistics.

[43] Zhuohan Xie, Rui Xing, Yuxia Wang, Jiahui Geng, Hasan Iqbal, Dhruv Sahnan, Iryna Gurevych, and Preslav Nakov. FIRE: Fact-checking with iterative retrieval and verification. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025. URL `https://arxiv.org/abs/2411.00784`. arXiv:2411.00784.

[44] Urchade Zaratiana, Nadi Tomeh, Pierre Holat, and Thierry Charnois. GLiNER: Generalist model for named entity recognition using bidirectional transformer. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Mexico City, Mexico, 2024. Association for Computational Linguistics.

Appendix A

Detailed Planning

Table A.1: Planned hours per week from 29 September 2026 (docs/roadmap.md).

Week	Development	Evaluation (Phase 4)	Model testing (Phase 5)	Testing	Total
Sep 29 – Oct 5	Sprint 4: 17	labelling 1–50: 11	–	2	30
Oct 6 – 12	Sprint 4: 17	labelling 51–100: 11, research questions: 3	–	2	33
Oct 13 – 19	Sprint 4: 16	labelling 101–150: 12	–	2	30
Oct 20 – 26	infrastructure: 10	harness: 12, re-label: 1	–	2	25
Oct 27 – Nov 2	infrastructure: 10	harness: 12, re-label, kappa, join: 5	–	3	30
Nov 3 – 9	infrastructure: 10	harness: 13	–	3	26
Nov 10 – 16	–	tuning: 10	Sprint 6: 10	3	23
Nov 17 – 23	–	–	Sprint 6: 10, Sprint 7: 18	3	31
Nov 24 – 30	–	–	Sprint 7: 18	3	21
Dec 1 – 10	–	–	Sprint 7: 24	10	34
Dec 11 – 17	interface: 35	–	–	–	35
Dec 18 – 24	–	–	–	final: 20	20
Dec 25 – 31	report: 35	–	–	–	35

Table A.2 lists the tasks of every sprint and their status on 16 October 2026, from `docs/roadmap.md`, where each item is checked against the code rather than the plan. Done means done and verified; partly done says what is left.

Table A.2: Tasks per sprint and their status on 16 October 2026.

Sprint	**Tasks**	**Status**
0.1 Setup and scraping core	Monorepo; FastAPI and settings; trafilatura extraction; RSS discovery; declarative YAML sources; compose with Neo4j, SearXNG, inference and backend	Done
0.2 Enrichment	Keywords and entities; claim extraction in English and Spanish; topics, sentiment and quality; embeddings; the enriched article model	Done
0.3 Fact-checking v1	Admission validators; claim selector; SearXNG and vector retrieval; evidence ranker; language-model verifier and confidence scorer	Done

Sprint	Tasks	Status
0.4 Job API and frontend v1	Synchronous and background analysis with polling; the text corrector; the first frontend; the lazy container and its lock fix; test-collection fixes	Done
1 Stabilise the core	Pipeline edge cases; progress events for every stage; the analysis cache; shared fakes and their contracts; failing tests fixed	Done
2 Testing and bug-fixing	Full regression; concurrency tests; load test of the synchronous endpoint; model-client failure handling; architecture documentation; the 21 September audit and its fixes	Done
3 Scraping strategies	Ingestion path; headless-browser and BeautifulSoup steps; routing by `requires_javascript`; 12 to 36 sources; strategy tests; topic-page discovery and the source probe	Done
4 Enrichment and fact-checking	Search fix with a "search unavailable" state and a keyless second route; alternative retrieval strategies; ranking and confidence regression tests; discovery by topic group; choice of up to twenty; the newsroom flow; the mission screen; the candidate ranking; alignment with the groups; topic emphasis; time to reception	Done (topic classifier descoped)
5 Neo4j and infrastructure	Graph schema, write path, read use-case and live tests; hosting decision; production compose; SearXNG for production; logging and monitoring; Terraform checked against an emulator; frontend on the VM behind Caddy	Partly: the first real deployment and a domain are left
Validation set	Daily batches; facts 1–150; blind 20% re-label; agreement and join	Partly: 139 facts, 108 labels to review, re-label not done
Evaluation harness	Research questions; harness; metrics; retrieval evaluation; stage attribution; journalist-effort metrics; X-Fact pilot	Partly: attribution and effort not in the harness
Experiments before the verdict	Tools; discovery yield; source health; selection blind and assisted; admission thresholds; topic classifier; time per stage; tables into the thesis	Partly: tools and first runs done
Tuning	Claim-selection thresholds; ranking, confidence and pertinence weights against the labels	Not started (P2)
6 Writing models	Local and hosted benchmark on the corrector's metrics; rubric and comparison table; cost and latency logging	Built, not run (P2)
7 Verification models	Verifier benchmark; agreement with the human labels; confidence re-tuning; default model chosen; results section	Built, not run

Sprint	Tasks	Status
8 Reader view	Topic feed; article view with verdicts, evidence and trust; design system; public access	Done
Testing and QA	Unit, concurrency and graph tests; full pass with inference; CI; full-pipeline integration test; harness tests; model comparison tests; guardrail stress test; smoke and load tests; final regression, bug-fix and deployment check; frontend end-to-end tests	Partly: December items left
9 Report	Methodology; benchmark results; delivery; wrap-up	In progress

Appendix B

News Source Catalogue

Table B.1 lists every configured source (Section 5.2.2) as of 16 October 2026. *Rel.* is the source's `reliability_index`; the groups are society (So), science (Sc), environment (En), culture (Cu) and health (He). A disabled source is not ingested, but its pages keep their rating when they are found as evidence.

Table B.1: The source catalogue.

Source	Lang.	Country	Type	Rel.	Groups	Status
20minutos	es	ES	news	0.78	So Sc He	enabled
ABC	es	ES	news	0.86	So Sc Cu	enabled
Agencia EFE	es	ES	news	0.95	So Sc	disabled[b]
BBC	en	UK	news	0.94	So Sc En He	enabled
Carbon Brief	en	UK	news	0.90	En	enabled
CNN	en	US	news	0.72	So Sc He	enabled
El Mundo	es	ES	news	0.87	So Sc He	enabled
El País	es	ES	news	0.90	So Sc En Cu	enabled
elDiario.es	es	ES	news	0.83	So En	enabled
Europa Press	es	ES	news	0.90	So Sc He	enabled
European Space Agency	en	EU	government	1.00	Sc	enabled
France 24 Español	es	FR	news	0.88	So He	enabled
Good News Network	en	US	blog	0.65	So En Cu He	enabled[p]
Infobae	es	AR	news	0.70	Sc He	enabled
La Vanguardia	es	ES	news	0.89	So Sc Cu	enabled
Maldita.es	es	ES	news	0.93	So He	disabled[a]
MIT News	en	US	academic	0.92	So Sc	enabled
Mongabay	en	US	news	0.85	En	enabled
Muy Interesante	es	ES	news	0.78	Sc Cu He	enabled
NASA	en	US	government	1.00	Sc	enabled
National Geographic	en	US	news	0.98	Sc En Cu	enabled[s]
Nature	en	UK	academic	0.98	Sc He	enabled
New Scientist	en	UK	news	0.88	Sc He	enabled
Newtral	es	ES	news	0.93	So He	disabled[a]
Positive News	en	UK	news	0.78	So En Cu	enabled[p]

Source	Lang.	Country	Type	Rel.	Groups	Status
Reasons to be Cheerful	en	US	news	0.80	So En He	enabled[p]
Reuters	en	UK	news	0.96	So	disabled[b]
RTVE Noticias	es	ES	news	0.93	So Sc Cu	enabled[s]
SINC	es	ES	news	0.99	Sc En He	enabled[s]
Smithsonian Magazine	en	US	news	0.90	Sc En Cu	enabled
The Conversation	en	AU	academic	0.90	So Sc He	enabled
The Guardian	en	UK	news	0.88	Sc En He	enabled
The Optimist Daily	en	US	news	0.70	So En He	enabled[o]
UN News	en	INT	government	0.92	So En He	enabled
World Health Organization	en	INT	government	0.97	He	enabled
Xataka Ciencia	es	ES	blog	0.75	Sc	enabled
Yale Environment 360	en	US	academic	0.90	En	enabled

[a] A fact-checker: rated as evidence, not ingested, because a fact-check quotes the claim it debunks.

[b] Refused every request from 23 September to 6 October 2026 (EFE 403, Reuters 401).

[p] Publishes only positive news (`positive_editorial`); the mission screen does not apply to it.

[s] Its feed is dead or stale; its articles are discovered through its topic section pages.

[o] Solutions journalism, added on 9 October 2026; not marked `positive_editorial`, because its feed also carries lifestyle tips, which the mission screen judges like anyone else's.

Appendix C

Annotation Guide

This is the full guide behind the custom validation set (Section 6.3), as the annotator and the AI labelling pass apply it.

C.1 What is labelled

One record holds one atomic, checkable claim, in the article's own wording. A proposed sentence that asserts two things is split into two records. A claim proposed by the daily batch is either labelled or skipped with one of these reasons:

opinion an opinion or interpretation, which no evidence can settle;

prediction a statement about the future;

trivial checkable but not worth checking;

fragment not a whole sentence (a heading, a caption, a sentence cut short);

duplicate the same claim as one already labelled;

other none of the above, with a note.

C.2 Labels

supported (`TRUE`) the evidence supports the claim as stated.

partially supported (`PARTIALLY_TRUE`) the central claim holds and a detail does not.

conflicting (`MISLEADING`) sources of equal standing conflict, or the claim is true but selected to mislead.

refuted (`FALSE`) the evidence refutes the claim.

not enough evidence (`UNVERIFIED`) there is not enough independent evidence to decide either way.

C.3 The four tie-break rules

Fixed before labelling began; the note of a record names the rule that decided a close call.

Rule 1. The organisation's own source. A claim whose only evidence is the communication of the organisation the story is about (a press release, report, website or post) is *not enough evidence*, and the source is named in the note. *Enforced on save.*

Rule 2. Numeric tolerance. A figure within ±10% of the source's, or a reasonable rounding of it, is supported. Beyond that, the claim is *partially supported* if the figure is a detail of an otherwise correct claim, and *refuted* if the figure is the claim.

Rule 3. Date of the evidence. Only evidence available on the article's publication date is admissible. A claim confirmed only later stays *not enough evidence*. *Enforced on save*: an evidence date after the article's is refused.

Rule 4. Hierarchy of sources. Primary source (report, official statistic, peer-reviewed study) > reference media outlet > press release > social media. The higher tier prevails; sources of the same tier that conflict with none decisive make the claim *conflicting*. The strongest tier used is recorded.

Also enforced on save: any label other than *not enough evidence* needs at least one evidence link.

C.4 Procedure

1. Take the claim as proposed, or split it; keep the article's wording.
2. Search for evidence independently of the system; the system's verdict is never shown.
3. Record the evidence links, the date of the newest evidence used and the tier of the strongest.
4. Apply the rules and choose the label; record the claimant, the claim type (factual, numerical or interpretive) and the topic.
5. Explain any close call in the note, naming the rule.

C.5 Synthetic refuted claims

Only a natural claim labelled *supported*, whose note states the value the evidence gives, is mutated, and each source only once. One element of what the claim asserts is substituted — a number, a date, an entity or a relation — never a detail, and never by negation. Figures that must agree are changed together. The substitute is plausible without the evidence. Everything else is kept: the article, date, topic, claim type, tier and evidence links, with the source's identifier and the kind of substitution recorded. A mutation whose source is later relabelled is listed for review.

C.6 AI-assisted labels

A label proposed by the AI labelling pass starts its note with "[AI label: Claude, *date* — not yet checked by the human annotator]". It follows this guide and cites the evidence it relied on. It does not count as gold until the annotator has reviewed it and kept or changed it; the agreement between the two is reported.

C.7 The blind review

20% of the records, the first in the order of the SHA-256 hash of their identifiers, are labelled again at least a week after the first pass, without the first label, the own-source flag or the note. Agreement and Cohen's κ are computed on the first label; $\kappa \geq 0.6$ is the threshold for the guide to be considered consistently applicable, and below it the rules behind the disagreements are revised.

Appendix D

Reproducibility

Everything in this thesis is produced from one repository, by the commands below. Every report states the commits its runs were made on.

D.1 Verifying the system

```
./scripts/check.sh           # backend + inference + frontend + labeller
./scripts/check.sh fast      # the invariants only, in seconds
./scripts/check.sh inference # the real models
./scripts/check.sh slow      # the whole stored corpus
./scripts/check.sh graph     # the graph against a real Neo4j
./scripts/check.sh gcp       # the cloud infrastructure against Floci
./scripts/check.sh labeller  # the labelling tool
```

Backend tests that need a real model skip when the inference service is not running; start it first (`docker compose up inference`, or `cd inference && uv run uvicorn src.main:app -port 8001`).

D.2 Running the stack

```
./scripts/dev.sh all     # compose services + the frontend dev server
```

The backend's configuration (`backend/.env`) is not in the repository; `backend/.env-example` lists every setting. On a CPU, use the production limits for the language model: one call at a time and a 180 s timeout.

D.3 Evaluation runs

Run inside the development stack with the evaluation layer, so that records carry their commit and the CPU limits apply:

```
cd docker
```

```
docker compose -f docker-compose.yml -f docker-compose.eval.yml \
  --env-file ../backend/.env run --rm --no-deps -T backend \
  python -m src.evaluation.cli run \
    --dataset data/evaluation/xfact_en_es_pilot.jsonl --model llama3.2:3b
# then, from backend/:
uv run python -m src.evaluation.cli report --run <run directory>
uv run python -m src.evaluation.cli table --run <dir> --run <dir>
uv run python -m src.evaluation.cli retrieval --run <dir> --run <dir>
```

Data read: `backend/data/evaluation/xfact_en_es_pilot.jsonl` (the pilot), `xfact_es_test.jsonl` (the Spanish test split), and the custom set, either the directory `manual/` or its merge `custom_en_es.jsonl` (`python labeller/app.py join`). The closed-book baseline is `scripts/closed_book_probe.py`; the time-elapsed analysis and the pilot's stage attribution are `scripts/xfact_time_elapsed.py`. Committed reports are under `backend/data/evaluation/reports/`.

D.4 Experiments before the verdict

```
cd backend
uv run python -m src.evaluation.experiments discovery --group environment
uv run python -m src.evaluation.experiments selection
uv run python -m src.evaluation.experiments blind-ai --round <round id>
uv run python -m src.evaluation.experiments admission --backend http
    ://127.0.0.1:8000
uv run python -m src.evaluation.experiments topics --backend http://127.0.0.1:8000
uv run python -m src.evaluation.experiments emphasis
```

Results, and the labelled data of the one-off studies (the mission screen's margin, the ranking's weights, the alignment of 9 October), are under `backend/data/evaluation/experiments/`.

D.5 Building this document

```
cd docs/final_document
latexmk -pdf main.tex      # pdflatex + bibtex
```

Set `\showpendingfalse` in `main.tex` to hide the remaining placeholders.